

BUILD YOUR OWN LANGUAGE MODEL IN SCALA

FROM ZERO TO A WORKING NEURAL NETWORK, NO EXPERIENCE
REQUIRED

Author: Rele Chiloane

Based on the open-source repository:
[chiloanerk/scala-neural-language-model](https://github.com/chiloanerk/scala-neural-language-model)

About This Book

This book teaches you how to build a language model (the kind of system at the heart of tools like ChatGPT) completely from scratch, in a programming language called Scala. No machine learning experience required. No advanced math required. No prior Scala experience required.

By the end, you will type `sbt "run chat"` into your terminal and have a real conversation with a neural network you built line by line with your own hands.

Who this is for: Anyone curious about how language models actually work, who wants to understand the real machinery rather than just use the tools.

What you'll learn: Programming fundamentals (via Scala), the mathematics of neural networks (through analogy and code), and how a complete ML system is designed and built.

Table of Contents

INTRODUCTION

- [Introduction: A Knowledgeable Friend](#)

PART 1: FOUNDATIONS: LEARNING TO THINK IN CODE

- [Chapter 1: Hello, Scala \(and Hello, Machine Learning\)](#)
- [Chapter 2: Turning Words Into Numbers](#)
- [Chapter 3: Context Windows: What the Model Gets to See](#)
- [Chapter 4: Vectors: The Language of Numbers](#)
- [Chapter 5: Matrices: Tables of Transformations](#)

PART 2: THE MACHINE: BUILDING A BRAIN CELL AT A TIME

- [Chapter 6: Embeddings: Words in Space](#)

- [Chapter 7: The Hidden Layer: Where the Magic Happens](#)
- [Chapter 8: The Output Layer and Softmax](#)
- [Chapter 9: Assembling the Forward Pass](#)
- [Chapter 10: Loss: Measuring How Wrong You Are](#)

PART 3: LEARNING: TEACHING THE MACHINE TO BE WRONG LESS

- [Chapter 11: Gradients: Which Way Is Downhill?](#)
- [Chapter 12: Backpropagation: Running the Chain Backwards](#)
- [Chapter 13: Updating Weights: SGD and the Training Step](#)
- [Chapter 14: The Training Loop: Epochs, Batches, and Progress](#)
- [Chapter 15: Saving and Loading: The Checkpoint](#)

PART 4: THE LIVING SYSTEM: FROM CODE TO CONVERSATION

- [Chapter 16: Evaluating What You Built](#)
- [Chapter 17: Generating Text: Temperature, Top-K, and Top-P](#)
- [Chapter 18: The CLI: Building the Conversation Interface](#)
- [Chapter 19: The Payoff: Run Chat with Your Own Language Model](#)

EXTENSION CHAPTERS (OPTIONAL)

- [Extension A: Speed Up Training with Apple Metal](#)
- [Extension B: Teaching Across Multiple Corpora: Replay Buffers and Continual Learning](#)
- [Extension C: Observability: Measuring What Your Model Really Does](#)

How to Use This Book

Each chapter ends with a **milestone**: something concrete you can run or print. Don't skip them. The milestones are where understanding becomes real.

The code in this book is drawn from a working Scala project. Every snippet you write is a piece of a real, running system. By Chapter 19, you'll have built the whole thing.

To run any code example, you'll need: - Java 21 or later - SBT (the Scala build tool) - A terminal

Chapter 1 walks you through installation.

Reference Implementation

The complete working project lives alongside this book. When a chapter says "here's the file we're building," you can always look at the finished version for reference. But try to build it yourself first: that's where the learning happens.

```
src/main/scala/  
├─ data/           ← Chapters 2-3  
├─ linalg/         ← Chapters 4-5  
├─ nn/             ← Chapters 6-10, 12-13  
├─ eval/           ← Chapter 10, 16  
├─ train/          ← Chapters 14-15  
├─ observability/ ← Extension C  
├─ compute/        ← Extension A  
└─ app/            ← Chapters 18-19
```

Introduction: A Knowledgeable Friend

There's a version of this book that starts with a diagram. Boxes connected by arrows. Labels like "embedding layer," "hidden state," "softmax distribution." The diagram is technically correct. It is also, if you've never built anything like this before, completely useless.

This is not that book.

This book starts with the assumption that you're curious and a little intimidated. Maybe you've heard the words "language model" and "neural network" enough times that you're starting to feel left out of a conversation. Maybe you've tried to read an explanation online

and bounced off a wall of Greek letters. Maybe you've wondered, genuinely, what's actually *inside* ChatGPT: not the corporate answer, but the real answer, the mathematical machinery underneath. If any of that sounds like you, then you're exactly the reader this book was written for.

What We're Going to Build

Here is the destination: by the end of this book, you will run a command in your terminal and have a conversation with a language model you built yourself.

Not a wrapper around someone else's model. Not an API call to a server in California. An actual neural network (weights, gradients, training loop, the whole thing) that you wrote, line by line, in a programming language called Scala.

It won't be GPT-4. We should be honest about that up front. GPT-4 has hundreds of billions of parameters; ours will have a few hundred thousand. GPT-4 was trained on most of the text ever written; ours will train on whatever text file you point it at. But here's the thing: it uses *exactly the same fundamental ideas*. The math is the same. The architecture is the same shape, just smaller. When you understand how ours works, you will genuinely understand how theirs works: not as a metaphor, but for real.

That understanding is what this book is actually selling. The working language model is just proof that you have it.

The Journey

We'll move through four stages.

First, foundations. We'll learn to write code in Scala: not all of Scala, just the parts we need. We'll learn how to turn text into numbers (which is the first thing any language model must do), and we'll build the mathematical building blocks: vectors and matrices.

Then, the machine. We'll build the neural network itself, one layer at a time. We'll start with embeddings (the model's way of representing words as points in space) and end with a

complete forward pass that turns a context of words into a probability distribution over what comes next.

Then, learning. This is the deep end of the pool, and we'll wade in carefully. We'll understand gradients, implement backpropagation, and watch a training loop run for the first time. This is the part where the machine goes from making random noise to making something that resembles language.

Finally, the living system. We'll add evaluation, text generation, a command-line interface, and the chat loop. We'll run the whole thing end to end.

A Note on Math

This book does not assume you know calculus. Or linear algebra. Or statistics.

It *will* introduce you to mathematical ideas (vectors, matrices, derivatives) because there's no honest way to explain neural networks without them. But every mathematical idea in this book will be explained the same way: with an analogy first, then with intuition, then with a formula, then with code. You'll never see a formula until you already understand what it's trying to say.

The honest truth is that the math in a neural network, while it looks intimidating, is mostly multiplication and addition. The hard part isn't the arithmetic: it's understanding *what* is being multiplied and *why*. That's what this book is for.

A Note on Scala

Scala is the programming language we'll use. You don't need to know it yet.

We picked Scala for a few reasons. It's expressive: the code tends to look a lot like the mathematical ideas it implements, which makes everything easier to understand. It's strongly typed, which means the compiler catches mistakes early (you'll appreciate this when we're doing matrix math). And frankly, it's a pleasure to write once you get the hang of it.

We'll introduce Scala features as we need them, in the order we need them. Chapter 1 covers the basics. The rest of the language will reveal itself naturally as the project requires it.

What You'll Need

- A computer (Mac, Linux, or Windows)
- Java 21 or later (Chapter 1 walks you through this)
- SBT, the Scala build tool (Chapter 1 covers this too)
- A text editor or IDE
- Curiosity and patience

That's it. No cloud account, no GPU, no paid subscription to anything. Everything in this book runs on your laptop.

How to Read This Book

Read it in order, at least the first time. Each chapter builds on the last. The earlier chapters in particular lay groundwork that everything else depends on.

Each chapter ends with a **milestone**: something concrete you can run and inspect. Don't skip the milestones. There's a big difference between reading "the dot product measures similarity" and actually computing one and seeing the result. The milestones are where abstract ideas become real.

Some chapters have **side notes**: short tangents marked off from the main text. These are for the reader who wants to know *more*. They're not required to understand the next chapter, but they're rewarding if you're curious.

One more thing. When you hit a chapter that feels hard (and Chapter 11, on gradients, might be that chapter for you) don't panic. Read it slowly. Come back to it the next day. The ideas are not intrinsically difficult; they're just unfamiliar. Familiarity is just exposure over time. Give it that time.

Let's Begin

There's a small program at the heart of everything we're going to build. It reads a word, and it guesses what word comes next. That's the entire job description of a language model: distilled to its essence.

By the time you've finished this book, that simple description will have expanded in your mind to include: how words become numbers, how numbers move through layers, how layers learn from mistakes, and how mistakes become understanding. You'll have watched a machine go from outputting random gibberish to producing something that feels, occasionally, like language.

That journey starts in the next chapter, with a blank Scala prompt and a blinking cursor.

Let's go.

Chapter 1: Hello, Scala (and Hello, Machine Learning)

Let's start with something honest: the phrase "language model" sounds more complicated than it needs to.

Strip away the jargon, and a language model is a program that reads some words and guesses what word comes next. That's it. The autocomplete on your phone is a language model. The search bar that finishes your sentence is a language model. ChatGPT is a language model: a very large, very expensive one, but the same basic idea.

We are going to build one from scratch.

"From scratch" means we won't use any machine learning libraries. No TensorFlow. No PyTorch. No magic black boxes. We'll write every piece ourselves, in a programming language called Scala, starting from the simplest building blocks. By the time we're done, you'll be able to look at your code and say: *I understand why every line is there.*

That understanding is the point. Let's begin.

What Is a Program?

Before we can write a language model, we need to know how to write a program. Let's start there.

A program is a set of instructions that a computer follows, in order. The computer is very literal: it does exactly what you tell it, nothing more, nothing less. This is both the most frustrating thing about programming (your bugs are always your fault) and the most liberating (the machine is predictable; it won't surprise you once you understand it).

Programs work on *data*. They take data in, do something to it, and put data out. A word processor takes keystrokes and produces a document. A navigation app takes your location and produces directions. Our language model will take words and produce a prediction.

In Scala, data comes in many shapes. Numbers. Text. Lists. Maps from one thing to another. We'll meet all of these. But first, let's meet Scala itself.

Installing Scala

To run Scala code, you need two things:

Java, which is the platform Scala runs on. Scala programs compile to Java bytecode and run on the Java Virtual Machine (JVM). You need Java 21 or later.

SBT, which stands for Scala Build Tool. SBT handles compiling your code, running your program, and managing dependencies.

INSTALLING JAVA

Check if you already have Java installed:

```
java -version
```

If you see something like `openjdk version "21.0.x"` or higher, you're good. If not:

- **Mac:** Install via Homebrew: `brew install openjdk@21`
- **Linux:** Use your package manager: `apt install openjdk-21-jdk` (Ubuntu/Debian)
- **Windows:** Download from adoptium.net and run the installer

INSTALLING SBT

- **Mac:** `brew install sbt`
- **Linux:** Follow the instructions at scala-sbt.org/download.html
- **Windows:** Download the `.msi` installer from the same page

Verify both are installed:

```
java -version
sbt --version
```

If both print version numbers, you're ready.

Your First Scala Session: The REPL

Scala has something called a REPL: a Read-Eval-Print Loop. It's an interactive prompt where you type a Scala expression, press Enter, and the computer evaluates it and shows you the result. It's perfect for exploring ideas without writing a full program.

Start the Scala REPL by typing `scala` in your terminal. (If `scala` isn't found, you can use `sbt console` from any directory instead: it starts the same thing.)

You'll see something like:

```
Welcome to Scala 3.3.3
Type in expressions for evaluation. Or try :help.

scala>
```

That `scala>` is the prompt, waiting for you. Let's type something:

```
scala> 1 + 1  
val res0: Int = 2
```

The REPL printed `val res0: Int = 2`. Let's read that: - `val` means "this is a value" (more on this in a moment) - `res0` is a name the REPL automatically gave our result (short for "result 0") - `Int` is the *type*: this is a whole number - `2` is the actual result

Try a few more:

```
scala> 10 * 3  
val res1: Int = 30  
  
scala> 100.0 / 7.0  
val res2: Double = 14.285714285714286  
  
scala> "hello"  
val res3: String = hello
```

Notice that `10 * 3` gives an `Int` (a whole number), but `100.0 / 7.0` gives a `Double` (a decimal number). The `.0` tells Scala we're working with decimals. And `"hello"` (text in double quotes) gives a `String`.

Values and Types

In Scala, data has a *type*. The type tells you what kind of thing it is and what you can do with it. This is one of Scala's most useful features: the language checks types for you, catching a whole class of mistakes before your code ever runs.

Here are the types you'll use most:

TYPE	WHAT IT IS	EXAMPLE
<code>Int</code>	A whole number	<code>42</code> , <code>-7</code> , <code>0</code>
<code>Double</code>	A decimal number	<code>3.14</code> , <code>-0.5</code> , <code>100.0</code>
<code>Boolean</code>	True or false	<code>true</code> , <code>false</code>
<code>String</code>	Text	<code>"hello"</code> , <code>"the cat"</code>

You can give a value a name using `val` :

```
scala> val x = 5
val x: Int = 5

scala> val greeting = "hello"
val greeting: String = hello

scala> val pi = 3.14159
val pi: Double = 3.14159
```

Now you can use those names:

```
scala> x * 2
val res4: Int = 10

scala> greeting + " world"
val res5: String = hello world
```

`val` means the value is *immutable*: once you set it, you can't change it. This is a design choice in Scala that we'll appreciate later: immutable values are easier to reason about, especially when code gets complex.

Side note: `val` vs `var` : Scala also has `var` , which creates a mutable variable you can reassign. We'll almost never use `var` in this book. Immutability keeps our code cleaner and easier to understand.

Functions

A function is a named transformation. It takes input, does something, and produces output.

In Scala, you define a function with `def` :

```
scala> def double(n: Int): Int = n * 2
def double(n: Int): Int

scala> double(5)
val res6: Int = 10

scala> double(21)
val res7: Int = 42
```

Let's read the definition: `def double(n: Int): Int = n * 2` - `def` means we're defining a function - `double` is the name - `(n: Int)` says the function takes one argument, named `n`, of type `Int` - `: Int` after the closing parenthesis is the *return type*: what the function produces - `= n * 2` is the body: what the function does

Functions can take multiple arguments:

```
scala> def add(a: Double, b: Double): Double = a + b
def add(a: Double, b: Double): Double

scala> add(3.0, 4.5)
val res8: Double = 7.5
```

Strings and Text Operations

Our language model works with text, so let's get comfortable with Strings.

A `String` is just a sequence of characters enclosed in double quotes. You can concatenate strings with `+` :

```
scala> "hello" + " " + "world"
val res9: String = hello world
```

Strings in Scala have a method called `.split`, which breaks a string into pieces wherever a delimiter appears:

```
scala> "the cat sat".split(" ")
val res10: Array[String] = Array(the, cat, sat)
```

The result is an `Array[String]`: an ordered collection of strings. But in this book, we'll prefer a close cousin called `Vector`. Call `.toVector` to convert:

```
scala> "the cat sat".split(" ").toVector
val res11: Vector[String] = Vector(the, cat, sat)
```

You can index into a `Vector` with parentheses:

```
scala> val words = "the cat sat".split(" ").toVector
val words: Vector[String] = Vector(the, cat, sat)

scala> words(0)
val res12: String = the

scala> words(2)
val res13: String = sat
```

Note: Scala (like most programming languages) counts from 0. The first element is at index 0, the second at index 1, and so on.

The `Vector`: Our Workhorse Collection

`Vector` is an immutable, ordered collection. Think of it like a numbered list. We'll use it constantly.

Create a `Vector` directly with `Vector(...)`:

```
scala> val numbers = Vector(10, 20, 30, 40)
val numbers: Vector[Int] = Vector(10, 20, 30, 40)

scala> numbers(1)
val res14: Int = 20

scala> numbers.length
val res15: Int = 4
```

One of the most useful things you can do with a Vector is `.map`: apply a function to every element:

```
scala> numbers.map(n => n * 2)
val res16: Vector[Int] = Vector(20, 40, 60, 80)
```

`n => n * 2` is an *anonymous function*: a function without a name. Read it as "given `n`, return `n * 2`." We'll use this pattern constantly.

You can also filter a Vector, keeping only elements that satisfy a condition:

```
scala> numbers.filter(n => n > 15)
val res17: Vector[Int] = Vector(20, 30, 40)
```

Why All This Matters: The First Glimpse

Here's a preview of why we're learning this. Our language model needs to:

1. Take a sentence like `"the cat sat on the mat"`
2. Split it into words: `Vector("the", "cat", "sat", "on", "the", "mat")`
3. Assign each unique word a number: `"the" → 0`, `"cat" → 1`, `"sat" → 2`, etc.
4. Convert the sentence to numbers: `Vector(0, 1, 2, 3, 0, 4)`
5. Create training examples: given `[0, 1, 2] → predict 3`, given `[1, 2, 3] → predict 0`, etc.

Steps 2–5 are all string manipulation and Vector operations: exactly what we've been learning. Chapter 2 will build the real version of this pipeline. But you've already seen every operation it needs.

Chapter Milestone

Let's end this chapter with a hands-on session that brings together everything you've learned.

Open your Scala REPL and try these. Type them yourself rather than copy-pasting: the act of typing builds familiarity.

```
// Step 1: Split a sentence into words
val sentence = "the quick brown fox jumps over the lazy dog"
val words = sentence.split(" ").toVector
// → Vector(the, quick, brown, fox, jumps, over, the, lazy, dog)

// Step 2: How many unique words?
val uniqueWords = words.distinct
// → Vector(the, quick, brown, fox, jumps, over, lazy, dog)
uniqueWords.length
// → 8

// Step 3: Map each word to its position in the unique list
val wordToId = uniqueWords.zipWithIndex.toMap
// → Map(the -> 0, quick -> 1, brown -> 2, fox -> 3, jumps -> 4, over -> 5, lazy -> 6,
dog -> 7)

// Step 4: Convert the sentence to IDs
val ids = words.map(w => wordToId(w))
// → Vector(0, 1, 2, 3, 4, 5, 0, 6, 7)
```

Run each line and observe the output. Notice in step 4 that "the" appears twice in the sentence (at positions 0 and 6), and both occurrences map to the same ID (0). That's exactly what we want: words are represented by their identity, not their position.

This is the germ of a language model's data pipeline. We've already written a prototype.

What You Learned

- Scala is a language where data has a *type*, and the compiler checks those types for you
 - `val` creates an immutable named value
 - `Int`, `Double`, `String`, `Boolean` are the basic types
 - `Vector` is an immutable ordered collection: our go-to data structure
 - `.map` and `.filter` transform collections element by element
 - `.split`, `.toVector`, `.distinct`, `.zipWithIndex`, `.toMap` are the string and collection tools we'll use to process text
-

Up Next

In Chapter 2, we'll build the real text pipeline. We'll write the code that takes any corpus of text, tokenizes it properly, builds a vocabulary, and converts tokens to integer IDs. That code will live in `data/TextPipeline.scala`, and it will be the foundation that every other module in the project depends on.

Chapter 2: Turning Words Into Numbers

Here's a problem that looks simple until you think about it.

Neural networks are mathematical machines. They take numbers as input, perform arithmetic on those numbers, and produce numbers as output. But language isn't numbers: it's words, sentences, meaning. How do you feed a sentence to a machine that can only do arithmetic?

The answer is: you turn the sentence into numbers first. This is called *tokenization*, and it's the very first step in any language model pipeline. Get this wrong and everything else falls apart. Get it right and you have a foundation you can build on.

In this chapter, we'll build a complete text tokenization pipeline: from raw text to a clean sequence of integer IDs. The code we write here will live in `data/TextPipeline.scala`, and every other module in the project will depend on it.

The Basic Idea

Imagine you're building a language model to predict the next word in a sentence. Your training data is a text file. Something like:

```
the cat sat on the mat
the cat ate the rat
```

Before the model can learn anything, you need to:

1. **Decide what a "token" is.** Usually a token is a word, but it could be a sub-word, a character, or even a byte. We'll use words.
2. **Build a vocabulary:** a list of all the distinct tokens. Each token gets a unique integer ID.
3. **Convert the text to IDs:** turn every token into its integer.
4. **Create training examples:** pairs of (context, target) where context is a sequence of recent token IDs and target is the next token ID.

That's the whole pipeline. Let's build it.

Case Classes: Giving Names to Shapes of Data

Before we write functions, let's define the data structures we'll work with. In Scala, `case class` is the tool for this.

A `case class` is a named bundle of data. Think of it as a custom type you define:

```
case class Example(context: Vector[Int], target: Int)
```

This defines a new type called `Example` that holds two things: - `context` : a `Vector[Int]` : a sequence of token IDs representing the recent history - `target` : an `Int` : the ID of the next token (what we want to predict)

To create an `Example` :

```
val ex = Example(context = Vector(1, 3, 7), target = 5)
```

And to read from it:

```
ex.context // -> Vector(1, 3, 7)
ex.target  // -> 5
```

Case classes give us a clean way to bundle related data together. They also come with useful built-in features: equality checking, a readable `toString` , and a `copy` method for creating modified versions. We'll use all of these.

Our vocabulary also gets its own case class:

```
case class Vocab(
  tokenToId: Map[String, Int],
  idToToken: Vector[String],
  unkToken: String = "<UNK>"
)
```

This holds: - `tokenToId` : a Map from token string to its integer ID. For example: `"cat" -> 2` . - `idToToken` : a Vector where the index is the ID. `idToToken(2)` gives `"cat"` . - `unkToken` : the special token we'll use for words we haven't seen before (UNK = "unknown").

The two fields `tokenToId` and `idToToken` are inverses of each other. One translates forwards, one backwards.

Side note (`Map` in Scala: A `Map[String, Int]` is a key-value store. Given a key (a `String`), you can look up its value (an `Int`). Think of it like a dictionary in the traditional sense: given a word, you look up its definition) except here, "definition" means "integer ID." You create one with `Map("cat" -> 2, "dog" -> 3)` , and look up a value with `myMap("cat")` .

Tokenization: From Text to Words

Our first task is splitting raw text into tokens. We want to: - Lowercase everything (so "Cat" and "cat" are the same token) - Split on non-word characters (spaces, punctuation, newlines) - Keep alphanumerics and apostrophes (so "don't" stays as one token)

Here's the tokenizer:

```
object TextPipeline:
  private val splitRegex = "[^a-z0-9']+".r

  def tokenize(text: String): Vector[String] =
    splitRegex
      .split(text.toLowerCase)
      .toVector
      .map(_.trim)
      .filter(_.nonEmpty)
```

Let's read this function carefully.

`text.toLowerCase` converts all uppercase letters to lowercase. "The Cat" becomes "the cat".

`splitRegex.split(...)` splits the text on any sequence of characters that matches the pattern `[^a-z0-9']+`. The `[^...]` notation means "any character NOT in this set." So we split on anything that isn't a lowercase letter, digit, or apostrophe.

`.toVector` converts the result to a `Vector[String]`.

`.map(_.trim)` strips leading and trailing whitespace from each token.

`.filter(_.nonEmpty)` removes any empty strings that might have crept in.

Try it mentally on "Hello, World!": 1. Lowercase: "hello, world!" 2. Split on `[^a-z0-9']+`: we split on `,` and `!`, giving `["hello", "world", ""]` 3. After `.trim` and `.filter(_.nonEmpty)`: `Vector("hello", "world")`

Side note: Regular expressions: `"[^a-z0-9']+"` is a regular expression, a mini-language for describing text patterns. The `.r` at the end compiles it into a `Regex` object.

You don't need to understand regex deeply; just know that ours means "one or more characters that aren't lowercase letters, digits, or apostrophes."

Building the Vocabulary

Once we have a list of tokens, we need to build a vocabulary: a mapping from token to integer ID.

The key design decision: what if our text has 50,000 unique words? That's a lot. For a small model, we might only want the top 3,000 most common words. Words outside the top 3,000 get mapped to a special token called `<UNK>` (for "unknown").

Here's `buildVocab` :

```
def buildVocab(tokens: Vector[String], maxVocab: Int, unkToken: String = "<UNK>"):
  Vocab =
    require(maxVocab >= 2, s"maxVocab must be >= 2, got $maxVocab")

    val freq = tokens.groupMapReduce(identity)(_ => 1)(_ + _)
    val sorted = freq.toVector.sortBy { case (tok, count) => (-count, tok) }
    val selected = sorted.take(maxVocab - 1).map(_._1)
    val idToToken = (unkToken +: selected).distinct.toVector
    val tokenToId = idToToken.zipWithIndex.map { case (t, idx) => t -> idx }.toMap

    Vocab(tokenToId = tokenToId, idToToken = idToToken, unkToken = unkToken)
```

Let's walk through it step by step.

`require(maxVocab >= 2, ...)` : Scala's built-in precondition check. If `maxVocab` is less than 2, it throws an error with the given message. We need at least 2 slots: one for `<UNK>` and one actual word.

`val freq = tokens.groupMapReduce(identity)(_ => 1)(_ + _)` (This counts how many times each token appears. Don't worry too much about the syntax; the result is a `Map[String, Int]` like `Map("the" -> 42, "cat" -> 7, ...)`. (`groupMapReduce` is a Scala 2.13+ function that groups by a key, maps each element to a value, and then reduces with a combining function) here, we group by the token itself, map each occurrence to `1`, and add them up.)

`val sorted = freq.toVector.sortBy { case (tok, count) => (-count, tok) }`: Sort the token-count pairs by frequency, descending. We negate the count so that higher counts sort first. For ties, sort alphabetically.

`val selected = sorted.take(maxVocab - 1).map(_._1)`: Take the top `maxVocab - 1` most frequent tokens (we reserve slot 0 for `<UNK>`). `.map(_._1)` extracts just the token string, discarding the count.

`val idToToken = (unkToken +: selected).distinct.toVector`: Prepend `<UNK>` to the selected tokens. Now `<UNK>` is at index 0. `.distinct` removes any duplicates (shouldn't happen, but defensive).

`val tokenToId = idToToken.zipWithIndex.map { case (t, idx) => t -> idx }.toMap`: Build the reverse mapping: pair each token with its index, then convert to a Map.

The result is a `Vocab` where `<UNK>` always has ID 0, and every other token has a stable integer ID.

Converting Tokens to IDs

Now that we have a `Vocab`, converting a list of tokens to IDs is straightforward:

```
def tokensToIds(tokens: Vector[String], vocab: Vocab): Vector[Int] =  
  tokens.map(vocab.toId)
```

And `vocab.toId` (defined in the `Vocab` case class) looks like:

```
def toId(token: String): Int = tokenToId.getOrElse(token, unkId)
```

`getOrElse` means: look up the token in the map; if it's not there, return `unkId` (which is the ID of `<UNK>`). Any word that wasn't common enough to make the vocabulary becomes `<UNK>`.

Sliding Windows: Creating Training Examples

Here's the core idea of training data for a language model: *for every position in the text, predict what comes next.*

Given text `["the", "cat", "sat", "on", "the", "mat"]` with IDs `[0, 1, 2, 3, 0, 4]` and a context size of 3, we can create these training examples:

CONTEXT	TARGET
<code>[0, 1, 2]</code>	<code>3</code> (the word "on")
<code>[1, 2, 3]</code>	<code>0</code> (the word "the")
<code>[2, 3, 0]</code>	<code>4</code> (the word "mat")

Each example says: "given these 3 tokens as context, the next token is this."

The function `buildExamples` generates all such examples from a sequence of IDs:

```
def buildExamples(ids: Vector[Int], contextSize: Int): Vector[Example] =
  require(contextSize >= 1, s"contextSize must be >= 1, got $contextSize")
  if ids.length <= contextSize then Vector.empty
  else
    Vector.tabulate(ids.length - contextSize) { i =>
      val ctx = ids.slice(i, i + contextSize)
      val tgt = ids(i + contextSize)
      Example(ctx, tgt)
    }
```

`Vector.tabulate(n)(f)` creates a Vector of length `n` where element `i` is `f(i)`. Here, `f(i)` creates an `Example` with context `ids[i..i+contextSize)` and target `ids[i+contextSize]`.

For our example with `contextSize = 3` and `ids.length = 6`, we get `6 - 3 = 3` examples, which matches the table above.

Splitting Into Train and Validation Sets

We won't train on *all* our examples: we'll hold some back to measure how well the model is learning on data it hasn't seen. This is the *validation set*.

```
def splitDeterministic[A](items: Vector[A], trainRatio: Double, seed: Int):  
  (Vector[A], Vector[A]) =  
    require(trainRatio > 0.0 && trainRatio < 1.0, s"trainRatio must be in (0,1), got  
$trainRatio")  
    val rnd = Random(seed)  
    val shuffled = rnd.shuffle(items)  
    val trainSize = math.max(1, (shuffled.length * trainRatio).toInt)  
    (shuffled.take(trainSize), shuffled.drop(trainSize))
```

We shuffle the examples (using a fixed `seed` for reproducibility), then take the first `trainRatio` fraction as training and the rest as validation. With `trainRatio = 0.9`, we keep 90% for training and 10% for validation.

The `seed` ensures that every time you run the program, you get the *same* split. This matters for reproducibility: if you re-run training with the same seed, you're comparing apples to apples.

Side note (Generics: Notice `[A]` after the function name. This makes `splitDeterministic` generic) it works with any type `A`, not just `Example`. You could split a `Vector[String]` or a `Vector[Int]` with the same function. We're writing it this way because it's genuinely useful for both training examples and anything else.

Persisting the Vocabulary

After we build a vocabulary, we want to save it to disk so we can reload it later without reprocessing the text. `VocabIO` handles this:

```
object VocabIO:  
  def save(vocab: Vocab, path: String): Unit = ...  
  def load(path: String): Vocab = ...
```


The vocabulary is saved as a plain text file, one token per line. The first line is always `<UNK>`, so that token IDs are preserved when we load it back. This simple format means you can open the vocabulary file in any text editor and read it.

Putting It All Together

Here's the complete pipeline from text to training examples:

```
// 1. Load and tokenize the text
val rawText = scala.io.Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)

// 2. Build the vocabulary (top 3000 words)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 3000)
println(s"Vocabulary size: ${vocab.size}")

// 3. Convert tokens to IDs
val ids = TextPipeline.tokensToIds(tokens, vocab)

// 4. Create training examples (context window of 3)
val examples = TextPipeline.buildExamples(ids, contextSize = 3)
println(s"Total examples: ${examples.length}")

// 5. Split into train and validation
val (trainExamples, valExamples) = TextPipeline.splitDeterministic(examples,
trainRatio = 0.9, seed = 42)
println(s"Train: ${trainExamples.length}, Val: ${valExamples.length}")

// 6. Save the vocabulary
VocabIO.save(vocab, "data/models/latest.vocab")
```

This is essentially what our training pipeline does. The actual code in the project adds some error handling and configuration, but the structure is identical.

Chapter Milestone

Open the Scala REPL and run the following. You'll need the source file `data/corpus/example-corpus.txt` which is included in the project.

```
// In the REPL, paste these one at a time

// First, let's simulate a tiny corpus
val text = "the cat sat on the mat the rat sat on the cat"

// Tokenize
val tokens = text.split("[^a-z0-9']").toVector.filter(_.nonEmpty)
println(s"Tokens: $tokens")
// → Vector(the, cat, sat, on, the, mat, the, rat, sat, on, the, cat)

// Count frequencies
val freq = tokens.groupMapReduce(identity)(_ => 1)(_ + _)
println(s"Frequencies: ${freq.toVector.sortBy(_._2)}")
// → Vector((the,4), (sat,2), (on,2), (cat,2), (mat,1), (rat,1))

// Build vocab manually (all words in order of frequency)
val idToToken = "<UNK>" +: freq.toVector.sortBy(_._2).map(_._1)
val tokenToId = idToToken.zipWithIndex.toMap
println(s"Vocab: $tokenToId")

// Convert to IDs
val ids = tokens.map(t => tokenToId.getOrElse(t, 0))
println(s"IDs: $ids")

// Create sliding window examples (context=3)
val examples = Vector.tabulate(ids.length - 3) { i =>
  (ids.slice(i, i + 3), ids(i + 3))
}
println(s"First example: context=${examples.head._1}, target=${examples.head._2}")
println(s"Total examples: ${examples.length}")
```

Run this and verify you understand every output. Pay particular attention to the IDs: notice that "the" (the most frequent word) gets ID 1, and every occurrence of "the" in the text maps to 1.

What You Learned

- `case class` creates named, immutable bundles of data: the `Example` and `Vocab` types
 - Tokenization converts raw text into a clean sequence of word strings
 - A vocabulary assigns a unique integer ID to each of the top N most frequent words
 - Rare words are mapped to `<UNK>` (ID 0)
 - Sliding windows over the token ID sequence generate training examples
 - Train/validation splitting holds back a fraction of data for evaluation
 - The vocabulary can be saved to disk and reloaded
-

Source Reference

The code in this chapter corresponds to: - `src/main/scala/data/TextPipeline.scala` : tokenization, vocab building, example generation, train/val split - `src/main/scala/data/VocabIO.scala` : vocabulary persistence

Up Next

We have text → token IDs. Now we need to understand the mathematical building blocks the neural network will use to process those IDs: vectors and matrices. Chapter 3 covers context windows in more detail, and Chapters 4 and 5 introduce the math.

Chapter 3: Context Windows: What the Model Gets to See

We've built the machinery to turn text into token IDs. Now let's think carefully about what we're actually asking the model to do.

A language model's job is to predict the next word. But predict based on *what*? All the text that came before? Just the last few words? Just the previous word?

The answer in our model is: **a fixed-size window of recent token IDs**. We call this the *context window*. Every prediction is made by looking at exactly the last N tokens: no more, no less.

This design choice shapes everything. The context window size N is a hyperparameter: a number you choose before training begins. If N is 3, the model sees 3 tokens and predicts the 4th. If N is 10, it sees 10 and predicts the 11th. Bigger windows capture more history, but require larger models and more compute.

In this chapter, we'll look more closely at how context windows work, finish the data pipeline, and arrive at the exact data structure the model will train on.

The Prediction Task, Precisely

Let's make the prediction task concrete.

Suppose our text is:

```
the cat sat on the mat
```

And our token IDs are:

```
the→1  cat→2  sat→3  on→4  mat→5
```

So the text as IDs is `[1, 2, 3, 4, 1, 5]`.

With a context window of size 3, here's every valid training example we can extract:

```
Position 0: context=[1, 2, 3] target=4    ("the cat sat" → "on")
Position 1: context=[2, 3, 4] target=1    ("cat sat on" → "the")
Position 2: context=[3, 4, 1] target=5    ("sat on the" → "mat")
```

That's it: only 3 examples from 6 tokens with a context size of 3. For each starting position, we take a window of 3 tokens as context, and the token immediately after the window is the target.

Notice: - "the" appears twice in the text (at positions 0 and 4). As a target, it carries different contexts. The model should learn that "cat sat on" is often followed by "the" in this kind of text. - We get `(text_length - context_size)` examples total. Longer text = more examples. This is why having lots of training data matters.

The `Example` Type

Each training example is represented by the `Example` case class we saw in Chapter 2:

```
final case class Example(context: Vector[Int], target: Int)
```

The `final` keyword just means you can't subclass `Example`: it's a design detail, not something to worry about.

A few concrete examples of this type in action:

```
val ex1 = Example(context = Vector(1, 2, 3), target = 4)
val ex2 = Example(context = Vector(2, 3, 4), target = 1)
val ex3 = Example(context = Vector(3, 4, 1), target = 5)
```

These are the three examples from the "the cat sat on the mat" example above (with N=3).

Case classes in Scala give us structural equality for free:

```
Example(Vector(1, 2, 3), 4) == Example(Vector(1, 2, 3), 4) // → true
Example(Vector(1, 2, 3), 4) == Example(Vector(1, 2, 3), 5) // → false
```

This is useful for tests: you can compare two `Example` values directly without writing custom equality logic.

Building Examples: `Vector.tabulate`

The `buildExamples` function uses `Vector.tabulate`, which is worth understanding:

```
Vector.tabulate(n)(f)
```

This creates a `Vector` of length `n`, where element `i` is `f(i)`. It's equivalent to writing:

```
(0 until n).map(i => f(i)).toVector
```

But more concise. In `buildExamples`:

```
def buildExamples(ids: Vector[Int], contextSize: Int): Vector[Example] =  
  require(contextSize >= 1, s"contextSize must be >= 1, got $contextSize")  
  if ids.length <= contextSize then Vector.empty  
  else  
    Vector.tabulate(ids.length - contextSize) { i =>  
      val ctx = ids.slice(i, i + contextSize)  
      val tgt = ids(i + contextSize)  
      Example(ctx, tgt)  
    }  
  }
```

For `ids = Vector(1, 2, 3, 4, 1, 5)` and `contextSize = 3`:- `ids.length - contextSize = 3`, so we create 3 examples - `i=0`: `ctx = ids.slice(0, 3) = Vector(1, 2, 3)`, `tgt = ids(3) = 4` - `i=1`: `ctx = ids.slice(1, 4) = Vector(2, 3, 4)`, `tgt = ids(4) = 1` - `i=2`: `ctx = ids.slice(2, 5) = Vector(3, 4, 1)`, `tgt = ids(5) = 5`

The edge case: if `ids.length <= contextSize`, there are no valid windows, so we return `Vector.empty`. A corpus too short to generate even one example is useless for training.

Shuffling and Splitting

Once we have all our examples, we need to split them into two sets:

Training set: Examples the model will learn from. It sees these during training, adjusts its weights based on them.

Validation set: Examples held back for evaluation. The model *never* trains on these. We use them to measure how well the model generalizes to data it hasn't seen.

Why split at all? Because a model that memorizes its training data isn't useful. We want it to learn *patterns*, not specific text. The validation set is our way of checking whether the model has learned something general.

The split is done with `splitDeterministic` :

```
def splitDeterministic[A](
  items: Vector[A],
  trainRatio: Double,
  seed: Int
): (Vector[A], Vector[A]) =
  require(trainRatio > 0.0 && trainRatio < 1.0, ...)
  val rnd = Random(seed)
  val shuffled = rnd.shuffle(items)
  val trainSize = math.max(1, (shuffled.length * trainRatio).toInt)
  (shuffled.take(trainSize), shuffled.drop(trainSize))
```

The function returns a *tuple*: two values at once, separated by a comma. In Scala, you can destructure a tuple directly:

```
val (trainExamples, valExamples) = TextPipeline.splitDeterministic(
  examples,
  trainRatio = 0.9,
  seed = 42
)
```

This binds `trainExamples` and `valExamples` in one line.

Why shuffle? Because our examples are in text order: all the examples from the beginning of the corpus come first. If we didn't shuffle, the training set would be "early text" and the validation set would be "late text," which could introduce a spurious bias. Shuffling ensures the split is random.

Why fix the `seed` ? So the split is reproducible. Run the same code twice, get the same split. This matters when you want to compare different model configurations fairly.

Saving the Vocabulary

The vocabulary needs to be saved to disk so the model can be loaded and used later. You train once, but you might want to run inference (make predictions) many times without re-training.

`VocabIO` handles this with a simple text format: one token per line, where the line number (0-indexed) is the token ID.

```
<UNK>
the
cat
sat
on
mat
```

Loading this file back reconstructs the same vocabulary: `idToToken(0) = "<UNK>"` , `idToToken(1) = "the"` , etc.

This format is deliberately simple. You can open it in a text editor and read it. You can write a tool in any language to parse it. There's no binary encoding to deal with, no version compatibility issues.

Here's the full flow from raw text to saved vocabulary:


```
// Load text
val rawText = scala.io.Source.fromFile("data/corpus/example-corpus.txt").mkString

// Tokenize
val tokens = TextPipeline.tokenize(rawText)

// Build vocab (keep top 3000 words)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 3000)

// Save vocab for later use
VocabIO.save(vocab, "data/models/latest.vocab")

// Convert tokens to IDs
val ids = TextPipeline.tokensToIds(tokens, vocab)

// Create examples with context window of 3
val examples = TextPipeline.buildExamples(ids, contextSize = 3)

// Split 90/10 train/val
val (train, val_) = TextPipeline.splitDeterministic(examples, 0.9, seed = 42)

println(s"Vocab size: ${vocab.size}")
println(s"Total examples: ${examples.length}")
println(s"Train: ${train.length}, Val: ${val_.length}")
```

Choosing a Context Size

What should `contextSize` be? This is a design decision with real trade-offs.

Smaller context (N=2 or 3): - Less information per example - Smaller model (fewer weights) - Faster training - Can only capture short-range dependencies ("the cat" → "sat")

Larger context (N=8 or 10): - More information per example - Larger model required (the input layer grows proportionally) - Slower training - Can capture longer-range dependencies ("the cat that sat on the mat" → ...)

For this book, we'll use N=3 for most examples. It's small enough that the model trains quickly, but large enough to produce something interesting. Real production models use context sizes in the thousands: but they also have billions of parameters. We're building a small model for learning purposes.

***Side note: modern transformers:** Large language models like GPT-4 can take context windows of 128,000 tokens or more. They use a different architecture (the transformer) that handles long context far more efficiently than our feed-forward approach. But the fundamental prediction task is the same: given N tokens, predict the next one.*

Chapter Milestone

This milestone brings together everything from Chapters 2 and 3. You'll run the full data pipeline on the included example corpus and inspect the results.

In your terminal, from the project directory:

```
sbt console
```

Then in the REPL:

```

import data.*
import scala.io.Source

// Load the example corpus
val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
println(s"Text length: ${rawText.length} characters")

// Tokenize
val tokens = TextPipeline.tokenize(rawText)
println(s"Token count: ${tokens.length}")
println(s"First 10 tokens: ${tokens.take(10)}")

// Build vocabulary
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 500)
println(s"Vocab size: ${vocab.size}")
println(s"Most common words: ${vocab.idToToken.slice(1, 11)}")

// Convert to IDs and build examples
val ids = TextPipeline.tokensToIds(tokens, vocab)
val examples = TextPipeline.buildExamples(ids, contextSize = 3)
println(s"Total examples: ${examples.length}")

// Show first example with human-readable tokens
val firstEx = examples.head
val ctxWords = firstEx.context.map(vocab.toToken)
val tgtWord = vocab.toToken(firstEx.target)
println(s"First example: context=$ctxWords → target='$tgtWord'")

// Split
val (train, validation) = TextPipeline.splitDeterministic(examples, 0.9, 42)
println(s"Train: ${train.length}, Validation: ${validation.length}")

```

When you see the first example printed as actual words (not IDs), that's the moment the data pipeline becomes tangible. You're looking at exactly the kind of prediction task the model will learn from.

What You Learned

- A language model sees a fixed-size window of recent tokens (the context) and predicts the next one

- `buildExamples` uses sliding windows to generate all valid `(context, target)` pairs from a token ID sequence
 - Shorter text = fewer examples; the context size determines how many we lose from the ends
 - Train/validation splitting is essential for measuring generalization
 - The `seed` parameter makes the shuffle (and therefore the split) reproducible
 - `VocabIO` persists the vocabulary in a plain text format that's human-readable and universally loadable
-

Source Reference

- `src/main/scala/data/TextPipeline.scala` : everything in this chapter
 - `src/main/scala/data/VocabIO.scala` : vocabulary persistence
-

Up Next

We have our data pipeline. We know what the model's inputs will look like: a `Vector[Int]` of token IDs representing a context window. Now we need to understand how to process those numbers mathematically. Chapter 4 introduces vectors: the fundamental unit of neural network computation.

Chapter 4: Vectors: The Language of Numbers

We've been working with token IDs: integers like `[1, 2, 3]` that represent words. But neural networks don't just work with individual integers: they work with *lists* of decimal numbers, doing arithmetic on all of them simultaneously.

The mathematical name for "a list of numbers" is a **vector**. You might have encountered vectors in a physics class as arrows pointing in a direction. The idea is related but more general: in machine learning, a vector is simply an ordered sequence of numbers.

This chapter introduces vectors, explains why they're the right tool for neural networks, and builds the Scala code for working with them. By the end, you'll be ready to understand the core arithmetic of a neural network.

What Is a Vector?

A vector is a list of numbers with a fixed length. Here are some examples:

```
v = [3.0, 1.5, -2.0, 0.7]    (length 4)
w = [0.1, 0.2, 0.3]         (length 3)
u = [100.0]                  (length 1)
```

Each number in the vector is called a **component** or **element**. We refer to the element at position `i` as `v[i]` (using 0-based indexing).

In Scala, we represent vectors as `Vector[Double]` :

```
val v: Vector[Double] = Vector(3.0, 1.5, -2.0, 0.7)
val w: Vector[Double] = Vector(0.1, 0.2, 0.3)
```

Because we use this type constantly, the project defines a type alias:

```
type Vec = Vector[Double]
```

After this definition, `Vec` means exactly the same thing as `Vector[Double]`. You can use either; `Vec` is just shorter.

Vectors as Measurements

The best intuition for a vector: it's a compact summary of several measurements about one thing.

Imagine you're baking bread and you want to describe a flour in terms of four properties: - Protein content (%) - Moisture content (%) - Particle size (µm) - Absorption rate (%)

You could write: "This flour has 12% protein, 14% moisture, 85µm particle size, 65% absorption."

Or you could write: `[12.0, 14.0, 85.0, 65.0]`.

That vector is a complete description of the flour: one number per property, in a fixed order. Different flours would give different vectors. Similar flours would give similar vectors.

This is exactly how our language model will work. Every word in the vocabulary will be represented as a vector of numbers. The numbers are learned during training, but the principle is the same: the vector is a compact summary of the word's "meaning" in a mathematical sense.

Vector Addition

The first operation is **vector addition**. To add two vectors, add their corresponding elements:

```
[1.0, 2.0, 3.0]
+ [4.0, 5.0, 6.0]
= [5.0, 7.0, 9.0]
```

In Scala:

```
def vecAdd(a: Vec, b: Vec): Vec =  
  requireSameLength(a, b, "vecAdd")  
  a.indices.map(i => a(i) + b(i)).toVector
```

We check that the vectors have the same length first (you can't add a vector of length 3 to one of length 4: the dimensions don't match). Then we sum element by element.

Intuition: Adding two flour descriptions gives you the "average direction" between them. If flour A is high-protein and flour B is high-moisture, their sum will have some of both properties. This doesn't always mean anything physically, but mathematically it's a useful operation.

Scalar Multiplication

A **scalar** is a single number (as opposed to a vector of numbers). To multiply a vector by a scalar, multiply every element:

```
2.0 × [1.0, 2.0, 3.0] = [2.0, 4.0, 6.0]
```

```
def scalarMul(a: Vec, s: Double): Vec = a.map(_ * s)
```

This scales the whole vector up or down. In the context of neural networks, scalars appear as learning rates and normalization factors.

The Dot Product

The **dot product** is the most important operation in neural networks. It takes two vectors of the same length and produces a single number (a scalar).

Formula: multiply the corresponding elements, then add up all the products.

```
[1.0, 2.0, 3.0] · [4.0, 5.0, 6.0]
= (1.0 × 4.0) + (2.0 × 5.0) + (3.0 × 6.0)
= 4.0 + 10.0 + 18.0
= 32.0
```

In Scala:

```
def dot(a: Vec, b: Vec): Double =
  requireSameLength(a, b, "dot")
  var sum = 0.0
  var i = 0
  while i < a.length do
    sum += a(i) * b(i)
    i += 1
  sum
```

Note the `while` loop instead of `.map` and `.sum`. Both would give the same result, but the loop avoids creating intermediate data structures. For performance-critical code (and the dot product is called millions of times during training), this matters.

INTUITION: HOW MUCH DO TWO VECTORS AGREE?

Here's the best intuition for the dot product: it measures how much two vectors *agree*.

Return to our flour example. Suppose you have a recipe that says "I work best with flour that has: `[high protein, low moisture, medium particle size, high absorption]`": encoded as `[1.0, -1.0, 0.0, 1.0]`.

And you have two flours: - Flour A: `[0.9, -0.8, 0.1, 0.9]` (mostly matches the recipe's preferences) - Flour B: `[-0.5, 0.6, 0.0, -0.4]` (mostly disagrees)

The dot product of the recipe vector with flour A will be high (they point in the same direction). The dot product with flour B will be low or negative (they point in opposite directions).

This is precisely how the output layer of our neural network works: the hidden layer produces a vector, and we compute the dot product of that vector with a *learned* weight vector for each word in the vocabulary. High dot product = the model thinks this word is likely next.

COMPUTING A DOT PRODUCT BY HAND

Let's be very concrete. Say $a = [2.0, 3.0, -1.0]$ and $b = [1.0, -2.0, 4.0]$.

Step 1: Multiply corresponding elements:

$$2.0 \times 1.0 = 2.0$$

$$3.0 \times (-2.0) = -6.0$$

$$(-1.0) \times 4.0 = -4.0$$

Step 2: Sum them:

$$2.0 + (-6.0) + (-4.0) = -8.0$$

Result: -8.0

Try verifying this in the Scala REPL:

```
val a = Vector(2.0, 3.0, -1.0)
val b = Vector(1.0, -2.0, 4.0)
// Using LinearAlgebra.dot:
a.zip(b).map { case (x, y) => x * y }.sum
// → -8.0
```

The Hadamard Product

Less famous than the dot product, but also useful: the **Hadamard product** (or element-wise product) multiplies corresponding elements but keeps them as a vector instead of summing.

$$[1.0, 2.0, 3.0] \odot [4.0, 5.0, 6.0] = [4.0, 10.0, 18.0]$$

```
def hadamard(a: Vec, b: Vec): Vec =
  requireSameLength(a, b, "hadamard")
  a.indices.map(i => a(i) * b(i)).toVector
```

We'll use the Hadamard product when we compute activation gradients in backpropagation: it lets us apply element-wise corrections to each position independently.

Vectors of Zeros

One more utility: creating a vector of all zeros. We need this constantly when initializing bias terms to zero.

```
def zeros(n: Int): Vec = Vector.fill(n)(0.0)
```

`Vector.fill(n)(value)` creates a Vector of length `n` where every element is `value`.

The `Vec` Type in Context

Here's how vectors appear in the neural network code. The `Params` case class (which holds all model weights) is full of them:

```
final case class Params(E: Matrix, W1: Matrix, b1: Vec, W2: Matrix, b2: Vec)
```

`b1` and `b2` are bias vectors: one number per neuron in each layer. They'll be initialized to `zeros(hiddenDim)` and `zeros(vocabSize)` respectively.

And the `ForwardCache` (which stores intermediate values during a forward pass):

```
final case class ForwardCache(x: Vec, z1: Vec, a1: Vec, logits: Vec, probs: Vec, ...)
```

Every intermediate result (the input vector `x`, the pre-activation `z1`, the post-activation `a1`, the output logits, the probabilities) is a `Vec`.

Vectors are the currency of neural network computation.

Chapter Milestone

Let's write a small program that computes dot products and explores what they measure.

Open `sbt console` and try:

```

// Define some vectors
val recipe = Vector(1.0, -1.0, 0.0, 1.0)    // "ideal flour profile"
val flourA  = Vector(0.9, -0.8, 0.1, 0.9)  // similar to ideal
val flourB  = Vector(-0.5, 0.6, 0.0, -0.4) // dissimilar

// Dot product by hand
def dot(a: Vector[Double], b: Vector[Double]): Double =
  a.zip(b).map { case (x, y) => x * y }.sum

val scoreA = dot(recipe, flourA)
val scoreB = dot(recipe, flourB)

println(s"Score for flour A: $scoreA")    // Should be positive, near 2.7
println(s"Score for flour B: $scoreB")    // Should be negative, near -1.3

// Vector addition
def vecAdd(a: Vector[Double], b: Vector[Double]): Vector[Double] =
  a.zip(b).map { case (x, y) => x + y }

println(s"flourA + flourB = ${vecAdd(flourA, flourB)}")

// Scalar multiplication
def scalarMul(v: Vector[Double], s: Double): Vector[Double] =
  v.map(_ * s)

println(s"2 × flourA = ${scalarMul(flourA, 2.0)}")

// Zeros
val zeros = Vector.fill(4)(0.0)
println(s"Zeros: $zeros")

```

Verify that flour A's score (dot product with the "recipe") is higher than flour B's. This is the dot product measuring agreement. In the neural network, this exact mechanism will be used to score how likely each word is as the next token.

What You Learned

- A vector is an ordered list of numbers with a fixed length
- `Vec` is a Scala type alias for `Vector[Double]` : our main data structure
- **Vector addition**: add corresponding elements (requires same length)

- **Scalar multiplication:** multiply every element by the same number
 - **Dot product:** multiply corresponding elements then sum: measures agreement between two vectors
 - **Hadamard product:** multiply corresponding elements, keep as vector
 - `Vector.fill(n)(0.0)` creates a zero vector of length `n`
-

Source Reference

- `src/main/scala/linalg/Types.scala` : the `Vec` type alias
 - `src/main/scala/linalg/LinearAlgebra.scala` : `vecAdd` , `scalarMul` , `hadamard` , `dot` , `zeros`
-

Up Next

Chapter 5 extends the idea from vectors to matrices. A matrix is a two-dimensional arrangement of numbers (a table) and it's the key ingredient for the linear transformations at the heart of a neural network.

Chapter 5: Matrices: Tables of Transformations

A vector is a list of numbers. A matrix is a table of numbers.

That might sound underwhelming: a spreadsheet is a table of numbers, and spreadsheets aren't what anyone usually has in mind when they talk about neural networks. But matrices are more than just organized data. The key insight is this: **a matrix is a function that transforms one vector into another.**

Give a matrix a vector, and it returns a new vector: usually of a different length. This transformation is the heart of every layer in a neural network. Understanding it will make the forward pass feel obvious rather than mysterious.

The `Matrix` Case Class

Our `Matrix` type is defined as:

```
final case class Matrix(  
  data: Vector[Double],  
  rows: Int,  
  cols: Int,  
  transposed: Boolean = false,  
  stride: Int = -1  
)
```

There's something worth noticing here: we store the matrix as a flat `Vector[Double]`, not as a `Vector[Vector[Double]]`. All the numbers are in one long list, and we use the `rows` and `cols` dimensions to interpret that list as a 2D table.

Why? Because flat arrays are more cache-friendly for the CPU. When you read elements sequentially from a flat array, the processor can prefetch them efficiently. A nested structure (a vector of vectors) scatters the data in memory, causing more cache misses and slower code.

The layout: For a matrix with `rows=2, cols=3` stored as `[1, 2, 3, 4, 5, 6]`, the elements are: Row 0: `data[0]=1 data[1]=2 data[2]=3` Row 1: `data[3]=4 data[4]=5 data[5]=6` To get element at row `r`, column `c`: the index into `data` is `r * cols + c`. This is row-major order.

Getting and Setting Elements

```
def get(r: Int, c: Int): Double = data(linearIndex(r, c))
```

`linearIndex` converts `(r, c)` to a flat index, handling the transpose case too. To read the element at row 1, column 2 of a 3×4 matrix:

```
val m = Matrix(Vector(1,2,3,4, 5,6,7,8, 9,10,11,12), rows=3, cols=4)
m.get(1, 2) // → 7 (row 1, col 2 in 0-indexed, row-major)
```

We also have `rowSlice`: extract an entire row as a `Vec`:

```
m.rowSlice(0) // → Vector(1.0, 2.0, 3.0, 4.0)
m.rowSlice(1) // → Vector(5.0, 6.0, 7.0, 8.0)
```

We'll use `rowSlice` constantly when looking up embeddings: row `i` of the embedding matrix is the embedding vector for token `i`.

Creating Matrices

Zero matrix:

```
Matrix.zeros(rows = 3, cols = 4)
// → all 12 elements are 0.0
```

From a function:

```
Matrix.fromFunction(rows, cols)((r, c) => f(r, c))
```

This is the most flexible constructor: provide a function that takes a row and column index and returns the element value.

For example, an identity matrix (1s on the diagonal, 0s elsewhere):

```
val identity = Matrix.fromFunction(3, 3)((r, c) => if r == c then 1.0 else 0.0)
// → [[1, 0, 0],
//      [0, 1, 0],
//      [0, 0, 1]]
```

Matrix-Vector Multiplication: The Key Operation

Here is the operation that makes neural networks work: **multiplying a matrix by a vector**.

Given a matrix **M** with shape **(rows × cols)** and a vector **v** with length **cols**, the product **M @ v** is a new vector with length **rows**.

The formula: the **i**-th element of the output is the **dot product** of row **i** of **M** with **v**.

M [0,0]	M [0,1]	M [0,2]		v [0]	=	[dot(M _row0, v)]
M [1,0]	M [1,1]	M [1,2]	×	v [1]	=	[dot(M _row1, v)]
M [2,0]	M [2,1]	M [2,2]		v [2]	=	[dot(M _row2, v)]

In Scala:

```
def matVecMul(m: Matrix, v: Vec): Vec =
  require(m.cols == v.length, ...)
  Vector.tabulate(m.rows) { r =>
    var sum = 0.0
    var c = 0
    while c < m.cols do
      sum += m.get(r, c) * v(c)
      c += 1
    sum
  }
```

Each row of the matrix is one dot product with the input vector.

CONCRETE EXAMPLE

Let's multiply a 2×3 matrix by a 3-element vector:

```

M = [[2, 0, 1],      v = [3]      output[0] = 2×3 + 0×1 + 1×2 = 8
     [0, 3, 2]]      [1]      output[1] = 0×3 + 3×1 + 2×2 = 7
                       [2]

```

The output has 2 elements (one per row of M). The input had 3 elements (one per column of M). The matrix *transformed* a 3-dimensional vector into a 2-dimensional one.

This is the fundamental operation of every linear layer in a neural network: take the input vector, transform it into a different-size vector using a weight matrix. The weight matrix is what the network learns.

The Transpose: Flipping Rows and Columns

The **transpose** of a matrix flips it along its diagonal: rows become columns and columns become rows.

For a matrix with shape `(rows × cols)`, the transpose has shape `(cols × rows)`.

```

M = [[1, 2, 3],      M^T = [[1, 4],
     [4, 5, 6]]        [2, 5],
                       [3, 6]]

```

In our code, the transpose is implemented as a *view*: it doesn't copy any data:

```

def transposeView: Matrix =
  Matrix(data = data, rows = cols, cols = rows, transposed = !transposed, stride =
    internalStride)

```

When `transposed = true`, the `linearIndex` function swaps the row and column before computing the flat index. Same data, different interpretation. This is an important optimization: during backpropagation, we'll need `w.transposeView` many times. Without this trick, each call would copy potentially millions of numbers.

We'll use the transpose during backpropagation (Chapter 12). Don't worry about it yet: just know it exists and why.

The Outer Product

The **outer product** of two vectors `a` (length `m`) and `b` (length `n`) produces an `(m × n)` matrix:

```
a = [a0, a1, a2]
b = [b0, b1, b2, b3]

outer(a, b) = [[a0×b0, a0×b1, a0×b2, a0×b3],
               [a1×b0, a1×b1, a1×b2, a1×b3],
               [a2×b0, a2×b1, a2×b2, a2×b3]]
```

Every possible product of one element from `a` with one element from `b`.

```
def outer(a: Vec, b: Vec): Matrix =
  Matrix.fromFunction(a.length, b.length)((r, c) => a(r) * b(c))
```

The outer product is used in backpropagation to compute weight gradients. When we're adjusting how much each weight contributed to an error, we end up computing an outer product of the "how wrong were we" vector with the "what was the input" vector. More on this in Chapter 12.

General Matrix Multiplication

When you need to multiply two matrices (not a matrix and a vector), use `matMul`:

```
def matMul(a: Matrix, b: Matrix): Matrix =
  require(a.cols == b.rows, ...)
  Matrix.fromFunction(a.rows, b.cols) { (r, c) =>
    var sum = 0.0
    var k = 0
    while k < a.cols do
      sum += a.get(r, k) * b.get(k, c)
      k += 1
    sum
  }
```

The shape rule: to multiply a ($m \times k$) by b ($k \times n$), the inner dimensions must match: `a.cols == b.rows`. The result has shape $(m \times n)$.

We'll use `matMul` during batch training, when we process multiple examples simultaneously.

The Matrix as a Transformation Machine

Let's make the "matrix as transformation" idea concrete.

Consider a matrix:

```
W = [[0.5, -0.3],  
      [0.2,  0.8],  
      [-0.1, 0.6]]
```

This is a (3×2) matrix. It takes 2-dimensional inputs and produces 3-dimensional outputs.

Suppose you have a 2D point $v = [1.0, 0.0]$ (on the x-axis):

```
W @ v = [0.5×1 + (-0.3)×0,    0.2×1 + 0.8×0,    (-0.1)×1 + 0.6×0]  
        = [0.5, 0.2, -0.1]
```

And $v = [0.0, 1.0]$ (on the y-axis):

```
W @ v = [(-0.3), 0.8, 0.6]
```

The matrix sent these two points to different locations in 3D space. Any other 2D input would be transformed to some 3D output determined by the matrix. The matrix *defines* the transformation.

In a neural network, the weight matrix `w1` defines how to transform the input (concatenated embeddings) into the hidden layer. Every element of `w1` is a learnable parameter: during training, we adjust these numbers to make the transformation produce useful hidden representations.

Chapter Milestone

Let's build and manipulate matrices in the REPL.

```

// In sbt console
import linalg.*

// Create a 3x2 matrix
val M = Matrix(
  data = Vector(1.0, 2.0,    // row 0
                3.0, 4.0,    // row 1
                5.0, 6.0),    // row 2
  rows = 3,
  cols = 2
)

// Access elements
println(M.get(0, 0)) // → 1.0
println(M.get(1, 1)) // → 4.0
println(M.get(2, 0)) // → 5.0

// Get a row
println(M.rowSlice(0)) // → Vector(1.0, 2.0)
println(M.rowSlice(2)) // → Vector(5.0, 6.0)

// Multiply M by a vector [1.0, 0.5]
val v = Vector(1.0, 0.5)
val result = LinearAlgebra.matVecMul(M, v)
// row 0: 1×1 + 2×0.5 = 2.0
// row 1: 3×1 + 4×0.5 = 5.0
// row 2: 5×1 + 6×0.5 = 8.0
println(result) // → Vector(2.0, 5.0, 8.0)

// Transpose: M was (3×2), M^T should be (2×3)
val MT = M.transposeView
println(s"Transposed shape: ${MT.rows}×${MT.cols}")
println(MT.rowSlice(0)) // → Vector(1.0, 3.0, 5.0) (was column 0 of M)

// Outer product of [1.0, 2.0] and [3.0, 4.0, 5.0]
val a = Vector(1.0, 2.0)
val b = Vector(3.0, 4.0, 5.0)
val outerProduct = LinearAlgebra.outer(a, b)
// Should be:
// [[1×3, 1×4, 1×5],
//  [2×3, 2×4, 2×5]]
// = [[3, 4, 5], [6, 8, 10]]
println(s"Outer product shape: ${outerProduct.rows}×${outerProduct.cols}")
println(outerProduct.rowSlice(0)) // → Vector(3.0, 4.0, 5.0)
println(outerProduct.rowSlice(1)) // → Vector(6.0, 8.0, 10.0)

```

The key moment in this milestone is verifying the `matVecMul` result by computing it by hand: row 0 dotted with `v` gives `2.0`, row 1 gives `5.0`, row 2 gives `8.0`. When you verify it manually and see it match the code, matrix multiplication becomes concrete.

What You Learned

- A `Matrix` stores elements in a flat `Vector[Double]` with explicit `rows` and `cols` dimensions (row-major order)
 - `Matrix.zeros(r, c)` creates a zero matrix; `Matrix.fromFunction(r, c)(f)` creates one with a formula
 - `matVecMul(M, v)`: each output element is a dot product of one row of `M` with `v`: this is the core neural network operation
 - `transposeView` flips rows and columns without copying data: used heavily in backpropagation
 - `outer(a, b)` produces an `(m × n)` matrix of all pairwise products: used for gradient computation
 - `matMul(a, b)` multiplies two matrices: used for batch training
-

Source Reference

- `src/main/scala/linalg/Types.scala`: the `Matrix` case class and `Vec` type alias
 - `src/main/scala/linalg/LinearAlgebra.scala`: `matVecMul`, `matMul`, `outer`, `addRowBias`, `reduceSumRows`
-

Up Next

We now have all the mathematical tools we need. Part 1 is complete.

In Part 2, we start building the actual neural network. Chapter 6 introduces embeddings: the mechanism by which words are turned into vectors that the network can process. After Chapter 6, the pieces will start clicking together.

Chapter 6: Embeddings: Words in Space

We have token IDs: integers that identify words. We have the mathematical tools to process vectors and matrices. Now we need to bridge the gap: how do we turn an integer like 42 into something a neural network can actually work with?

The answer is called an **embedding**.

The Problem with Integers

Suppose the word "cat" has ID 42 and the word "kitten" has ID 891. Those numbers are completely arbitrary: 42 and 891 have no mathematical relationship to each other, even though "cat" and "kitten" are semantically similar.

If we fed those integers directly into a neural network, the network would have no way to know that 42 and 891 are "close" in meaning. It would just see two very different numbers.

We need a representation where similar words are close to each other in some mathematical sense. And that's what embeddings give us.

What Is an Embedding?

An **embedding** is a dense vector of real numbers that represents a word. Instead of the integer 42, the word "cat" becomes something like:

```
cat → [0.31, -0.12, 0.88, 0.45, -0.67, 0.23, ...] (many dimensions)
```

Two key properties make embeddings useful:

1. **They're dense:** every dimension has a value, unlike one-hot encodings where most values are zero.
2. **They're learned:** the values aren't hand-crafted; the neural network discovers them during training in a way that helps it predict well.

After training, words that tend to appear in similar contexts end up with similar embeddings. "Cat" and "kitten" might end up with vectors that point in roughly the same direction. "Cat" and "skyscraper" would end up pointing in very different directions.

This is remarkable: starting from arbitrary random initialization, training causes the network to organize words in a meaningful geometric space. We don't program this in explicitly; it emerges.

The Embedding Matrix

All word embeddings are stored in a single matrix called **E** :

```
E has shape: (vocabSize × embedDim)
```

- **vocabSize** rows: one row per word in the vocabulary
- **embedDim** columns: the number of dimensions in each embedding

For a vocabulary of 500 words and an embedding dimension of 16:

```
E = (500 × 16) matrix
```

Row `i` of `E` is the embedding vector for token ID `i`. To look up the embedding for token 42, you read row 42:

```
val embedding = E.rowSlice(42) // → a Vec of length embedDim
```

This is just `rowSlice`: the operation we already know.

The `Params` Case Class

All model parameters (including the embedding matrix) live in a case class called `Params`:

```
final case class Params(E: Matrix, W1: Matrix, b1: Vec, W2: Matrix, b2: Vec)
```

Right now, focus on `E`. The other parameters (`W1`, `b1`, `W2`, `b2`) belong to layers we haven't built yet.

`Params` is immutable: every time we update the model's weights during training, we create a new `Params` with updated values. The old `Params` is discarded. This immutability makes the code much easier to reason about: no shared mutable state that can be accidentally modified.

Initializing the Embedding Matrix

When training starts, we initialize all parameters with small random values. But not completely random: there's a specific strategy.

Xavier uniform initialization (also called Glorot initialization) initializes weights by sampling uniformly from the range `[-bound, bound]` where:

```
bound = sqrt(6) / sqrt(fanIn + fanOut)
```

`fanIn` is the number of inputs to a layer, `fanOut` is the number of outputs.


```
private def xavierUniform(fanOut: Int, fanIn: Int, rnd: Random): Matrix =
  val bound = math.sqrt(6.0) / math.sqrt(fanIn.toDouble + fanOut.toDouble)
  Matrix.fromFunction(fanOut, fanIn)((_, _) => rnd.between(-bound, bound))
```

Why not just use `Random.nextDouble()` ? If weights are too large, signals get amplified exponentially as they pass through layers. If weights are too small, signals vanish. Xavier initialization chooses a scale that keeps signals in a stable range from the very beginning of training, making learning much more reliable.

Why uniform specifically? We want values spread evenly across the range, with no preference for any particular value. Uniform distribution achieves this.

Initializing All Parameters

`initParams` creates the complete `Params` from a `ModelConfig` :

```
def initParams(cfg: ModelConfig, seed: Int): Params =
  val rnd = Random(seed)

  val E = xavierUniform(cfg.vocabSize, cfg.embedDim, rnd)
  val W1 = xavierUniform(cfg.hiddenDim, cfg.contextSize * cfg.embedDim, rnd)
  val b1 = Vector.fill(cfg.hiddenDim)(0.0)
  val W2 = xavierUniform(cfg.vocabSize, cfg.hiddenDim, rnd)
  val b2 = Vector.fill(cfg.vocabSize)(0.0)

  Params(E = E, W1 = W1, b1 = b1, W2 = W2, b2 = b2)
```

Notice: - `E` has shape `(vocabSize × embedDim)` : one row per vocabulary word - `W1` has shape `(hiddenDim × contextSize*embedDim)` : takes the flattened context embedding as input - `b1` is a zero vector of length `hiddenDim` - `W2` has shape `(vocabSize × hiddenDim)` : produces one score per vocabulary word - `b2` is a zero vector of length `vocabSize`

We initialize biases to zero because there's no reason to prefer one direction initially. Weights get Xavier initialization because symmetry-breaking (having different initial values) is important for learning.

The `seed` ensures the same initialization is produced every time, which matters for reproducibility.

The `ModelConfig`

Before we can call `initParams`, we need a `ModelConfig`:

```
final case class ModelConfig(  
  contextSize: Int,  
  embedDim: Int,  
  hiddenDim: Int,  
  vocabSize: Int,  
  activation: String = "tanh"  
)
```

These are the architectural choices: - `contextSize`: how many tokens of context the model sees at once - `embedDim`: how many dimensions per word embedding (typically 16–128) - `hiddenDim`: how many neurons in the hidden layer (typically 32–256) - `vocabSize`: how many words in the vocabulary - `activation`: which activation function: `"tanh"` or `"relu"`

For a beginner model on a small corpus:

```
val cfg = ModelConfig(  
  contextSize = 3,  
  embedDim = 16,  
  hiddenDim = 64,  
  vocabSize = 500  
)
```

This will produce a model with: - `E`: $500 \times 16 = 8,000$ parameters - `w1`: $64 \times 48 = 3,072$ parameters ($48 = \text{contextSize} \times \text{embedDim} = 3 \times 16$) - `b1`: 64 parameters - `w2`: $500 \times 64 = 32,000$ parameters - `b2`: 500 parameters

Total: 43,636 parameters. That's tiny compared to GPT-4's hundreds of billions, but it's enough to learn real patterns from a small corpus.

Embedding Lookup: The First Step of the Forward Pass

When the model receives a context window of token IDs, the very first thing it does is look up the embedding for each token:

```
// context = Vector(2, 5, 7) -- three token IDs
val embeddingVectors = context.map(id => p.E.rowSlice(id))
// → Vector(
//   Vector(-0.12, 0.34, ...), // embedding for token 2
//   Vector(0.67, -0.23, ...), // embedding for token 5
//   Vector(0.45, 0.11, ...)   // embedding for token 7
// )
```

Now we have three embedding vectors. But the next layer expects a single vector as input. So we **concatenate** them:

```
val x = context.flatMap(id => p.E.rowSlice(id))
// → one long vector: embedding(2) ++ embedding(5) ++ embedding(7)
// length = contextSize × embedDim = 3 × 16 = 48
```

`flatMap` applies `rowSlice` to each token ID and concatenates all the results into one flat `Vec`. This is the input to the first linear layer.

This is the actual code from `LanguageModel.forward`:

```
val x = context.flatMap(id => p.E.rowSlice(id))
```

One line. The entire embedding lookup and concatenation.

Why Concatenate Instead of Average?

You might wonder: why concatenate the embeddings (making a longer vector) rather than averaging them (keeping the same size)?

Concatenation preserves **positional information**. The first `embedDim` dimensions of `x` always come from the first context token. The second block comes from the second token. The network can learn to treat these positions differently.

Averaging would collapse this: you'd lose track of which token was in which position. "Cat sat on" and "on sat cat" would produce the same average embedding, even though they're different contexts.

Concatenation keeps the positions distinct. The cost is that `x` is longer (`contextSize × embedDim` rather than just `embedDim`), which means `w1` must be larger. That's a reasonable trade-off.

Chapter Milestone

Let's create a small model, initialize it, and perform the embedding lookup step manually.

```
// In sbt console
import nn.{LanguageModel, ModelConfig}
import linalg.LinearAlgebra

// Create a tiny model config
val cfg = ModelConfig(contextSize = 3, embedDim = 4, hiddenDim = 8, vocabSize = 10)

// Initialize parameters with a fixed seed
val params = LanguageModel.initParams(cfg, seed = 42)

println(s"Embedding matrix E: ${params.E.rows} rows × ${params.E.cols} cols")
// → 10 rows × 4 cols (one row per vocabulary word)

// Look up embedding for token ID 3
val emb3 = params.E.rowSlice(3)
println(s"Embedding for token 3: $emb3")
// → a Vector of 4 doubles (small random values from Xavier init)

// Look up embedding for token ID 7
val emb7 = params.E.rowSlice(7)
println(s"Embedding for token 7: $emb7")
// → a different Vector of 4 doubles

// Simulate a context window: tokens [3, 7, 1]
val context = Vector(3, 7, 1)
val x = context.flatMap(id => params.E.rowSlice(id))
println(s"Concatenated input x: $x")
println(s"Length of x: ${x.length}")
// → length should be 3 × 4 = 12

// Verify: first 4 elements of x should equal emb3
println(s"First 4 of x == emb3? ${x.take(4) == emb3}") // → true
```

When you see that the first 4 elements of `x` match `emb3` exactly, the concatenation step becomes concrete. `x` is literally the three embedding vectors laid end to end.

What You Learned

- An embedding maps a token ID to a dense vector of real numbers: a learned representation

- The embedding matrix `E` has shape `(vocabSize × embedDim)`: row `i` is the embedding for token `i`
 - `rowSlice(i)` extracts row `i` as a `Vec`
 - Xavier uniform initialization ensures weights start at a stable scale
 - `Params` holds all model parameters; it's immutable
 - `ModelConfig` specifies the architectural choices: context size, embedding dimension, hidden dimension, vocab size
 - The forward pass begins by looking up and concatenating embeddings for all context tokens
-

Source Reference

- `src/main/scala/nn/LanguageModel.scala`: `Params`, `ModelConfig`, `initParams`, `xavierUniform`, start of `forward`
-

Up Next

We have `x`: the concatenated embedding vector. Now we need to pass it through the first linear layer and a non-linear activation function to get the hidden layer representation. That's Chapter 7.

Chapter 7: The Hidden Layer: Where the Magic Happens

We have `x`: a vector of concatenated embeddings, length `contextSize × embedDim`. This vector summarizes what the model is currently looking at: the recent context, flattened into a sequence of numbers.

Now we need to *transform* that vector into something more useful. Something that captures higher-level patterns: the kind of pattern that lets the model recognize "cat sat on the" as a context likely to be followed by "mat."

This is the job of the **hidden layer**.

Two Steps in One

The hidden layer does two things in sequence:

1. **A linear transformation:** multiply `x` by a weight matrix `W1` and add a bias vector `b1`
2. **A non-linear activation:** apply a function that "squishes" each element into a bounded range

Together: `a1 = activation(W1 @ x + b1)`

Let's understand each step.

Step 1: The Linear Transformation

$$z1 = W1 @ x + b1$$

- `W1` has shape `(hiddenDim × inputDim)` where `inputDim = contextSize × embedDim`

- `x` has length `inputDim`
- `b1` has length `hiddenDim`
- `z1` has length `hiddenDim`

This is `matVecMul(W1, x)` followed by adding the bias vector `b1`. The result `z1` is the *pre-activation*: the raw output before the squishing function is applied.

What does this transformation do? Each element of `z1` is a different weighted combination of the elements of `x`. The weights in `W1` are what the model learns: they determine *which aspects of the context* each hidden neuron responds to.

You can think of each row of `W1` as a "detector": it's tuned to a particular pattern in the input. After training, some rows might become sensitive to things like "the word at position 2 is a verb" or "positions 0 and 1 form a common bigram." The model discovers these patterns on its own.

The bias `b1` is a small adjustment that lets each neuron shift its response up or down regardless of the input. It's like a baseline activation level.

Step 2: The Activation Function

Here's where non-linearity enters.

After computing `z1 = W1 @ x + b1`, we have a vector of numbers in some arbitrary range. Now we apply an activation function to every element independently.

Without an activation function, multiple linear layers would be equivalent to a single linear layer. No matter how many `W @ x + b` transformations you stack, the result can always be expressed as a single linear transformation. You'd never gain expressive power by adding more layers.

With a non-linear activation function, things you can't express with a single layer become expressible with two layers. The non-linearity is what makes deep networks qualitatively more powerful than shallow ones.

The Tanh Activation

The most common activation in our model is **tanh**: the hyperbolic tangent.

The tanh function squishes any real number into the range $(-1, 1)$:

```
tanh(-∞) → -1  
tanh(0)  → 0  
tanh(+∞) → +1
```

For moderate inputs:

```
tanh(-3) ≈ -0.995  
tanh(-1) ≈ -0.762  
tanh(0)  = 0.000  
tanh(1)  ≈ 0.762  
tanh(3)  ≈ 0.995
```

The curve is smooth (differentiable everywhere: important for backpropagation) and symmetric around zero.

In Scala:

```
def tanhVec(v: Vec): Vec = v.map(tanh)
```

Just apply **tanh** to every element.

Intuition: tanh is a soft version of a switch. For large negative inputs, it returns almost -1. For large positive inputs, almost +1. In between, it interpolates smoothly. Each neuron is like a dimmer switch, ranging from fully-off (-1) to fully-on (+1).

The ReLU Activation

The other option is **ReLU**: Rectified Linear Unit. Much simpler:

```
ReLU(x) = max(0, x)
```

Negative inputs become 0. Positive inputs pass through unchanged.

```
ReLU(-3) = 0
ReLU(-1) = 0
ReLU(0)  = 0
ReLU(1)  = 1
ReLU(3)  = 3
```

```
def relu(v: Vec): Vec = v.map(x => math.max(0.0, x))
```

ReLU has advantages: it's computationally trivial, and it doesn't have the "vanishing gradient" problem that tanh can have for large inputs (where the gradient approaches zero). But it can also produce "dead neurons": neurons that get stuck at 0 and never activate.

For our small models, tanh and ReLU both work fine. We'll default to tanh.

The `linearActivation` Operation

The hidden layer computation (linear transform plus activation) is wrapped in a single backend operation called `linearActivation`:

```
// In CpuBackend:
def linearActivation(W: Matrix, x: Vec, b: Vec, activation: String): (Vec, Vec) =
  val z = vecAdd(matVecMul(W, x), b)
  val a = activation.toLowerCase match
    case "tanh" => tanhVec(z)
    case "relu" => relu(z)
    case _      => throw IllegalArgumentException(s"Unknown activation: $activation")
  (z, a)
```

This returns *both* `z1` (pre-activation) and `a1` (post-activation). We save both because backpropagation needs `z1` to compute the activation gradient.

In the forward pass:

```
val (z1, a1) = backend.linearActivation(p.W1, x, p.b1, activation)
```

One line. The hidden layer is complete.

Tracing the Dimensions

Let's trace the shapes through a concrete example.

Configuration: `contextSize=3, embedDim=4, hiddenDim=8, vocabSize=10`

- Input `x` : length `3 × 4 = 12`
- `W1` : shape `(8 × 12)` : transforms 12-dimensional input to 8-dimensional output
- `b1` : length `8`
- `z1 = W1 @ x + b1` : length `8`
- `a1 = tanh(z1)` : length `8` (same shape, just squished)

The hidden layer compressed the 12-dimensional input to an 8-dimensional representation. This compression forces the model to find a compact summary of the context.

Visualizing the Hidden Layer

Here's a concrete picture of what's happening. Suppose after some training, `W1` has learned something meaningful.

Row 0 of `W1` might be tuned to detect the pattern "the last token was a verb." When `x` encodes a context ending in a verb, `dot(W1_row0, x)` will be large and positive. After `tanh`, `a1[0]` will be close to 1.0.

Row 3 might be tuned to detect "the context contains a name at position 0." When the first token is a proper noun, `a1[3]` will activate.

These "detectors" are learned, not designed. We just provide the architecture and the data; the training process discovers what patterns are useful for next-word prediction.

The hidden layer's 8-dimensional output vector `a1` is a compact encoding of what the model "thinks" about the current context: a summary of which patterns it detected.

Chapter Milestone

Let's trace the full hidden layer computation manually for a specific input.

```
// In sbt console
import nn.{LanguageModel, ModelConfig}
import linalg.{LinearAlgebra, Matrix}

// Same small model from Chapter 6
val cfg = ModelConfig(contextSize = 3, embedDim = 4, hiddenDim = 8, vocabSize = 10)
val params = LanguageModel.initParams(cfg, seed = 42)

// Build input x from context [3, 7, 1]
val context = Vector(3, 7, 1)
val x = context.flatMap(id => params.E.rowSlice(id))
println(s"x (length ${x.length}): $x")

// Step 1: linear transformation z1 = W1 @ x + b1
val z1_raw = LinearAlgebra.matVecMul(params.W1, x)
val z1 = LinearAlgebra.vecAdd(z1_raw, params.b1)
println(s"z1 (pre-activation, length ${z1.length}):")
println(z1.map(v => f"$v%.3f").mkString(" "))

// Step 2: apply tanh
val a1 = z1.map(x => math.tanh(x))
println(s"\na1 (post-activation, length ${a1.length}):")
println(a1.map(v => f"$v%.3f").mkString(" "))

// Verify all values of a1 are in (-1, 1)
val allInRange = a1.forall(v => v > -1.0 && v < 1.0)
println(s"\nAll a1 values in (-1, 1): $allInRange") // → true

// See which neurons are most activated
val activations = a1.zipWithIndex.sortBy(_._1)
println(s"\nMost activated hidden neurons: ${activations.take(3).map { case (v, i) =>
s"neuron $i: ${f"$v%.3f"}" }}")
```

The key observation: `z1` values can be any real numbers (possibly large positive or negative). After `tanh`, every value in `a1` is strictly between -1 and 1. The squishing happened.

What You Learned

- The hidden layer applies a linear transformation followed by a non-linear activation:
`a1 = activation(W1 @ x + b1)`
 - `W1` has shape `(hiddenDim × inputDim)` : each row is a "detector" for some pattern in the input
 - `b1` is a bias vector that shifts activations up or down
 - **tanh** squishes values to `(-1, 1)` smoothly
 - **ReLU** passes positive values unchanged and zeros negatives: `max(0, x)`
 - Non-linearity is essential: without it, stacking layers would accomplish nothing
 - `linearActivation` returns both `z1` (pre-activation) and `a1` (post-activation) because both are needed later
-

Source Reference

- `src/main/scala/linalg/LinearAlgebra.scala` : `tanhVec` , `relu` , `tanhGrad` , `reluGrad`
 - `src/main/scala/compute/CpuBackend.scala` : `linearActivation`
 - `src/main/scala/nn/LanguageModel.scala` : the `(z1, a1)` computation in `forward`
-

Up Next

We have `a1` : a hidden representation of the context. Now we need to turn that into a probability distribution over the vocabulary: what are the chances of each word coming next? Chapter 8 covers the output layer and the softmax function.

Chapter 8: The Output Layer and Softmax

We have `a1`: the hidden layer's 8-dimensional summary of the context. Now we need to turn this into something actionable: **a probability distribution over every word in the vocabulary**.

"Probability distribution" means: for each of the 500 (or 3,000, or however many) words in the vocabulary, what's the probability that it's the next word? Every probability is between 0 and 1, and they all sum to 1.

Getting from `a1` to that distribution requires two steps: a second linear transformation (to produce *logits*), and a function called **softmax** (to turn logits into probabilities).

Step 1: The Output Linear Layer

```
logits = W2 @ a1 + b2
```

- `W2` has shape `(vocabSize × hiddenDim)`
- `a1` has length `hiddenDim`
- `b2` has length `vocabSize`
- `logits` has length `vocabSize`

One output value per word in the vocabulary. These are called **logits**: raw scores, not yet probabilities. A logit can be any real number: positive, negative, large, small. Higher means more likely, but the scale is arbitrary.

In Scala:

```
val logits = backend.vecAdd(backend.matVecMul(p.W2, a1), p.b2)
```

After this line, `logits` is a vector of length `vocabSize`, with one score per word.

Step 2: Softmax

Logits give us a ranking, but not probabilities. We need to convert them.

Softmax turns a vector of arbitrary real numbers into a proper probability distribution:

```
softmax(logits)[i] = exp(logits[i]) / sum(exp(logits[j]) for all j)
```

In words: exponentiate every logit, then divide each by the total. The result is non-negative (since `exp` is always positive) and sums to 1 (since we divide by the total).

For a tiny example with 3 words and logits `[2.0, 1.0, 0.5]` :

```
exp(2.0) = 7.389
exp(1.0) = 2.718
exp(0.5) = 1.649

total = 7.389 + 2.718 + 1.649 = 11.756

probs = [7.389/11.756, 2.718/11.756, 1.649/11.756]
       = [0.629, 0.231, 0.140]
```

The highest-scoring word ("word 0" with logit 2.0) gets 62.9% probability. Word 1 gets 23.1%. Word 2 gets 14.0%.

The voting machine analogy: Softmax is like an election where each candidate gets `exp(score)` votes. We then report the fraction of total votes each candidate received. The exponentiation amplifies differences: a candidate with score 3 gets about 7× more votes than a candidate with score 1, even though the logit difference is only 2.

Numerical Stability: The Max Subtraction Trick

There's a practical problem with the naive softmax formula: `exp(logit)` can overflow for large logits. If a logit is 1000, `exp(1000)` is astronomically large: larger than any floating point number can represent. You get `Infinity`, and everything breaks.

The fix is elegant: before exponentiating, subtract the maximum logit from all logits.

```
stabilized_logits[i] = logits[i] - max(logits)
probs[i] = exp(stabilized_logits[i]) / sum(exp(stabilized_logits[j]))
```

The maximum stabilized logit is now 0, so $\exp(0) = 1$. All other values are negative, so their exponents are in $(0, 1)$. No overflow.

Does this change the answer? No. Subtracting a constant from all logits before exponentiating doesn't change the ratios:

$$\begin{aligned} \exp(a - c) / \sum(\exp(b - c)) &= \exp(a)/\exp(c) / (\sum(\exp(b))/\exp(c)) \\ &= \exp(a) / \sum(\exp(b)) \end{aligned}$$

The $\exp(c)$ terms cancel. We get exactly the same probabilities: just computed without numerical disaster.

```
def softmaxStable(logits: Vec): Vec =
  require(logits.nonEmpty, "softmax requires non-empty vector")
  val maxLogit = logits.max
  val shiftedExp = logits.map(x => exp(x - maxLogit))
  val denom = shiftedExp.sum
  shiftedExp.map(_ / denom)
```

The Top-K Predictions

After softmax, we have a probability distribution. To see what the model thinks is most likely, we sort by probability and take the top k :

```
def argTopK(v: Vec, k: Int): Vector[(Int, Double)] =
  v.zipWithIndex
    .map { case (value, idx) => (idx, value) }
    .sortBy { case (_, value) => -value }
    .take(math.max(k, 1))
    .toVector
```


This returns a list of `(index, probability)` pairs sorted by probability, highest first. Index `i` is the token ID; the probability tells you how confident the model is.

For example, calling `argTopK(probs, k=3)` might return:

```
Vector((42, 0.312), (7, 0.189), (156, 0.094))
```

meaning: token 42 has 31.2% probability, token 7 has 18.9%, token 156 has 9.4%. The model's best guess is token 42.

Putting the Output Layer Together

Here's the complete output layer in the forward pass:

```
val logits = backend.vecAdd(backend.matVecMul(p.W2, a1), p.b2)
val probs  = backend.softmaxStable(logits)
```

Two lines. From hidden representation `a1` to a probability distribution over the entire vocabulary.

The full forward pass, all together:

```
val x      = context.flatMap(id => p.E.rowSlice(id))      // embedding lookup +
concat
val (z1, a1) = backend.linearActivation(p.W1, x, p.b1, activation) // hidden layer
val logits = backend.vecAdd(backend.matVecMul(p.W2, a1), p.b2)    // output layer
val probs  = backend.softmaxStable(logits)                      // probabilities
```

Four lines. That's the entire neural network forward pass.

What the Model Produces Right Now

At this point, we haven't trained the model. `w2` (and all other weights) contain small random numbers from Xavier initialization.

What does `probs` look like for a freshly initialized model?

Roughly uniform: each word gets approximately $1/\text{vocabSize}$ probability. With 500 words, each gets about 0.2%. The model has no preferences yet: it considers every word equally likely.

This is expected and correct. Training will change these probabilities to reflect real patterns in the data.

Chapter Milestone

Let's complete the output layer and see the first model prediction.

```
// In sbt console
import nn.{LanguageModel, ModelConfig}
import linalg.LinearAlgebra
import data.*
import scala.io.Source

// Load vocab and build a tiny model
val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 100)

val cfg = ModelConfig(contextSize = 3, embedDim = 8, hiddenDim = 16, vocabSize =
vocab.size)
val params = LanguageModel.initParams(cfg, seed = 42)

// Get the first context window from the corpus
val ids = TextPipeline.tokensToIds(tokens, vocab)
val examples = TextPipeline.buildExamples(ids, contextSize = 3)
val firstEx = examples.head

// Run the full forward pass
val cache = LanguageModel.forward(params, firstEx.context)

// Show top-5 predicted tokens
val topK = LinearAlgebra.argTopK(cache.probs, k = 5)
println("Top 5 predictions (untrained model):")
topK.foreach { case (id, prob) =>
  println(f"  '${vocab.toToken(id)}%-12s'  prob: ${prob * 100}%0.2f%%")
}

// Show the actual correct answer
println(s"\nActual next word: '${vocab.toToken(firstEx.target)}'")
println(s"Its probability: ${cache.probs(firstEx.target) * 100}%0.2f%%")

// Check: all probabilities sum to 1
println(f"\nSum of all probs: ${cache.probs.sum}%0.6f")  // → very close to 1.0
```

Look at the output. The probabilities will be near-uniform (the model doesn't know anything yet. The actual next word probably won't be in the top 5. That's fine) by the end of Part 3, training will change this dramatically.

The important thing: everything *works*. The shapes are right, the probabilities sum to 1, the plumbing connects. The model is making predictions: just bad ones.

What You Learned

- `logits = W2 @ a1 + b2` produces one raw score per vocabulary word
 - **Softmax** converts logits to a probability distribution: exponentiate, then divide by the sum
 - The **max subtraction trick** prevents numerical overflow in softmax: subtract `max(logits)` before exponentiating
 - `argTopK` finds the `k` words with the highest probability: the model's top guesses
 - A freshly initialized model produces approximately uniform probabilities: it doesn't know anything yet
 - Training will adjust the weights so that the distribution peaks on the correct next words
-

Source Reference

- `src/main/scala/linalg/LinearAlgebra.scala` : `softmaxStable` , `argTopK`
 - `src/main/scala/nn/LanguageModel.scala` : logits and probs computation in `forward`
-

Up Next

We now have the full forward pass: embeddings → hidden layer → output layer → probabilities. Chapter 9 assembles all the pieces into the complete `LanguageModel.forward` function, introduces the `ForwardCache` , and ties up loose ends like Xavier initialization.

Chapter 9: Assembling the Forward Pass

We've built the forward pass piece by piece over the last three chapters: - Chapter 6: embedding lookup and concatenation → `x` - Chapter 7: hidden layer linear transform and activation → `z1`, `a1` - Chapter 8: output layer and softmax → `logits`, `probs`

Now let's assemble the complete picture: the full `forward` function, the `ForwardCache` that stores every intermediate result, and the `ModelConfig` that specifies the architecture.

The Complete Forward Pass

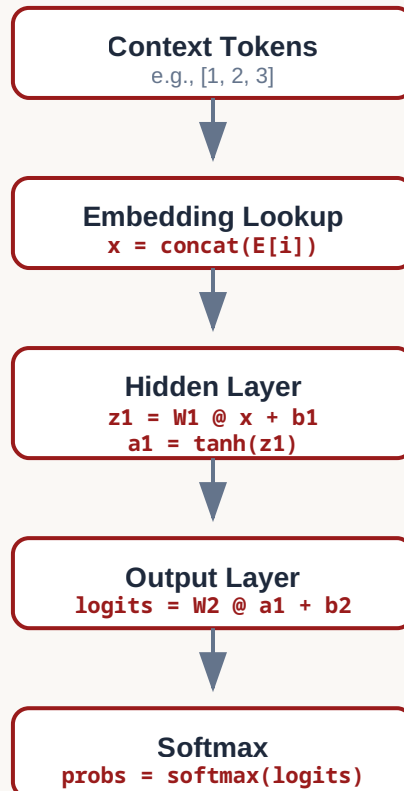


Figure: The complete forward pass architecture of the language model.

Here is the full `forward` function from `LanguageModel.scala` :

```
def forward(
  p: Params,
  context: Vector[Int],
  activation: String = "tanh",
  backend: ComputeBackend = CpuBackend.Default
): ForwardCache =
  require(context.nonEmpty, "context cannot be empty")

  val x      = context.flatMap(id => p.E.rowSlice(id))
  val (z1, a1) = backend.linearActivation(p.W1, x, p.b1, activation)
  val logits = backend.vecAdd(backend.matVecMul(p.W2, a1), p.b2)
  val probs  = backend.softmaxStable(logits)

  ForwardCache(x = x, z1 = z1, a1 = a1, logits = logits, probs = probs, context =
context)
```

Four lines of actual computation, wrapped in a function that takes parameters (`p`) and a context (`Vector[Int]`) and returns a `ForwardCache`.

Simple. Let's look at each piece.

The `require` Statement

```
require(context.nonEmpty, "context cannot be empty")
```

`require` is Scala's built-in precondition mechanism. If the condition is false, it throws an `IllegalArgumentException` with the given message. It's a contract: this function promises to work correctly given a non-empty context, and it refuses to proceed with an invalid input.

You'll see `require` throughout the codebase, protecting every function's assumptions. For a beginner, this is a great habit: state what you need before you need it. When something goes wrong, the error message points directly at the violated assumption.

The ForwardCache

```
final case class ForwardCache(  
  x: Vec,  
  z1: Vec,  
  a1: Vec,  
  logits: Vec,  
  probs: Vec,  
  context: Vector[Int]  
)
```

Why save all these intermediate values? Two reasons:

- 1. Backpropagation needs them.** When we run the backward pass (Chapter 12), we'll need `z1` to compute the activation gradient, and `a1` to compute the gradient of `w2`. The forward pass is run first, and we cache everything so backward pass doesn't have to recompute it.
- 2. Clarity.** The `ForwardCache` makes the function's output explicit: these are all the things we computed, named and bundled together. Contrast this with returning a tuple of 5 anonymous values: that would be far harder to read and use.

The `context` is also stored in the cache because backpropagation needs to know *which token IDs* to scatter embedding gradients to. More on this in Chapter 12.

The ComputeBackend Parameter

```
backend: ComputeBackend = CpuBackend.Default
```

Every operation in the forward pass goes through a *backend*: an abstraction over where the computation runs. `CpuBackend.Default` is the standard CPU implementation. If you have an Apple Silicon Mac and have compiled the Metal library, you can pass a `MetalBackend` instead to run on the GPU.

For the core chapters of this book, we'll always use the CPU backend. Extension Chapter A covers the GPU backend.

The default value (= `CpuBackend.Default`) means you can call `forward` without specifying a backend and get the CPU version automatically:

```
// Equivalent:  
LanguageModel.forward(params, context)  
LanguageModel.forward(params, context, backend = CpuBackend.Default)
```

Shapes at a Glance

Let's trace every tensor's shape for `ModelConfig(contextSize=3, embedDim=16, hiddenDim=64, vocabSize=500)` :

VARIABLE	SHAPE/LENGTH	DESCRIPTION
<code>context</code>	<code>3</code>	Input token IDs
<code>E</code>	<code>500 × 16</code>	Embedding matrix
<code>x</code>	<code>48</code>	Concatenated embeddings (3 × 16)
<code>W1</code>	<code>64 × 48</code>	Hidden layer weights
<code>b1</code>	<code>64</code>	Hidden layer bias
<code>z1</code>	<code>64</code>	Pre-activation hidden values
<code>a1</code>	<code>64</code>	Post-activation hidden values
<code>W2</code>	<code>500 × 64</code>	Output layer weights
<code>b2</code>	<code>500</code>	Output layer bias
<code>logits</code>	<code>500</code>	One raw score per vocab word
<code>probs</code>	<code>500</code>	One probability per vocab word

Data flows from 3 integers (token IDs) to a 500 -element probability distribution. The bottleneck is the 64-dimensional hidden layer: it forces the model to compress the 48-dimensional input into a 64-dimensional summary before expanding back to 500 dimensions.

Default Parameters in Scala

Notice `activation: String = "tanh"`: a **default parameter**. You can call `forward` without specifying the activation, and it defaults to tanh. Or you can override it:

```
// Uses tanh (default)
LanguageModel.forward(params, context)

// Uses relu
LanguageModel.forward(params, context, activation = "relu")
```

Default parameters reduce the amount you need to remember when calling functions. The most common case is handled automatically; unusual cases require explicit specification.

The Full Pipeline: From Text to Prediction

Let's put everything from Part 1 and Part 2 together. Here's what it looks like when the pieces connect:

```
Raw text
  ↓ TextPipeline.tokenize
Vector[String] (tokens)
  ↓ TextPipeline.buildVocab
Vocab
  ↓ TextPipeline.tokensToIds
Vector[Int] (token IDs)
  ↓ TextPipeline.buildExamples
Vector[Example] (context, target pairs)
  ↓ LanguageModel.initParams
Params (randomly initialized weights)
  ↓ LanguageModel.forward(params, example.context)
ForwardCache (all intermediate values, including probs)
  ↓ LinearAlgebra.argmax(cache.probs, k=5)
Top-5 predictions
```

This is the complete system: every module we've built, connected. The model is running. It's making predictions. They're garbage predictions, because the weights are random. But Part 3 changes that.

Chapter Milestone

Let's run a complete forward pass and inspect every intermediate value.

```

// In sbt console
import nn.{LanguageModel, ModelConfig}
import linalg.LinearAlgebra
import data.*
import scala.io.Source

// Set up the full pipeline
val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 200)

val cfg = ModelConfig(
  contextSize = 3,
  embedDim = 8,
  hiddenDim = 16,
  vocabSize = vocab.size
)
val params = LanguageModel.initParams(cfg, seed = 42)

// Run forward on the first example
val ids = TextPipeline.tokensToIds(tokens, vocab)
val examples = TextPipeline.buildExamples(ids, contextSize = 3)
val ex = examples.head

println(s"Context words: ${ex.context.map(vocab.toToken)}")
println(s"Target word:   '${vocab.toToken(ex.target)}'")

val cache = LanguageModel.forward(params, ex.context)

// Inspect the cache
println(s"\n--- ForwardCache contents ---")
println(s"x      length: ${cache.x.length}      (contextSize × embedDim = 3 × 8)")
println(s"z1     length: ${cache.z1.length}      (hiddenDim)")
println(s"a1     length: ${cache.a1.length}      (hiddenDim)")
println(s"logits length: ${cache.logits.length} (vocabSize)")
println(s"probs  length: ${cache.probs.length} (vocabSize)")

println(s"\nprobs sum: ${f"${cache.probs.sum}%.8f"}") // → very close to 1.0

println(s"\nTarget word probability: ${f"${cache.probs(ex.target) * 100}%.3f"}%")

val top5 = LinearAlgebra.argTopK(cache.probs, 5)
println(s"\nTop 5 predicted next words:")
top5.foreach { case (id, p) =>
  val marker = if id == ex.target then " ← CORRECT" else ""
  println(f"  '${vocab.toToken(id)}%-15s'  ${p * 100}%.2f%%$marker")
}

```

The correct answer (target word) almost certainly won't be in the top 5: the model is randomly initialized. But see how low its probability is compared to the top predictions. That gap is what training will close.

What You Learned

- `LanguageModel.forward` is 4 lines of computation, returning a `ForwardCache`
 - `require` states function preconditions explicitly: a good defensive programming habit
 - `ForwardCache` stores all intermediate values (x, z1, a1, logits, probs, context) because backpropagation needs them
 - The `ComputeBackend` parameter abstracts where computation runs (CPU vs GPU); defaults to CPU
 - Default parameters (= `"tanh"`) reduce boilerplate for common cases
 - A freshly initialized model makes uniform, random predictions: training is what makes it useful
-

Source Reference

- `src/main/scala/nn/LanguageModel.scala` : `forward` , `ForwardCache` , `ModelConfig` , `initParams`
 - `src/main/scala/compute/ComputeBackend.scala` : the backend abstraction
 - `src/main/scala/compute/CpuBackend.scala` : the default CPU implementation
-

Up Next

The forward pass is complete. But we haven't answered the most important question: *how wrong is the model?* Chapter 10 introduces the loss function and perplexity: the tools for measuring the distance between the model's predictions and the truth.

Chapter 10: Loss: Measuring How Wrong You Are

The forward pass produces a probability distribution. For a given context, the model assigns a probability to every word in the vocabulary. The higher the probability of the correct next word, the better the model is doing.

But "better" needs to be a number. We need a single number that tells us, precisely, how wrong the model is. We call this number the **loss**. The goal of training is to minimize the loss.

This chapter introduces **cross-entropy loss** (the standard way to measure error in a classification problem) and **perplexity**, a more intuitive quantity derived from the loss.

What Would a Perfect Model Look Like?

Before defining loss, let's think about what we want.

Suppose the correct next word is "cat" (token ID 42). A perfect model would assign probability 1.0 to token 42 and probability 0.0 to everything else.

A random model (uniformly distributed) would assign approximately $1/500 = 0.2\%$ to token 42 (assuming a vocab of 500).

A trained model might assign 15% to token 42: not perfect, but much better than random.

We need a function that:

- Returns 0 (or near 0) when the model assigns probability 1.0 to the correct word
- Returns a large positive number when the model assigns very low probability to the correct word
- Increases smoothly as the model gets worse

Cross-entropy loss does exactly this.

Cross-Entropy Loss

The cross-entropy loss for one example is:

```
loss = -log(probs[target])
```

That's it. Take the probability the model assigned to the correct word, and return the negative log of it.

Let's see what this gives us for various probabilities:

MODEL'S PROBABILITY FOR CORRECT WORD	LOSS
1.00 (perfect)	0.00
0.90 (excellent)	0.105
0.50 (good)	0.693
0.10 (poor)	2.303
0.01 (bad)	4.605
0.001 (terrible)	6.908

The relationship is non-linear: going from 50% to 1% confidence (which feels like a big drop) increases the loss by about 6× the amount that going from 100% to 90% does. High-confidence wrong answers are penalized heavily.

The teacher grading analogy: Imagine a multiple-choice test where students don't just pick an answer: they assign a confidence percentage to each option. The grader uses this scoring rule: your grade is $-\log(\text{confidence you gave the correct answer})$.

- If you said "I'm 90% sure it's A" and it was A, you get grade 0.105 (nearly perfect).
- If you said "I'm 1% sure it's A" and it was A, you get grade 4.605 (failing).

The logarithm means: being wrong with *high confidence* is far more costly than being wrong with low confidence. This pushes the model toward honest probability estimates.

Implementation

```
def crossEntropy(probs: Vec, target: Int, epsilon: Double = 1e-12): Double =  
  require(target >= 0 && target < probs.length, s"target index out of range: $target")  
  -log(math.max(probs(target), epsilon))
```

The `epsilon` prevents `log(0)` from returning negative infinity. If the model assigns exactly 0 probability to the correct word (which shouldn't happen with softmax, but floating-point arithmetic can produce tiny numbers), we floor it at `1e-12`.

Using this:

```
def lossFromCache(cache: ForwardCache, target: Int, ...): Double =  
  backend.crossEntropy(cache.probs, target)
```

Perplexity

Cross-entropy loss is useful for math (especially calculus) but not very intuitive for humans. What does a loss of 4.3 *mean*?

Perplexity gives a more interpretable quantity:

```
perplexity = exp(loss)
```

Or equivalently (since $\text{loss} = -\log(\text{prob})$):

```
perplexity = 1 / probs[target]
```

Perplexity measures how many words the model considers equally plausible as the next word.

Some examples: - Loss = 0.0 → Perplexity = 1.0 (perfect: one word is certain) - Loss = 2.303 → Perplexity $\approx$ 10 (model thinks ~10 words are equally likely) - Loss = 6.215 → Perplexity = 500 (model is completely random over 500 words)

For a randomly initialized model with `vocabSize = 500`, you'd expect perplexity near 500. After training on a small corpus, perplexity in the range 50–150 is reasonable, depending on how much data you have and how complex the text is.

The mental model: "My language model has perplexity 73" means "for the typical context, my model is as uncertain as if it were choosing uniformly among 73 equally plausible next words."

In Scala:

```
object Metrics:
  def perplexity(loss: Double): Double = math.exp(loss)

  def meanLoss(examples: Vector[Example], params: Params, cfg: ModelConfig, backend:
ComputeBackend): Double =
    if examples.isEmpty then 0.0
    else
      val total = examples.map { ex =>
        val cache = LanguageModel.forward(params, ex.context, cfg.activation, backend)
        LanguageModel.lossFromCache(cache, ex.target, backend)
      }.sum
      total / examples.length
```

`meanLoss` runs the forward pass on every example and averages the losses. This is how we compute the validation loss: run on all validation examples and take the mean.

The Training Signal

The loss is more than a measurement: it's the *training signal*. Everything we'll do in Part 3 is aimed at finding the weights that minimize the average loss on the training set.

The connection: if we know the loss, and if we know how much each weight contributed to the loss, we can adjust every weight in a direction that reduces the loss. This is backpropagation: the topic of Chapters 11 and 12.

But first, we need to understand what "how much each weight contributed to the loss" means mathematically. That's the concept of a **gradient**, coming up in Chapter 11.

Chapter Milestone

Let's compute loss and perplexity for our untrained model and understand what those numbers mean.

```

// In sbt console
import nn.{LanguageModel, ModelConfig}
import eval.Metrics
import data.*
import scala.io.Source

// Set up
val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 200)

val cfg = ModelConfig(contextSize = 3, embedDim = 8, hiddenDim = 16, vocabSize =
vocab.size)
val params = LanguageModel.initParams(cfg, seed = 42)

val ids = TextPipeline.tokensToIds(tokens, vocab)
val examples = TextPipeline.buildExamples(ids, contextSize = 3)
val (train, validation) = TextPipeline.splitDeterministic(examples, 0.9, 42)

// Compute loss on first example
val ex = examples.head
val cache = LanguageModel.forward(params, ex.context)
val loss = LanguageModel.lossFromCache(cache, ex.target)
val perp = Metrics.perplexity(loss)

println(f"Loss on first example: $loss%.4f")
println(f"Perplexity:           $perp%.1f")
println(f"Vocab size:           ${vocab.size}")
println(f"Probability assigned to correct word: ${cache.probs(ex.target) * 100}%.3f%
%")

// Compute mean loss over a small sample
val sample = examples.take(50)
val sampleLoss = sample.map { ex =>
  val c = LanguageModel.forward(params, ex.context)
  LanguageModel.lossFromCache(c, ex.target)
}.sum / sample.length
val samplePerp = Metrics.perplexity(sampleLoss)
println(f"\nMean loss over 50 examples: $sampleLoss%.4f")
println(f"Mean perplexity:           $samplePerp%.1f")
println(s"\nExpected perplexity for random model: ~${vocab.size}")
println(s"Our model is close to this: it hasn't learned anything yet.")
println(s"After training, perplexity should drop significantly.")

```

When you run this, you should see perplexity close to `vocab.size`. Write down this number: it's your baseline. After training in Part 3, you'll compare and see how much the model improved.

What You Learned

- **Loss** is a single number measuring how wrong the model is: `loss = -log(probs[target])`
 - Cross-entropy loss is 0 for a perfect prediction and increases as the correct word's probability decreases
 - The `epsilon` floor (`1e-12`) prevents `log(0)`
 - **Perplexity** = `exp(loss)` : more interpretable: "how many words does the model think are equally likely?"
 - A random model has perplexity $\approx$ `vocabSize`; a trained model should have much lower perplexity
 - `Metrics.meanLoss` computes the average loss across a set of examples: used for train/val evaluation
 - The loss is the training signal: everything in Part 3 is about minimizing it
-

Source Reference

- `src/main/scala/eval/Metrics.scala` : `perplexity` , `meanLoss`
 - `src/main/scala/linalg/LinearAlgebra.scala` : `crossEntropy`
 - `src/main/scala/nn/LanguageModel.scala` : `lossFromCache`
-

Up Next

Part 2 is complete. We can run the forward pass and measure how wrong the model is.

Part 3 is where the model starts learning. Chapter 11 introduces **gradients**: the mathematical concept that tells us which direction to adjust each weight to reduce the loss.

Chapter 11: Gradients: Which Way Is Downhill?

The model makes a prediction. We measure how wrong it is (the loss). Now what?

To improve the model, we need to adjust its weights: the numbers in E , W_1 , b_1 , W_2 , b_2 . But there are tens of thousands of these numbers. How do we know which ones to change, and by how much?

The answer is **gradients**. This chapter explains what gradients are, why they point downhill, and how to compute the first and most important one in our network.

The Loss Landscape

Imagine every possible setting of the model's weights as a point in a very high-dimensional space. For each point (each set of weights), you can evaluate the loss. This creates a **loss landscape**: a surface where the height at any point is the loss value for those weights.

We want to find the lowest point in this landscape: the weights that produce the minimum loss.

We can't see the whole landscape (it has tens of thousands of dimensions). But we can do something useful at any point: we can feel the slope of the ground under our feet.

The slope in each direction tells us: if I move in this direction, does the loss go up or down? And how steeply?

A **gradient** is a vector that encodes this information for every weight simultaneously. The i -th element of the gradient says: "if you increase weight i by a tiny amount, the loss increases by this much."

To reduce the loss, move opposite to the gradient: in the direction of steepest descent. This is exactly what we'll do: subtract (a fraction of) the gradient from the weights.

The Foggy Valley Metaphor

Picture yourself lost in a fog, trying to find the bottom of a valley. You can't see the valley: the fog is too thick. But you can feel the slope of the ground under your feet.

The strategy: always take a step downhill. Feel the slope, step in the direction that goes down, repeat.

This is **gradient descent**. The gradient is the slope. Each training step is one downhill step. After many steps, you converge toward the bottom.

Three things can go wrong: 1. **Step too large:** you overshoot the bottom and land on the other side 2. **Step too small:** you converge very slowly 3. **Local minimum:** you get stuck in a small dip that isn't the deepest valley

For now, don't worry about these problems. They're real, but solvable. The key insight is the mechanism: the gradient tells you the local slope, and you step downhill.

The Gradient of the Loss with Respect to the Logits

Rather than deriving every gradient from scratch, let's focus on the first one (the gradient of the loss with respect to the logits) because it has a beautiful, simple form.

Recall: $\text{probs} = \text{softmax}(\text{logits})$ - $\text{loss} = -\log(\text{probs}[\text{target}])$

We want: "how does the loss change when we change each logit?"

The answer (derived from calculus, but we'll just state and verify it):

```
dLoss/dLogits[i] = probs[i] - (1 if i == target else 0)
```

In words: the gradient is just `probs[i] - 1` for the target word, and `probs[i]` for every other word.

Let's see why this makes sense intuitively.

Suppose `target = 42` and `probs[42] = 0.3` (the model is 30% confident in the right answer). Then: - `dLogits[42] = 0.3 - 1 = -0.7` (a negative gradient. Increasing logit 42 would increase the probability of the correct word (which reduces loss). So the gradient is negative) the loss *decreases* when we push logit 42 up. - `dLogits[7] = 0.2 - 0 = 0.2` (if word 7 has 20% probability): a positive gradient. Word 7 is stealing probability from the correct word. Reducing logit 7 would reduce its probability and push more probability toward word 42.

The gradient is saying: "push the correct word's logit up, push everyone else's down, in proportion to how much probability they're stealing."

Implementation

```
val dLogits = cache.probs.indices.map { i =>
  val t = if i == target then 1.0 else 0.0
  cache.probs(i) - t
}.toVector
```

This is one of the cleaner moments in the codebase. The gradient is computed in one pass over the probabilities: subtract 1 from the target index, leave everything else alone.

Let's verify with a tiny example:

```
probs    = [0.10, 0.60, 0.20, 0.10]
target   = 2
dLogits  = [0.10-0, 0.60-0, 0.20-1, 0.10-0]
          = [0.10, 0.60, -0.80, 0.10]
```

- Word 2 (the correct one, probability 20%) has a large negative gradient: `-0.80`. The model is underconfident in the right answer; we'll push its logit up.

- Word 1 (probability 60%) has a large positive gradient: `+0.60` . It's stealing the most probability; we'll push its logit down.
- Words 0 and 3 (probability 10% each) have small positive gradients.

The magnitudes are proportional to the probabilities: words that got more probability get pushed down harder.

The Gradient Has the Right Shape

Notice: `dLogits` is a vector with the same length as `logits` : one gradient value per logit. This is the pattern throughout backpropagation: **the gradient of the loss with respect to a tensor has the same shape as that tensor**.

`dLogits` has shape `(vocabSize,)` because `logits` has shape `(vocabSize,)` . Later: - `dW2` has shape `(vocabSize × hiddenDim)` because `W2` has that shape - `dW1` has shape `(hiddenDim × inputDim)` because `W1` has that shape - `dE` has shape `(vocabSize × embedDim)` because `E` has that shape

This shape-correspondence is a useful sanity check: if your gradient has the wrong shape, something went wrong.

The `Grads` Case Class

All gradients are bundled in a `Grads` case class that mirrors `Params` :

```
final case class Grads(dE: Matrix, dW1: Matrix, db1: Vec, dW2: Matrix, db2: Vec)
```

`Grads` has exactly the same structure as `Params` . This makes the update step simple: each weight is adjusted by its corresponding gradient.

Chapter Milestone

Let's compute `dLogits` for a real example and inspect it.


```

// In sbt console
import nn.{LanguageModel, ModelConfig}
import data.*
import scala.io.Source

val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 100)

val cfg = ModelConfig(contextSize = 3, embedDim = 8, hiddenDim = 16, vocabSize =
vocab.size)
val params = LanguageModel.initParams(cfg, seed = 42)

val ids = TextPipeline.tokensToIds(tokens, vocab)
val ex = TextPipeline.buildExamples(ids, contextSize = 3).head

val cache = LanguageModel.forward(params, ex.context)

// Compute dLogits manually
val target = ex.target
val dLogits = cache.probs.indices.map { i =>
  val t = if i == target then 1.0 else 0.0
  cache.probs(i) - t
}.toVector

// Inspect
println(s"Target token: '${vocab.toToken(target)}' (ID $target)")
println(f"Target prob: ${cache.probs(target) * 100}%.2f%%")
println(f"dLogits[target] = ${dLogits(target)}%.4f (should be prob - 1 = $
{cache.probs(target) - 1}%.4f)")

// Verify: dLogits should sum to zero (they always do: conservation of gradient)
val sum = dLogits.sum
println(f"\nSum of dLogits: $sum%.8f (should be ~0.0)")

// Find the logit with the most negative gradient (the one we should push up most)
val mostNegative = dLogits.zipWithIndex.minBy(_._1)
println(s"\nMost negative gradient: dLogits[${mostNegative._2}] = ${f"$
{mostNegative._1}%.4f"}")
println(s"This is the target token (${mostNegative._2 == target}): we should push it
up")

// Find the logit with the most positive gradient (competing word)
val mostPositive = dLogits.zipWithIndex.maxBy(_._1)
println(s"\nMost positive gradient: dLogits[${mostPositive._2}] = ${f"$
{mostPositive._1}%.4f"}")

```

```
println(s"Token: '${vocab.toToken(mostPositive._2)}': the model is most wrong about  
this word stealing probability")
```

Notice that `dLogits.sum ≈ 0.0`. This is always true: the sum of `probs[i] - indicator[i]` over all `i` is `sum(probs) - 1 = 1 - 1 = 0`. It's a useful sanity check.

What You Learned

- A **gradient** measures how the loss changes when each parameter changes slightly
 - We want to move weights in the direction that *decreases* the loss: opposite the gradient
 - The gradient of the loss with respect to the logits is: `dLogits[i] = probs[i] - (1 if i==target else 0)`
 - This gradient naturally pushes the correct word's logit up and competing words' logits down
 - The `Grads` case class mirrors `Params`: every parameter has a corresponding gradient of the same shape
 - `dLogits.sum = 0` always: a useful sanity check
-

Source Reference

- `src/main/scala/nn/LanguageModel.scala`: `backward` function (specifically the `dLogits` computation)
-

Up Next

We have `dLogits`. Now we need to propagate this gradient backwards through every layer to get gradients for `w2`, `b2`, `w1`, `b1`, and `E`. Chapter 12 builds the complete backward pass.

Chapter 12: Backpropagation: Running the Chain Backwards

We have `dLogits` : the gradient of the loss with respect to the output logits. Now we need to trace that gradient backwards through the network, computing a gradient for every parameter.

This process is called **backpropagation** (or backprop). It's the algorithm that makes training neural networks tractable.

The Chain Rule: Gradients Flow Backwards

The core principle of backpropagation is the **chain rule** from calculus. The intuitive version:

If A affects B and B affects C, then how A affects C can be computed from how A affects B and how B affects C.

In our network, the loss depends on `logits` , which depend on `w2` and `a1` , which depend on `w1` and `x` , which depends on `E` . A change in any weight propagates forward through the network to affect the loss.

Backpropagation runs this in reverse: we start with the gradient of the loss with respect to the last thing computed (`logits`), and work backwards, computing gradients for each layer.

At each layer, the local rule is: the gradient at the input is the gradient at the output, transformed by the layer's local derivative. Gradients flow backwards, accumulating information about how each parameter contributes to the final loss.

Layer 2: Gradients for W2 and b2

The output layer computes: $\text{logits} = W2 @ a1 + b2$

We have $d\text{Logits}$. Now we need $dW2$ and $db2$.

For $dW2$: The gradient of a matrix multiply with respect to the weight matrix is an **outer product**:

```
dW2 = outer(dLogits, a1)
```

Shape check: $d\text{Logits}$ has length vocabSize , $a1$ has length hiddenDim , and `outer` gives $(\text{vocabSize} \times \text{hiddenDim})$: exactly the shape of $W2$. ✓

Intuition: $W2[i, j]$ determines how much hidden unit j contributes to logit i . The gradient $dW2[i, j]$ should be large and positive when logit i needed to go down a lot ($d\text{Logits}[i]$ is large positive) AND hidden unit j was highly activated ($a1[j]$ is large positive): because increasing $W2[i, j]$ would increase logit i , making things worse.

For $db2$: The bias just adds to the logits directly, so:

```
db2 = dLogits
```

The gradient of the loss with respect to each bias element is simply how much the loss gradient for that element's logit was.

```
val dW2 = backend.outer(dLogits, cache.a1)
val db2 = dLogits
```

Propagating Through Layer 2 to Get da1

We also need to know how the loss depends on $a1$: because $a1$ came from $W1$, and we'll need $da1$ to continue propagating backwards.

For $\text{logits} = W2 @ a1$, the gradient with respect to $a1$ is:

```
da1 = W2^T @ dLogits
```

The transpose of `W2` applied to `dLogits`. Shape check: `W2^T` is `(hiddenDim × vocabSize)`, `dLogits` is `(vocabSize,)`, result is `(hiddenDim,)`: the shape of `a1`. ✓

```
val da1 = backend.matVecMul(p.W2.transposeView, dLogits)
```

Now we know how much the loss wants each hidden unit to change. But `a1` was produced by the activation function applied to `z1`. So we need to propagate through the activation next.

Propagating Through the Activation Function

```
a1 = tanh(z1) (or relu(z1) )
```

The gradient of `tanh` is: $d(\tanh(z))/dz = 1 - \tanh(z)^2$

And since `a1 = tanh(z1)`, we can write: `tanh_gradient(z1)[i] = 1 - a1[i]^2`.

The rule: multiply `da1` element-wise by the activation's local gradient (Hadamard product):

```
dz1 = da1 ⊙ tanh_gradient(z1)
```

In code:

```
val dz1 = applyActivationGrad(da1, cache.z1, activation, backend)
```

where:

```
private def applyActivationGrad(grad: Vec, z: Vec, activation: String, backend:
ComputeBackend): Vec =
  activation.toLowerCase match
    case "relu" => backend.hadamard(grad, backend.reluGrad(z))
    case "tanh" => backend.hadamard(grad, backend.tanhGrad(z))
    case _      => throw new IllegalArgumentException(...)
```

Note: we pass `cache.z1` (the pre-activation values from the forward pass cache) to compute the gradient. This is why `ForwardCache` stores `z1`: we need it here.

ReLU gradient: `reluGrad(z)[i] = 1 if z[i] > 0 else 0`: the gradient simply passes through for positive pre-activations and blocks for negative ones ("dead neurons").

Layer 1: Gradients for W_1 and b_1

Now we have `dz1`. We apply the same pattern as Layer 2.

```
z1 = W1 @ x + b1
```

```
dW1 = outer(dz1, x)      ← same pattern as dW2
db1 = dz1                 ← same pattern as db2
dx  = W1T @ dz1         ← gradient flowing to the input
```

```
val dW1 = backend.outer(dz1, cache.x)
val db1 = dz1

val dx = backend.matVecMul(p.W1.transposeView, dz1)
```

`dx` is the gradient with respect to the concatenated embedding input. We'll need this to update the embedding matrix.

The Embedding Gradient: Scattering

The input `x` was formed by concatenating the embeddings of the context tokens:

```
val x = context.flatMap(id => p.E.rowSlice(id))
```

`dx` has length `contextSize × embedDim`. The first `embedDim` elements are the gradient for the embedding of `context[0]`, the next block for `context[1]`, and so on.

To update the embedding matrix `E`, we need to scatter these gradient blocks back to the right rows:

```
private def scatterEmbeddingGrad(dx: Vec, context: Vector[Int], vocabSize: Int,
embedDim: Int): Matrix =
  val accum = Array.fill(vocabSize * embedDim)(0.0)

  var pos = 0
  while pos < context.length do
    val tokenId = context(pos)
    val baseDx = pos * embedDim
    val baseRow = tokenId * embedDim
    var j = 0
    while j < embedDim do
      accum(baseRow + j) += dx(baseDx + j)
      j += 1
    pos += 1

  Matrix(accum.toVector, vocabSize, embedDim)
```

The `+=` is crucial. The same token ID might appear multiple times in the context (consider "the cat sat on the mat" ("the" appears twice). If token 1 appears at positions 0 and 4, its gradient contributions from both positions must be *accumulated*) not just the last one. Using `=` instead of `+=` would silently discard earlier gradient contributions.

This is one of the trickier implementation details in the codebase, and it has a real test in `LanguageModelSuite` to verify correctness.

The Complete `backward` Function

Putting it all together:

```

def backward(p: Params, cache: ForwardCache, target: Int, activation: String =
"tanh", ...): Grads =
  // 1. Gradient through softmax + cross-entropy
  val dLogits = cache.probs.indices.map { i =>
    val t = if i == target then 1.0 else 0.0
    cache.probs(i) - t
  }.toVector

  // 2. Gradient for W2, b2
  val dW2 = backend.outer(dLogits, cache.a1)
  val db2 = dLogits

  // 3. Gradient flowing back through layer 2
  val da1 = backend.matVecMul(p.W2.transposeView, dLogits)

  // 4. Gradient through activation
  val dz1 = applyActivationGrad(da1, cache.z1, activation, backend)

  // 5. Gradient for W1, b1
  val dW1 = backend.outer(dz1, cache.x)
  val db1 = dz1

  // 6. Gradient flowing back through layer 1
  val dx = backend.matVecMul(p.W1.transposeView, dz1)

  // 7. Scatter embedding gradient
  val dE = scatterEmbeddingGrad(dx, cache.context, p.E.rows, p.E.cols)

  Grads(dE = dE, dW1 = dW1, db1 = db1, dW2 = dW2, db2 = db2)

```

Seven steps. Each step computes a gradient by applying a local rule. The code flows in the opposite direction from the forward pass: that's why it's called *backpropagation*.

Why We Don't Need Calculus

Formally, backpropagation uses the chain rule of calculus. But notice: we never wrote a derivative symbol, never took a limit, never did any calculus.

What we did instead: for each layer, we stated the gradient rule in plain language ("the gradient of a matrix multiply with respect to the weight matrix is an outer product") and

implemented it directly. These rules can be derived using calculus, but once derived, they're just patterns you memorize and apply.

You can think of backpropagation as a lookup table: for each operation type (matrix multiply, bias add, activation function), there's a corresponding gradient rule. Apply the rules in reverse order.

Chapter Milestone

Let's run the complete backward pass and inspect the resulting `Grads`.

```

// In sbt console
import nn.{LanguageModel, ModelConfig}
import data.*
import scala.io.Source

val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 100)

val cfg = ModelConfig(contextSize = 3, embedDim = 8, hiddenDim = 16, vocabSize =
vocab.size)
val params = LanguageModel.initParams(cfg, seed = 42)

val ids = TextPipeline.tokensToIds(tokens, vocab)
val ex = TextPipeline.buildExamples(ids, contextSize = 3).head

val cache = LanguageModel.forward(params, ex.context)
val grads = LanguageModel.backward(params, cache, ex.target)

// Verify shapes mirror params
println("--- Gradient shapes (should match parameter shapes) ---")
println(s"dE:  ${grads.dE.rows}x${grads.dE.cols}    (E: ${params.E.rows}x${
params.E.cols})")
println(s"dW1: ${grads.dW1.rows}x${grads.dW1.cols}    (W1: ${params.W1.rows}x${
params.W1.cols})")
println(s"db1: ${grads.db1.length}                    (b1: ${params.b1.length})")
println(s"dW2: ${grads.dW2.rows}x${grads.dW2.cols}    (W2: ${params.W2.rows}x${
params.W2.cols})")
println(s"db2: ${grads.db2.length}                    (b2: ${params.b2.length})")

// Check: most rows of dE should be zero (only context tokens get gradients)
val nonZeroEmbeddingRows = (0 until grads.dE.rows).count { r =>
  grads.dE.rowSlice(r).exists(_ != 0.0)
}
println(s"\nNon-zero rows in dE: $nonZeroEmbeddingRows (should equal contextSize =
3)")
println(s"Context tokens: ${ex.context}")

// Check gradient for target token vs non-target
val targetGrad = grads.db2(ex.target)
val nonTargetGrads = grads.db2.zipWithIndex.filter(_._2 != ex.target).map(_._1)
println(f"\ndTargetLogit (b2 grad): ${targetGrad}%.4f    (should be negative)")
println(f"Mean non-target grad:    ${nonTargetGrads.sum / nonTargetGrads.length}%.4f
(should be positive)")

```

The key observations: 1. Gradient shapes match parameter shapes 2. Most rows of `dE` are zero: only the 3 context tokens got gradients 3. The target word's output bias gradient is negative (we want to push it up)

What You Learned

- **Backpropagation** traces the gradient backwards through the network, layer by layer
 - **Layer 2 gradients:** `dw2 = outer(dLogits, a1)` , `db2 = dLogits` , `da1 = W2^T @ dLogits`
 - **Activation gradient:** element-wise multiply by `tanh'(z1) = 1 - tanh(z1)^2` (or `reluGrad(z1)`)
 - **Layer 1 gradients:** `dw1 = outer(dz1, x)` , `db1 = dz1` , `dx = W1^T @ dz1`
 - **Embedding gradient scatter:** accumulate (not overwrite) gradient contributions for each token in the context: the same token appearing multiple times must add up
 - `Grads` mirrors `Params` : every parameter has a gradient of the same shape
-

Source Reference

- `src/main/scala/nn/LanguageModel.scala` : `backward` , `applyActivationGrad` , `scatterEmbeddingGrad`
 - `src/main/scala/linalg/LinearAlgebra.scala` : `tanhGrad` , `reluGrad` , `hadamard` , `outer`
-

Up Next

We have `Grads` . We know which direction to move every weight. Chapter 13 applies the update: SGD, the simplest optimization algorithm, and the complete `trainStep` function.

Chapter 13: Updating Weights: SGD and the Training Step

We have gradients. We know which direction makes the loss increase for each parameter. The update is simple: move in the opposite direction.

This is **Stochastic Gradient Descent** (SGD). It's the algorithm that trains neural networks. It's also, at its core, three lines of arithmetic.

The SGD Update Rule

```
new_weight = old_weight - learning_rate × gradient
```

Or in code: `w - lr * dw`

That's it.

- `w` is the current weight
- `dw` is its gradient (how much the loss increases when `w` increases)
- `lr` is the **learning rate**: a small positive number controlling step size
- The result is the new, slightly better weight

We subtract `lr * dw` because we want to move *opposite* the gradient: downhill, not uphill.

The Learning Rate

The learning rate is perhaps the most important hyperparameter in training.

Too large: The steps are too big. You might overshoot the minimum and end up with a higher loss than you started with. With very large learning rates, training can become chaotic or diverge entirely.

Too small: The steps are tiny. Training is stable but very slow. You might need 100× more iterations to converge.

Just right: Loss decreases smoothly and consistently.

Typical values range from `0.001` to `0.1`. The right value depends on your model size, dataset, and architecture. In practice, you experiment.

In our project's presets: - `quick : lr = 0.05` (faster, less stable) - `balanced : lr = 0.02` (good default) - `thorough : lr = 0.01` (careful, slow)

L2 Regularization

Optionally, we can add **L2 regularization** (also called weight decay):

```
new_weight = old_weight - lr × (gradient + l2 × old_weight)
```

The `l2 × old_weight` term is a gentle force pulling every weight toward zero. This prevents any single weight from growing very large, which tends to cause overfitting: the model memorizing specific patterns rather than learning general rules.

The intuition: if a weight is large and positive, it has a large influence on the output. L2 regularization says "unless this influence is clearly justified by the training signal, shrink it down."

In code:

```
def applyWeightDecay(w: Matrix, dw: Matrix): Matrix =  
  if l2 <= 0.0 then dw  
  else dw.zipMap(w)((grad, weight) => grad + l2 * weight)
```

We add `l2 * weight` to each gradient before the update. The weight effectively "decays" toward zero with each step.

Gradient Clipping

A safety mechanism: if the gradient norm gets very large, cap it.

```
private def clipGradients(g: Grads, maxNorm: Double): Grads =  
  val norm = math.sqrt(/* sum of squares of all gradient elements */)   
  if norm <= maxNorm || norm == 0.0 then g  
  else  
    val scale = maxNorm / norm  
    // multiply all gradients by scale  
    ...
```

If the total gradient norm exceeds `maxNorm`, we scale all gradients down proportionally so the total norm equals `maxNorm`. This prevents a single bad batch from causing an explosion in the weights.

Gradient clipping is used with `clipNorm: Option[Double]`. Using `Option` here is a Scala pattern worth understanding:

```
def update(p: Params, g: Grads, lr: Double, l2: Double = 0.0, clipNorm: Option[Double]  
= None): Params =  
  val gradients = clipNorm match  
    case Some(maxNorm) if maxNorm > 0.0 => clipGradients(g, maxNorm)  
    case _                               => g
```

`Option[Double]` can be either `Some(value)` (clipping is enabled) or `None` (no clipping). Pattern matching extracts the value if present. This is more expressive than using a magic number like `-1.0` to signal "no clipping."

The `update` Function

Here's the complete weight update:

```

def update(p: Params, g: Grads, lr: Double, l2: Double = 0.0, clipNorm: Option[Double]
= None): Params =
  require(lr > 0.0, s"learning rate must be > 0, got $lr")

  val gradients = clipNorm match
    case Some(maxNorm) if maxNorm > 0.0 => clipGradients(g, maxNorm)
    case _                               => g

  def applyWeightDecay(w: Matrix, dw: Matrix): Matrix =
    if l2 <= 0.0 then dw
    else dw.zipMap(w)((grad, weight) => grad + l2 * weight)

  def updateMat(w: Matrix, dw: Matrix): Matrix =
    w.zipMap(dw)((wv, gv) => wv - lr * gv)

  def updateVec(v: Vec, dv: Vec): Vec =
    LinearAlgebra.vecSub(v, LinearAlgebra.scalarMul(dv, lr))

  val dEAdj = applyWeightDecay(p.E, gradients.dE)
  val dW1Adj = applyWeightDecay(p.W1, gradients.dW1)
  val dW2Adj = applyWeightDecay(p.W2, gradients.dW2)

  Params(
    E = updateMat(p.E, dEAdj),
    W1 = updateMat(p.W1, dW1Adj),
    b1 = updateVec(p.b1, gradients.db1),
    W2 = updateMat(p.W2, dW2Adj),
    b2 = updateVec(p.b2, gradients.db2)
  )

```

The result is a **new** `Params` with updated weights. The original `p` is unchanged: this is immutability in action. The Scala garbage collector will reclaim the old `Params` once it's no longer referenced.

The `trainStep` Function

The complete single-example training step: forward pass, loss, backward pass, update.

```
def trainStep(
  p: Params,
  ex: Example,
  lr: Double,
  l2: Double = 0.0,
  clipNorm: Option[Double] = None,
  activation: String = "tanh",
  backend: ComputeBackend = CpuBackend.Default
): (Params, Double) =
  val cache = forward(p, ex.context, activation, backend)
  val loss = lossFromCache(cache, ex.target, backend)
  val grads = backward(p, cache, ex.target, activation, backend)
  val updated = update(p, grads, lr, l2 = l2, clipNorm = clipNorm)
  (updated, loss)
```

It takes the current `Params` and one `Example`, and returns the updated `Params` along with the loss for that example.

This is the fundamental unit of learning. Every epoch is many `trainStep` calls, one per training example.

Watching the Loss Fall

The real payoff of this chapter is watching it happen. Let's run `trainStep` 100 times on a small corpus and print the loss every 10 steps.

What should we expect? The first call will have high loss (random model). Each subsequent call will have slightly lower loss: the model is learning. The decrease won't be perfectly smooth (a single example is noisy), but the trend should be clearly downward.

Chapter Milestone

```
// In sbt console
import nn.{LanguageModel, ModelConfig}
import data.*
import scala.io.Source

val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val vocab = TextPipeline.buildVocab(tokens, maxVocab = 100)

val cfg = ModelConfig(contextSize = 3, embedDim = 8, hiddenDim = 32, vocabSize =
vocab.size)
var params = LanguageModel.initParams(cfg, seed = 42)

val ids = TextPipeline.tokensToIds(tokens, vocab)
val examples = TextPipeline.buildExamples(ids, contextSize = 3)

// Run 100 training steps, cycling through examples
val lr = 0.05
var totalLoss = 0.0
println("Step | Loss      | Running avg")
println("-----|-----|-----")

for step <- 0 until 100 do
  val ex = examples(step % examples.length)
  val (newParams, loss) = LanguageModel.trainStep(params, ex, lr)
  params = newParams
  totalLoss += loss

  if (step + 1) % 10 == 0 then
    val avgLoss = totalLoss / (step + 1)
    println(f"  ${step+1}%3d | ${loss}%.4f | ${avgLoss}%.4f")

// Compare initial vs final prediction on the first example
val firstEx = examples.head
val initParams = LanguageModel.initParams(cfg, seed = 42)
val initCache = LanguageModel.forward(initParams, firstEx.context)
val finalCache = LanguageModel.forward(params, firstEx.context)

println(s"\nContext: ${firstEx.context.map(vocab.toToken)}")
println(s"Target: '${vocab.toToken(firstEx.target)}'")
println(f"Initial prob for target: ${initCache.probs(firstEx.target) * 100}%.2f%%")
println(f"After training:          ${finalCache.probs(firstEx.target) * 100}%.2f%%")
```

The target word's probability should increase after training: perhaps from 1% to 5% or 10%. That's the model learning.

Note the `var params = ...` and `params = newParams` in the loop: this is one of the few places in the project where we use mutation. Each iteration creates a new `Params` (immutably) and we update our local variable to point to it. The Scala immutability guarantee holds for the `Params` values themselves; we're just updating which one we're looking at.

What You Learned

- SGD update: `new_weight = old_weight - lr × gradient`
 - The **learning rate** controls step size; too large → instability, too small → slow
 - **L2 regularization** (`l2`) adds a small force pulling weights toward zero, preventing overfitting
 - **Gradient clipping** (`clipNorm`) caps the gradient norm to prevent exploding updates
 - `Option[Double]` is Scala's idiomatic way to express "maybe a value": `None` or `Some(x)`
 - `update` returns a new `Params` (immutable): the original is unchanged
 - `trainStep` = forward + loss + backward + update, returns `(Params, Double)`
 - The loss decreases with each step: the model is genuinely learning
-

Source Reference

- `src/main/scala/nn/LanguageModel.scala` : `update` , `clipGradients` , `trainStep` , `trainBatchStep`
-

Up Next

We can train on one example at a time. But real training processes thousands of examples per second by batching them together. Chapter 14 introduces mini-batches, epochs, validation, early stopping, and the complete training loop.

Chapter 14: The Training Loop: Epochs, Batches, and Progress

Training on one example at a time works, but it's noisy and slow. Real training processes many examples together, runs for many passes over the full dataset, and stops automatically when progress stalls.

This chapter introduces the complete training loop: mini-batches, epochs, validation, early stopping, and the live progress display.

Why Mini-Batches?

When you train on a single example, the gradient you compute is based on just one data point. A single example can be noisy: maybe this particular example is unusual, or maybe you happened to pick an outlier. The gradient computed from one example might point in a direction that's helpful for this example but harmful for others.

Mini-batches solve this by processing multiple examples simultaneously and averaging their gradients. A batch of 32 examples gives you a gradient that's an average of 32 individual gradients: much more reliable.

Additionally, batches can be processed using matrix multiplication on the full batch at once, which is much faster than 32 separate calls.

The trade-off: - **Larger batches**: smoother gradients, faster per-epoch, but each update uses more memory - **Smaller batches**: noisier gradients, slower per-epoch, but the noise can help escape local minima

A batch size of 32–128 is typical.

The `trainBatchStep` Function

`trainBatchStep` is like `trainStep`, but for a batch:

```
def trainBatchStep(p: Params, batch: Vector[Example], lr: Double, ...): (Params, Double) =
  val contexts = batch.map(_.context)
  val targets  = batch.map(_.target)
  val cache    = forwardBatch(p, contexts, activation, backend)
  val losses   = lossFromBatchCache(cache, targets, backend)
  val grads    = backwardBatch(p, cache, targets, activation, backend)
  val scale    = 1.0 / batch.length.toDouble
  val scaled   = Grads(
    dE = grads.dE.map(_ * scale),
    dW1 = grads.dW1.map(_ * scale),
    db1 = grads.db1.map(_ * scale),
    dW2 = grads.dW2.map(_ * scale),
    db2 = grads.db2.map(_ * scale)
  )
  val updated = update(p, scaled, lr, ...)
  (updated, losses.sum / losses.length.toDouble)
```

The key difference from `trainStep`: we scale the gradients by `1/batchSize` before applying them. This normalizes the update: otherwise, a batch of 32 examples would produce a gradient 32× larger than a single example, and you'd need to adjust the learning rate accordingly.

Epochs

One **epoch** is one complete pass over the entire training dataset.

In practice, we typically train for many epochs (10, 20, 50) because: 1. The model doesn't fully absorb the patterns from a single pass 2. With mini-batches, each update only sees a fraction of the data

Between epochs, we evaluate the validation loss. This is the key metric: is the model improving on data it hasn't trained on?

TrainConfig : All the Knobs

The training loop is controlled by `TrainConfig` :

```
final case class TrainConfig(  
  epochs: Int = 10,           // How many passes over the data  
  learningRate: Double = 0.05, // Step size  
  lrDecay: Double = 1.0,      // Multiply LR by this each epoch (< 1 = decay)  
  l2: Double = 0.0,           // L2 regularization strength  
  clipNorm: Option[Double] = None, // Gradient clipping (None = disabled)  
  shuffleEachEpoch: Boolean = true, // Shuffle training data each epoch  
  seed: Int = 42,             // Random seed for shuffling  
  patience: Int = 0,          // Early stopping: stop after this many non-improving  
epochs (0 = disabled)  
  activation: String = "tanh", // tanh or relu  
  batchSize: Int = 0,          // 0 = single-example mode; > 0 = mini-batch size  
  ...  
)
```

The most important ones for beginners: - `epochs` : more epochs = more training time, but risk of overfitting - `learningRate` : start with `0.02` ; adjust if training is unstable or too slow - `patience` : early stopping threshold (more on this below) - `batchSize` : `32` is a safe default; `0` means single-example mode

Learning Rate Decay

As training progresses, the model gets closer to a good solution. Large steps that were useful early in training can start overshooting near the end.

Learning rate decay reduces the learning rate each epoch:

```
lr_at_epoch_e = initial_lr × lrDecay^e
```

With `lrDecay = 0.95`, the learning rate decreases by 5% per epoch. After 20 epochs, it's about $0.95^{20} \approx 0.36$ of the initial value. The model takes smaller, more careful steps as it converges.

In the code:

```
val currentLr = cfg.learningRate * math.pow(cfg.lrDecay, epoch.toDouble)
```

Early Stopping

What if the validation loss stops improving: or starts getting worse? We could let training continue for all `epochs`, but that wastes time and might overfit.

Early stopping halts training when the validation loss hasn't improved in `patience` consecutive epochs.

Example: with `patience = 5`, if validation loss doesn't improve for 5 epochs in a row, training stops and we restore the weights from the best epoch.

The `classifyTrajectory` function analyzes the loss history:

```
enum TrainingStatus:  
  case Improving, Stalled, Regressing
```

Each epoch is classified based on: - Whether the validation loss beat the previous best - Whether the 3-epoch trend is downward or upward - Whether the generalization gap (val - train) is widening

If the status is `Stalled` or `Regressing` for `patience` consecutive epochs, training stops.

The Epoch Table

During training, the loop displays an epoch-by-epoch table:

Epoch	Train Loss	Val Loss	Val Perplexity	Status	Best Δ%
1	3.2451	3.4102	30.25	Improving	+12.3%
2	2.9873	3.1240	22.74	Improving	+8.4%
3	2.8765	3.0891	21.98	Improving	+1.1%
4	2.8120	3.1015	22.25	Stalled	-0.4%
5	2.7890	3.1280	22.84	Stalled	-1.5%

Reading this table: - **Train Loss < Val Loss**: normal, means the model fits training data better than unseen data - **Best Δ%**: how much the current val loss improved vs the best so far (positive = better) - **Improving → Stalled**: the model stopped making progress; early stopping will trigger soon - **Generalization gap** (`val - train`): if this widens quickly, the model is overfitting

The `TrainResult`

`Trainer.train` returns a `TrainResult` :

```
final case class TrainResult(  
  params: Params,           // The best model weights  
  history: Vector[EpochMetrics], // One entry per epoch  
  interrupted: Boolean,     // Did training stop early?  
  ...  
)
```

The `history` lets you analyze what happened: plot the loss curve, identify when overfitting started, compare runs.

Running the Training Loop

In the interactive CLI, training is invoked via the `train` command. But you can also call `Trainer.train` directly from code:

```
import train.{Trainer, TrainConfig}

val result = Trainer.train(
  trainSet    = trainExamples,
  valSet      = valExamples,
  config      = cfg,
  trainConfig = TrainConfig(
    epochs      = 20,
    learningRate = 0.02,
    lrDecay     = 0.95,
    patience    = 5,
    batchSize   = 32
  )
)

println(s"Best val perplexity: ${result.history.map(_.valPerplexity).min}")
```

Chapter Milestone

Run a real training session using the command-line interface.

From the project directory:

```
sbt "run train --input data/corpus/example-corpus.txt --preset quick --yes"
```

The `--preset quick` runs 5 epochs with a small model. The `--yes` skips confirmation prompts.

Watch the output. You'll see: 1. Vocabulary building 2. Example generation 3. The epoch table updating in real time 4. A summary when training completes

Then run it again with `--preset balanced` (20 epochs) and compare:


```
sbt "run train --input data/corpus/example-corpus.txt --preset balanced --yes"
```

Note the final validation perplexity. This is your trained model's quality score.

After training completes, run:

```
sbt "run predict --context 'the cat'"
```

You should see a list of predicted next words with probabilities. These will make *some* sense, because the model has actually learned patterns from the corpus.

What You Learned

- **Mini-batches** average gradients over multiple examples for smoother, faster training
 - Batch gradients are scaled by `1/batchSize` before application
 - An **epoch** is one complete pass over the training data
 - `TrainConfig` controls all training hyperparameters
 - **Learning rate decay** reduces the step size each epoch as the model converges
 - **Early stopping** halts training when validation loss stops improving, preventing wasted time and overfitting
 - `TrainingStatus.Improving/Stalled/Regressing` classifies the training trajectory each epoch
 - `TrainResult.history` records per-epoch metrics for analysis
-

Source Reference

- `src/main/scala/train/Trainer.scala` : `TrainConfig` , `EpochMetrics` , `TrainResult` , `classifyTrajectory` , `createProgressBar`
 - `src/main/scala/train/TrainingDisplay.scala` : live progress rendering
 - `src/main/scala/nn/LanguageModel.scala` : `trainBatchStep` , `forwardBatch` , `backwardBatch`
-

Up Next

Training produces a model. Chapter 15 covers saving that model to disk (checkpoints) so you can use it later without retraining.

Chapter 15: Saving and Loading: The Checkpoint

Training takes time. A small model on a small corpus trains in minutes; a larger model on a larger corpus might take hours. The last thing you want is to lose that work when the process exits.

Checkpointing is the practice of saving model state to disk periodically. A checkpoint lets you:

- Resume training if it was interrupted
- Load the model for inference without retraining
- Compare models trained under different conditions
- Roll back to an earlier version if something goes wrong

This chapter covers `CheckpointIO` : the module that saves and loads model parameters.

What Needs to Be Saved?

To fully reconstruct a model, you need:

1. **The parameters** (`Params`): the weight matrices `E` , `W1` , `W2` and bias vectors `b1` , `b2`
2. **The model config** (`ModelConfig`): `contextSize` , `embedDim` , `hiddenDim` , `vocabSize` :
because you need these to interpret the weights correctly
3. **The vocabulary** (`Vocab`): the mapping from tokens to IDs: a model is useless without its vocabulary

These three things together are a complete, self-contained model. Given them, you can run inference on any text.

The Checkpoint File Format

`CheckpointIO` uses a plain-text format: one key-value pair per line, separated by `=`.

Here's what a tiny checkpoint might look like:

```
config.contextSize=3
config.embedDim=4
config.hiddenDim=8
config.vocabSize=10
E.rows=10
E.cols=4
E.data=0.1234,-0.5678,0.2345,...
W1.rows=8
W1.cols=12
W1.data=-0.0821,0.1452,...
b1.size=8
b1.data=0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0
W2.rows=10
W2.cols=8
W2.data=0.3421,-0.1234,...
b2.size=10
b2.data=0.0,0.0,...
```

A real checkpoint with `vocabSize=500`, `embedDim=24`, `hiddenDim=128` would have tens of thousands of values in each `.data` line.

Why plain text?

1. **Debuggable:** you can open it in a text editor and see what's inside
2. **Portable:** any language that can parse key-value files can read it
3. **No dependencies:** no binary serialization library needed
4. **Version-tolerant:** if you add fields later, older files just won't have them: easy to handle with defaults

The trade-off: plain text is larger than binary (a `Double` takes 20+ characters vs 8 bytes). For our models (a few MB), this is fine. For production models with billions of parameters, you'd want binary formats like numpy's `.npy` or HDF5.

Saving: `CheckpointIO.save`

```
object CheckpointIO:

  def save(params: Params, config: ModelConfig, path: Path): Unit =
    val sb = new StringBuilder
    sb.append(s"config.contextSize=${config.contextSize}\n")
    sb.append(s"config.embedDim=${config.embedDim}\n")
    sb.append(s"config.hiddenDim=${config.hiddenDim}\n")
    sb.append(s"config.vocabSize=${config.vocabSize}\n")

    def appendMatrix(name: String, m: Matrix): Unit =
      sb.append(s"$name.rows=${m.rows}\n")
      sb.append(s"$name.cols=${m.cols}\n")
      sb.append(s"$name.data=${m.data.mkString(",")}\n")

    def appendVec(name: String, v: Vector[Double]): Unit =
      sb.append(s"$name.size=${v.length}\n")
      sb.append(s"$name.data=${v.mkString(",")}\n")

    appendMatrix("E", params.E)
    appendMatrix("W1", params.W1)
    appendVec("b1", params.b1)
    appendMatrix("W2", params.W2)
    appendVec("b2", params.b2)

    Files.write(path, sb.result().getBytes(StandardCharsets.UTF_8))
```

`StringBuilder` is an efficient way to build a large string piece by piece. Appending to a `StringBuilder` is fast; repeatedly concatenating strings with `+` would be quadratic. For a checkpoint with 40,000+ values, this matters.

`m.data.mkString(",")` serializes a `Vector[Double]` to a comma-separated string. `.split(",").map(_.toDouble)` reverses it.

Loading: `CheckpointIO.load`

```
def load(path: Path): (Params, ModelConfig) =
  val raw = Files.readAllLines(path, StandardCharsets.UTF_8)
  val entries = raw.toArray(...).toVector
    .filter(_.contains("="))
    .map { line =>
      val Array(k, v) = line.split("=", 2)
      k.trim -> v.trim
    }
    .toMap

  def getInt(key: String): Int = entries.getOrElse(key, throw ...).toInt
  def getDoubles(key: String): Vector[Double] =
    entries(key).split(",").toVector.map(_.toDouble)

  def parseMatrix(name: String): Matrix =
    val rows = getInt(s"$name.rows")
    val cols = getInt(s"$name.cols")
    val data = getDoubles(s"$name.data")
    Matrix(data, rows, cols)

  ...

  val cfg = ModelConfig(
    contextSize = getInt("config.contextSize"),
    ...
  )
  val params = Params(E = parseMatrix("E"), ...)
  (params, cfg)
```

The loading logic: 1. Read all lines 2. Filter and parse into a `Map[String, String]` 3. Extract values by key, converting to the right types 4. Reconstruct `ModelConfig` and `Params`

Returns a tuple `(Params, ModelConfig)` . Pattern-match to destructure:

```
val (params, config) = CheckpointIO.load(Path.of("data/models/latest.ckpt"))
```

The Vocabulary File

The vocabulary is saved separately, by `VocabIO` :

```
object VocabIO:
  def save(vocab: Vocab, path: String): Unit =
    val lines = vocab.idToToken.mkString("\n")
    Files.writeString(Path.of(path), lines, StandardCharsets.UTF_8)

  def load(path: String): Vocab =
    val lines = Files.readAllLines(Path.of(path),
StandardCharsets.UTF_8).toScala(Vector)
    val idToToken = lines
    val tokenToId = idToToken.zipWithIndex.toMap
    Vocab(tokenToId = tokenToId, idToToken = idToToken)
```

One token per line. Line number = token ID. Simple.

The Persistent Model Paths

By convention, the project uses these paths:

```
data/models/latest.ckpt    ← current model weights
data/models/latest.vocab   ← vocabulary
data/models/latest.replay  ← replay buffer (optional, for continual learning)
```

After training, the `train` command automatically saves to these paths. The `predict` and `chat` commands automatically load from them. This convention is what makes `sbt "run chat"` work without you specifying any files.

Verifying Round-Trip Fidelity

A checkpoint is only useful if loading it gives you exactly the same model you saved. Let's verify:

```
// Save and load, check that predictions match
val originalPrediction = LanguageModel.forward(params, context)
CheckpointIO.save(params, config, Path.of("test.ckpt"))
val (loadedParams, loadedConfig) = CheckpointIO.load(Path.of("test.ckpt"))
val loadedPrediction = LanguageModel.forward(loadedParams, context)

// These should be identical
assert(originalPrediction.probs == loadedPrediction.probs)
```

The test suite in `CheckpointIOSuite` does exactly this: train, save, load, compare.

Chapter Milestone

Train a model, save it, inspect the checkpoint file, then reload and verify.

```
# Train a quick model
sbt "run train --input data/corpus/example-corpus.txt --preset quick --yes"
```

Now open `data/models/latest.ckpt` in a text editor. Look at: - The config values at the top - The shape of `E` (rows = vocabSize, cols = embedDim) - The first few values of `E.data`: small decimals from Xavier init, modified by training

Now verify the checkpoint works:

```
sbt "run predict --context 'the quick brown'"
```

This loads from `data/models/latest.ckpt` and `data/models/latest.vocab` automatically. You'll see a ranked list of predicted next words.

If you want to verify round-trip fidelity in the REPL:

```
// In sbt console
import train.CheckpointIO
import nn.LanguageModel
import java.nio.file.Path

// Load the existing checkpoint
val (params, config) = CheckpointIO.load(Path.of("data/models/latest.ckpt"))
println(s"Loaded model: contextSize=${config.contextSize}, vocab=${config.vocabSize}")

// Run prediction
import data.*
import scala.io.Source
val vocab = data.VocabIO.load("data/models/latest.vocab")
val context = Vector("the", "quick").map(vocab.toId)
val cache = LanguageModel.forward(params, context)

import linalg.LinearAlgebra
val top5 = LinearAlgebra.argTopK(cache.probs, 5)
println("Top 5 predictions:")
top5.foreach { case (id, prob) =>
  println(f"  '${vocab.toToken(id)}%-12s'  ${prob * 100}%.2f%%")
}
```

What You Learned

- A checkpoint saves `Params` + `ModelConfig` to disk in plain-text key-value format
 - Plain text is chosen for debuggability and portability over binary efficiency
 - `VocabIO` saves the vocabulary separately: one token per line
 - `CheckpointIO.load` returns `(Params, ModelConfig)` as a tuple
 - The `data/models/latest.*` convention lets the CLI commands find the model automatically
 - Round-trip fidelity (save → load → same predictions) should be tested
-

Source Reference

- `src/main/scala/train/CheckpointIO.scala` : `save` , `load`
 - `src/main/scala/data/VocabIO.scala` : vocabulary persistence
-

Up Next

Part 3 is complete. We have a fully trainable model with checkpointing. Part 4 finishes the story: Chapter 16 covers evaluation, Chapter 17 covers text generation, Chapter 18 covers the CLI, and Chapter 19 is the payoff.

Chapter 16: Evaluating What You Built

Training a model is one thing. Understanding whether the model is actually learning something useful is another.

This chapter is about reading the signals: how to distinguish a model that's genuinely learning from one that's memorizing training data, and what the numbers mean in plain language.

Training Loss vs Validation Loss

Every epoch, the training loop computes two loss values:

- **Training loss**: average loss on the training set: examples the model trained on
- **Validation loss**: average loss on the validation set: examples the model has never seen

These two numbers tell different stories.

Training loss measures how well the model *fits* the training data. It should decrease across epochs. If it doesn't, the model isn't learning.

Validation loss measures how well the model *generalizes* to new data. This is the number that really matters for a useful model.

The Three Patterns

Watch for these patterns in your epoch table:

Healthy learning:

Epoch	Train	Val	Gap	
1	3.42	3.61	0.19	← val slightly higher, normal
2	3.01	3.20	0.19	
3	2.75	2.91	0.16	
4	2.58	2.74	0.16	← both decreasing, gap stable

Both losses decrease together. The gap (val - train) stays roughly stable. This is what you want.

Overfitting:

Epoch	Train	Val	Gap	
1	3.42	3.61	0.19	
2	2.98	3.15	0.17	
3	2.61	2.95	0.34	← gap widening
4	2.31	3.12	0.81	← val started increasing!
5	2.08	3.45	1.37	

Training loss keeps dropping. Validation loss stops dropping and starts rising. The generalization gap widens rapidly. The model is memorizing the training data: it's getting better at reciting examples it's seen, but worse at handling new ones.

When overfitting starts, stop training. The best model was at epoch 2 or 3.

Underfitting:

Epoch	Train	Val	Gap
1	4.21	4.30	0.09
2	4.18	4.27	0.09
3	4.16	4.25	0.09
4	4.15	4.24	0.09

Both losses are barely moving. The model isn't learning much. Possible causes: learning rate too low, model too small, not enough training data, too few epochs.

The Generalization Gap

The **generalization gap** is $(\text{val_loss} - \text{train_loss}) / \text{val_loss}$.

A small gap (say, under 0.1) is healthy: the model is doing slightly better on training data than validation, as expected.

A large gap (over 0.3–0.5) is a warning sign of overfitting.

In the code:

```
val gap = (valLoss - trainLoss) / math.max(math.abs(valLoss), 1e-9)
```

`classifyTrajectory` uses the gap as one signal for classifying training status:

```
val gapWidening = prevGap.exists(g => gap > g + 0.01)
```

If the gap increased by more than 1 percentage point from the previous epoch, that's considered "widening."

Reading the Status Column

The epoch table includes a `Status` column:

STATUS	MEANING
Improving	Val loss beat the previous best, or trend is clearly downward
Stalled	Val loss is flat: not getting better or worse
Regressing	Val loss is getting worse

The patience counter increments on `Stalled` or `Regressing` epochs. When it reaches `patience`, training stops and the best checkpoint is restored.

What Perplexity Means in Practice

Perplexity = $\exp(\text{val_loss})$. Some reference points:

PERPLEXITY	WHAT IT MEANS
$\sim \text{vocab_size}$	No learning: uniform distribution
100–200	Very limited learning, small corpus
30–80	Reasonable for a small model on simple text
10–30	Good; model has learned strong patterns
Under 10	Excellent; model is very confident (may be overfitting)

For our small models trained on limited data, target perplexity in the 30–100 range depending on corpus size.

`Metrics.meanLoss`

To evaluate on a dataset:

```
object Metrics:
  def perplexity(loss: Double): Double = math.exp(loss)

  def meanLoss(
    examples: Vector[Example],
    params: Params,
    cfg: ModelConfig,
    backend: ComputeBackend
  ): Double =
    if examples.isEmpty then 0.0
    else
      val total = examples.map { ex =>
        val cache = LanguageModel.forward(params, ex.context, cfg.activation, backend)
        LanguageModel.lossFromCache(cache, ex.target, backend)
      }.sum
      total / examples.length
```

Run forward on every example, compute loss, average. That's the validation loss.

Comparing Runs

After training, the metrics are saved to `data/metrics/latest-summary.txt`. Here's how to read a summary:

```
Run ID:      train-2026-05-09T17-35-55.591214Z-980a189f
Backend:     GPU (Metal fp32)
Train loss:  3.8412
Val loss:    4.3017
Val perp:    73.82
Train perp:  46.48
Gen gap:     10.7%
Epochs:     13 / 50
Status:      early_stopped
```

The run stopped at epoch 13 (out of a maximum of 50) because early stopping triggered. Validation perplexity is 73.82, meaning on average the model thinks ~74 words are equally likely.

If you're comparing two runs, look at `Val perp`: lower is better.

Chapter Milestone

Let's analyze a trained model's loss history.

First, run a proper training session:

```
sbt "run train --input data/corpus/example-corpus.txt --preset balanced --yes"
```

Then in the REPL:

```
// In sbt console
import train.CheckpointIO
import eval.Metrics
import nn.LanguageModel
import data.*
import scala.io.Source
import java.nio.file.Path

// Load the trained model
val (params, config) = CheckpointIO.load(Path.of("data/models/latest.ckpt"))
val vocab = VocabIO.load("data/models/latest.vocab")

// Rebuild examples from the corpus
val rawText = Source.fromFile("data/corpus/example-corpus.txt").mkString
val tokens = TextPipeline.tokenize(rawText)
val ids = TextPipeline.tokensToIds(tokens, vocab)
val examples = TextPipeline.buildExamples(ids, config.contextSize)
val (train, validation) = TextPipeline.splitDeterministic(examples, 0.9, 42)

// Evaluate
val trainLoss = train.take(200).map { ex =>
  val cache = LanguageModel.forward(params, ex.context)
  LanguageModel.lossFromCache(cache, ex.target)
}.sum / 200.0

val valLoss = validation.take(200).map { ex =>
  val cache = LanguageModel.forward(params, ex.context)
  LanguageModel.lossFromCache(cache, ex.target)
}.sum / 200.0

println(f"Train loss (sample): $trainLoss%.4f (perplexity: $
{Metrics.perplexity(trainLoss)}%.1f)")
println(f"Val loss (sample): $valLoss%.4f (perplexity: $
{Metrics.perplexity(valLoss)}%.1f)")
println(f"Generalization gap: ${((valLoss - trainLoss) / valLoss * 100)}%.1f%%")
```

Compare the perplexity to the random baseline from Chapter 10. How much did the model improve?

What You Learned

- **Training loss** measures fit to training data; **validation loss** measures generalization

- Validation loss is the metric that matters for a useful model
 - The three patterns: healthy learning, overfitting, underfitting
 - **Generalization gap** = `(val - train) / val` ; a widening gap signals overfitting
 - `TrainingStatus.Improving/Stalled/Regressing` classifies each epoch's trajectory
 - Early stopping prevents wasted training time and overfitting
 - Perplexity gives an intuitive scale: "how many words does the model consider equally likely?"
-

Source Reference

- `src/main/scala/eval/Metrics.scala` : `perplexity` , `meanLoss`
 - `src/main/scala/train/Trainer.scala` : `classifyTrajectory` , `EpochMetrics`
-

Up Next

We can train and evaluate a model. Chapter 17 covers the other side: using the model to generate text: turning a probability distribution into actual words with temperature, top-K, and top-P sampling.

Chapter 17: Generating Text: Temperature, Top-K, and Top-P

Prediction and generation are different things.

Prediction asks: given this context, which word has the highest probability? It returns a ranking.

Generation asks: given this context, produce the next word. It returns one word, sampled from the probability distribution. Then it uses that word as part of the context for the next prediction. Repeat until you have a sentence.

This chapter covers the sampling strategies that turn probabilities into text.

The Naive Approach: Argmax

The simplest strategy: always pick the word with the highest probability.

```
val nextId = cache.probs.zipWithIndex.maxBy(_._1)._2
```

This is called **greedy decoding** or **argmax sampling**. It's deterministic: given the same input, you always get the same output.

The problem: greedy decoding produces **repetitive**, **boring** text. If "the" is always the most likely next word, you'll get "the the the the the..." forever. Even with more interesting data, argmax tends to get stuck in loops.

Real language is probabilistic. Not every "the" is followed by the most common word that follows "the": there's variety, and variety is what makes text feel natural.

Temperature Scaling

Temperature changes the shape of the probability distribution before sampling.

The idea: divide all logits by a temperature value T before applying softmax.

```
adjusted_probs = softmax(logits / T)
```

Effect: - $T < 1.0$ (low temperature): sharpens the distribution. High-probability words become even more likely; low-probability words fade away. More predictable, less diverse. - $T = 1.0$: no change. Original distribution. - $T > 1.0$ (high temperature): flattens the distribution. High-probability words become less dominant; low-probability words get more of a chance. More creative, more random, more likely to make mistakes.

Concrete example with logits `[2.0, 1.0, 0.5, 0.2]` :

```
T=0.5 probs: [0.823, 0.148, 0.025, 0.004]  ← very confident in word 0
T=1.0 probs: [0.594, 0.218, 0.132, 0.056]  ← original
T=2.0 probs: [0.410, 0.272, 0.200, 0.118]  ← much more spread out
```

With low temperature, you almost always pick word 0. With high temperature, you might pick word 2 or 3 occasionally, producing surprising variety.

The personality dial: Low temperature = confident, repetitive, safe. High temperature = creative, surprising, occasionally incoherent. Good text generation often uses $T = 0.7-1.2$.

Top-K Sampling

Instead of sampling from all `vocabSize` words, **top-K sampling** restricts the candidate pool to the `K` most probable words. Everything else gets probability 0.

```
def argTopK(v: Vec, k: Int): Vector[(Int, Double)] =
  v.zipWithIndex
    .map { case (value, idx) => (idx, value) }
    .sortBy { case (_, value) => -value }
    .take(math.max(k, 1))
    .toVector
```

After getting the top-K candidates, renormalize their probabilities to sum to 1, then sample.

Why top-K? The probability distribution has a long tail of very unlikely words. Including them in sampling means you'll occasionally generate complete nonsense (rare words that don't fit the context). Top-K cuts off the tail.

Choosing K: - $K = 1$: greedy decoding (always pick the top word) - $K = 10-40$: commonly used range; good balance - $K = \text{vocabSize}$: no restriction, same as standard sampling

With `K = 10` and `vocabSize = 500`, you're sampling from the 2% most likely words. The others don't even get considered.

Top-P (Nucleus) Sampling

Top-K has a limitation: the "right" K varies with the distribution. Sometimes the top word has 90% probability (K=1 is fine). Sometimes the top 20 words each have ~5% probability (K=20 is better). A fixed K doesn't adapt.

Top-P sampling (nucleus sampling) adapts by keeping the smallest set of words whose probabilities sum to at least `P`.

Algorithm: 1. Sort words by probability, highest first 2. Add words to the "nucleus" until the cumulative probability exceeds `P` 3. Sample from the nucleus

For `P = 0.9` : - If one word has 95% probability, the nucleus contains just that word - If the top 15 words together sum to 90%, the nucleus contains those 15 words

Top-P naturally adapts to confident predictions (small nucleus) and uncertain predictions (larger nucleus).

Choosing P: `P = 0.9` or `P = 0.95` are common. Higher values include more candidates.

Combining Temperature, Top-K, and Top-P

These strategies compose. A typical generation call:

1. Compute logits
2. Apply temperature: `adjusted_logits = logits / T`
3. Compute softmax: `probs = softmax(adjusted_logits)`
4. Apply top-K filter: keep only the top K words
5. Apply top-P filter: within the top-K, keep only the nucleus
6. Renormalize

7. Sample

In the chat command's sampling logic:

```
// Temperature
val scaledLogits = cache.logits.map(_ / temperature)
val probs = LinearAlgebra.softmaxStable(scaledLogits)

// Top-K filter
val topKCandidates = LinearAlgebra.argTopK(probs, topK)

// Top-P filter within top-K
var cumulative = 0.0
val nucleusCandidates = topKCandidates.takeWhile { case (_, p) =>
    cumulative += p
    cumulative - p < topP // keep if adding this word doesn't exceed P
}

// Renormalize
val totalP = nucleusCandidates.map(_._2).sum
val renormalized = nucleusCandidates.map { case (id, p) => (id, p / totalP) }

// Sample
// (sample from renormalized using a random number in [0, 1))
```

Autoregressive Generation

Generation is autoregressive: each token is conditioned on all previous tokens.

The generation loop:

1. Start with a seed context (e.g., "the quick")
2. Run forward pass → probability distribution
3. Sample a token using temperature/top-K/top-P
4. Append the sampled token to the context
5. If context is longer than contextSize, drop the oldest token
6. Go to step 2
7. Stop when maxTokens is reached or a stop condition is met

This sliding window (always keeping the most recent `contextSize` tokens) is the key constraint. The model doesn't remember the beginning of a long conversation; it only sees the last few tokens.

The "Ban UNK" Flag

One practical option: `banUnk` (ban the unknown token). If enabled, the `<UNK>` token's probability is set to 0 before sampling. This prevents the model from generating `<UNK>` as output: which would be meaningless text.

```
if banUnk then
  val unkId = vocab.unkId
  val maskedProbs = probs.updated(unkId, 0.0)
  val total = maskedProbs.sum
  maskedProbs.map(_ / total) // renormalize
```

Chapter Milestone

Let's experiment with sampling parameters and see how they change the model's output.

First, make sure you have a trained model. Then run chat with different settings:

```
# Conservative: low temperature, small top-K
sbt 'run chat --temperature 0.5 --topK 5 --maxTokens 30 --banUnk true'

# Default
sbt 'run chat --temperature 0.8 --topK 40 --topP 0.9 --maxTokens 30 --banUnk true'

# Creative: high temperature
sbt 'run chat --temperature 1.5 --topK 50 --maxTokens 30 --banUnk true'
```

For each, type the same seed phrase (like "the cat") and observe the difference. With low temperature, the output will be consistent and repetitive. With high temperature, it'll be more varied: possibly incoherent, but more surprising.

For a manual experiment in the REPL:

```
// In sbt console
import train.CheckpointIO
import nn.LanguageModel
import linalg.LinearAlgebra
import data.*
import scala.util.Random
import java.nio.file.Path

val (params, config) = CheckpointIO.load(Path.of("data/models/latest.ckpt"))
val vocab = VocabIO.load("data/models/latest.vocab")

val seed = "the cat".split(" ").map(vocab.toId).toVector
val rnd = Random(42)

// Generate 20 tokens with temperature 0.8, top-K 20
var context = seed
val generated = (0 until 20).map { _ =>
  val cache = LanguageModel.forward(params, context.takeRight(config.contextSize))

  // Apply temperature
  val scaled = cache.logits.map(_ / 0.8)
  val probs = LinearAlgebra.softmaxStable(scaled)

  // Top-K sample
  val candidates = LinearAlgebra.argTopK(probs, 20)
  val total = candidates.map(_._2).sum
  val renorm = candidates.map { case (id, p) => (id, p / total) }

  // Sample
  val r = rnd.nextDouble() * total
  var cumulative = 0.0
  val nextId = renorm.find { case (_, p) => cumulative += p; cumulative >=
r }.map(_._1).getOrElse(0)

  context = context :+ nextId
  vocab.toToken(nextId)
}

println(seed.map(vocab.toToken).mkString(" ") + " " + generated.mkString(" "))
```

What You Learned

- **Argmax** (greedy decoding): always pick the highest-probability word: boring and repetitive
 - **Temperature**: divides logits before softmax: low T = confident/repetitive, high T = creative/random
 - **Top-K**: only sample from the K most probable words: cuts the long tail
 - **Top-P (nucleus)**: sample from the smallest set of words summing to probability P: adapts to the distribution's shape
 - **Autoregressive generation**: each token is generated using the context of all previous tokens; the context window slides forward
 - Good defaults: `temperature=0.8, topK=40, topP=0.9`
-

Source Reference

- `src/main/scala/linalg/LinearAlgebra.scala` : `argTopK` , `softmaxStable`
 - `src/main/scala/app/Main.scala` : the `runChat` function with full sampling logic
-

Up Next

Chapter 18 zooms out to show the whole architecture: how the CLI launcher connects every module we've built into a complete, interactive system.

Chapter 18: The CLI: Building the Conversation Interface

For nineteen chapters, we've been building instruments. Embeddings, matrix operations, forward passes, training loops, checkpoints, samplers. Each instrument does one thing.

`app/Main.scala` is the conductor. It holds every instrument we've built and coordinates them into a complete, interactive program. This chapter is the architecture tour.

The Launcher Pattern

When you run `sbt "run"`, Scala executes `app.Main.main`. This enters a **launcher loop**:

```
@main def main(args: String*): Unit =
  while true do
    println(menu)
    val choice = readLine("> ").trim
    choice match
      case "1" | "train"      => runTrain(args)
      case "2" | "predict"   => runPredict(args)
      case "3" | "chat"      => runChat(args)
      case "4" | "benchmark" => runBenchmark(args)
      case "5" | "chunk"     => runChunk(args)
      case "6" | "gpu-info"  => runGpuInfo()
      case "7" | "metal-build" => runMetalBuild()
      case "8" | "help"      => showHelp()
      case "9" | "exit" | "quit" => sys.exit(0)
      case _ => println(s"Unknown command: $choice")
```

The `while true` loop means the program keeps running, returning to the menu, until you type `exit`. Each command runs its logic, then the loop iterates and shows the menu again.

When you run `sbt "run train ..."` with arguments, the loop skips the menu and dispatches directly to the right command based on `args`.

runTrain : Orchestrating the Training Pipeline

`runTrain` is the longest and most complex function in `Main.scala`. Here's its structure:

1. Parse arguments → `TrainConfig`, `ModelConfig`
2. Load or initialize model
 - a. If `--fresh` or no checkpoint: `initParams(cfg, seed)`
 - b. If checkpoint exists: `CheckpointIO.load(...)`
3. Load and tokenize corpus
 - a. `TextPipeline.tokenize(text)`
 - b. `TextPipeline.buildVocab(tokens, maxVocab)`
 - c. `TextPipeline.tokensToIds(tokens, vocab)`
 - d. `TextPipeline.buildExamples(ids, contextSize)`
 - e. `TextPipeline.splitDeterministic(examples, ratio, seed)`
4. Select backend (CPU or Metal GPU)
5. Run `Trainer.train(trainSet, valSet, cfg, trainConfig)`
6. Handle interrupt signals
7. Save checkpoint, vocab, metrics

Every function you've built shows up here in sequence. The pipeline you assembled chapter by chapter becomes the training command.

runPredict : Single-Step Prediction

1. Load model + vocab
2. Parse `--context` flag into tokens
3. Tokenize and look up token IDs
4. Run `LanguageModel.forward(params, contextIds)`
5. Apply `--topK` filter
6. Print ranked predictions with probabilities

This is the `predict` command:

```
sbt 'run predict --context "the cat sat" --topK 10'
```

Output:

Top 10 predictions for context: "the cat sat"

1. 'on' 32.41%
2. 'in' 18.72%
3. 'and' 9.31%
- ...

runChat : The Multi-Turn Generation Loop

1. Load model + vocab
2. Parse sampling parameters (temperature, topK, topP, banUnk, maxTokens)
3. Enter interactive loop:
 - a. Read user input
 - b. Tokenize user input
 - c. Append to sliding context window
 - d. Generate maxTokens output tokens using sampling
 - e. Print generated text
 - f. Repeat

The sliding context window is maintained across turns:

```
var contextWindow: Vector[Int] = Vector.empty

// On each user message:
val userTokens = TextPipeline.tokensToIds(TextPipeline.tokenize(input), vocab)
contextWindow = (contextWindow ++ userTokens).takeRight(config.contextSize)

// Generate response:
val response = generateTokens(params, config, vocab, contextWindow, maxTokens, ...)
```

The `takeRight(config.contextSize)` is the sliding window: we always keep the most recent `contextSize` tokens.

runBenchmark : Measuring Throughput

The benchmark command tests how fast the model runs: examples per second:

```
sbt "run benchmark --input data/corpus/example-corpus.txt --sample 2000"
```

It samples 2000 examples, runs them through the training loop with different backends and batch sizes, and reports throughput. This is useful for comparing CPU vs GPU performance and choosing the right batch size.

`runChunk` : Splitting Large Corpora

Large text files (1+ GB) can be split into smaller chunks for streaming training:

```
sbt "run chunk --input data/corpus/large.txt --lines 2000 --yes"
```

This creates files like `data/corpus/chunks/large-part1.txt`, `large-part2.txt`, etc. The training command can then train on each chunk sequentially, which avoids loading a multi-gigabyte file into memory at once.

Argument Parsing: `CliHelpers`

`CliHelpers.scala` provides simple argument parsing:

```
object CliHelpers:
  def getArg(args: Seq[String], flag: String): Option[String] =
    args.sliding(2).collectFirst {
      case Seq(f, v) if f == flag => v
    }

  def getIntArg(args: Seq[String], flag: String, default: Int): Int =
    getArg(args, flag).map(_.toInt).getOrElse(default)

  def getDoubleArg(args: Seq[String], flag: String, default: Double): Double =
    getArg(args, flag).map(_.toDouble).getOrElse(default)
```

`args.sliding(2)` produces consecutive pairs: `[("--flag", "value"), ("--other", "val2"), ...]`. We search for the pair where the first element matches the flag and return the second element.

This is simple, not production-grade (no support for `--flag=value` syntax, no required-argument validation), but it works for our purposes.

Interrupt Handling: The SIGINT Hook

Long training runs can be interrupted with Ctrl+C. The training loop installs a signal handler that catches the interrupt:

```
val cancelRequested = new AtomicBoolean(false)
val handler = new SignalHandler:
  override def handle(sig: Signal): Unit =
    cancelRequested.set(true)
    display.onCancellationRequested(currentEpoch.get())
Signal.handle(new Signal("INT"), handler)
```

When Ctrl+C is pressed, `cancelRequested` is set to `true`. The training loop checks this flag between batches:

```
if cancelRequested.get() then
  // break out of training loop
  interrupted = true
```

After training exits (either normally or via interrupt), the program asks:

```
Interrupted. Choose action [b=save best, c=save current, d=discard]:
```

This gives you control: save the best checkpoint seen during training, save the latest checkpoint, or discard everything.

This is a quality-of-life feature that prevents losing hours of training progress to an accidental Ctrl+C.

Presets: Opinionated Defaults

The training command supports preset configurations:

```
--preset quick:      5 epochs, patience=3,  hidden=32,  embed=16, lr=0.05
--preset balanced: 20 epochs, patience=5,  hidden=64,  embed=24, lr=0.02
--preset thorough: 50 epochs, patience=10, hidden=128, embed=48, lr=0.01
```

These are hard-coded in `Main.scala` as sensible starting points. A beginner can run:

```
sbt "run train --input data/corpus/example-corpus.txt --preset balanced --yes"
```

...and get a decent model without understanding all the hyperparameters.

How All the Modules Connect

Here's a module dependency map for the full system:

```
app.Main
├─ data.TextPipeline      (tokenize, buildVocab, buildExamples, split)
├─ data.VocabIO           (load, save vocab)
├─ nn.LanguageModel       (initParams, forward, backward, update, trainStep)
├─ train.Trainer          (train)
├─ train.CheckpointIO     (save, load)
├─ train.ReplayBuffer     (continual learning: Extension B)
├─ eval.Metrics           (perplexity, meanLoss)
├─ compute.BackendSelector (cpu vs gpu backend selection)
├─ observability.*        (metrics, profiling: Extension C)
└─ app.CliHelpers         (argument parsing)
```

Every module we've built fits into this map. `Main.scala` is the only file that imports from multiple packages simultaneously: all other modules have narrow, focused responsibilities.

Chapter Milestone

Let's use the interactive launcher to run all four main commands in sequence.

```
sbt "run"
```

From the menu: 1. Type `train` and follow the prompts to train a model on the example corpus 2. Type `predict` and enter a context 3. Type `chat` and have a short conversation 4. Type `benchmark` and observe throughput numbers 5. Type `exit`

Then try the non-interactive versions:

```
# Train
sbt "run train --input data/corpus/example-corpus.txt --preset quick --yes"

# Predict
sbt 'run predict --context "the quick brown" --topK 5'

# Chat with specific parameters
sbt 'run chat --temperature 0.8 --topK 40 --maxTokens 50 --banUnk true'
```

Each command loads the model from `data/models/latest.ckpt` and `data/models/latest.vocab`. The train command saves there. They share a convention, not explicit coordination.

What You Learned

- `app.Main` implements a launcher loop that routes to command handlers
- `runTrain` orchestrates the entire training pipeline: parse → load/init → tokenize → train → save
- `runPredict` runs a single forward pass and displays top-K predictions
- `runChat` implements multi-turn generation with a sliding context window
- Argument parsing uses `sliding(2)` to match `--flag value` pairs
- Interrupt handling (`SIGINT`) lets training stop gracefully with a save prompt
- Presets provide opinionated defaults for common training scenarios

- Every module has a narrow responsibility; `Main` is the only coordinator
-

Source Reference

- `src/main/scala/app/Main.scala` : the launcher and all command handlers
 - `src/main/scala/app/CliHelpers.scala` : argument parsing utilities
-

Up Next

One more chapter. Chapter 19 is the payoff: training a model on a real corpus and having your first real conversation.

Chapter 19: The Payoff: Run Chat with Your Own Language Model

You've built everything.

You have a tokenizer that turns text into numbers. You have a vocabulary that maps words to IDs and back. You have a neural network with an embedding layer, a hidden layer, and an output layer. You have a training loop with mini-batches, early stopping, and checkpointing. You have a sampling system with temperature and top-K and top-P. You have a CLI that ties it all together.

Now it's time to use it.

Step 1: Get a Corpus

The model can only learn from data you give it. The included `data/corpus/example-corpus.txt` is tiny: useful for testing, not for generating interesting text.

For a better experience, use a real corpus. Some good options:

TinyStories (recommended for beginners): - Short, simple sentences written for children - Download: `tinystories_train.txt` from HuggingFace datasets - Small enough to train in 20–30 minutes on a laptop - Produces surprisingly readable output

News articles, Wikipedia excerpts, or any plain text you find interesting. The model will try to match the style of its training corpus.

Your own writing: a collection of documents, notes, or stories. The model will pick up your vocabulary and style.

For now, you can proceed with the example corpus and get small but real results.

Step 2: Choose a Training Preset

The three presets balance speed vs quality:

PRESET	EPOCHS	MODEL SIZE	TRAINING TIME	EXPECTED PERPLEXITY
<code>quick</code>	5	Small (hidden=32)	~1 min	High (90–200)
<code>balanced</code>	20	Medium (hidden=64)	~5 min	Medium (40–90)
<code>thorough</code>	50	Larger (hidden=128)	~15 min	Lower (20–60)

Times vary with corpus size and your hardware. Start with `balanced`.

Step 3: Train

```
sbt "run train --input data/corpus/example-corpus.txt --preset balanced --yes"
```

You'll see: - Vocabulary building (how many unique words, how many examples) - The epoch table updating live (train loss, val loss, perplexity, status) - Early stopping or completion - A summary: best validation perplexity

Write down the final validation perplexity. This is your model's quality score.

On a larger corpus (TinyStories or similar):

```
sbt "run train --input data/corpus/tinystories_train.txt --preset balanced --yes --maxVocab 3000 --contextSize 5"
```

With a larger corpus and bigger context window, you'll get much better results.

Step 4: Test Predictions

Before chatting, look at raw predictions:

```
sbt 'run predict --context "once upon a" --topK 10'
```

On a model trained with TinyStories, you'd expect to see words like "time", "there", "day" near the top: because "once upon a" almost always precedes those words in children's stories.

Try a few different contexts:

```
sbt 'run predict --context "the little girl" --topK 5'  
sbt 'run predict --context "she said" --topK 5'
```

The predictions reveal what the model has learned. Sensible, contextually appropriate predictions mean real learning happened.

Step 5: Chat

```
sbt 'run chat --temperature 0.8 --topK 40 --topP 0.9 --banUnk true --maxTokens 50'
```

Type something. The model responds with up to 50 tokens.

Some prompts that tend to work well with TinyStories-trained models: - `once upon a time` - `the little boy` - `she looked at`

With the small example corpus, the responses will be less coherent: but they'll still show learned patterns from the training text.

What to Expect

Let's be honest: your model won't pass the Turing test.

Small models trained on limited data have real limitations: - Short memory (only `contextSize` tokens, typically 3–5) - Limited vocabulary - Tendency to repeat common phrases - No understanding of meaning: it's pattern matching

But the patterns are real. The model has genuinely learned something from the data. Words that commonly follow each other in the training text will appear together in the generated text. The style of the training corpus will be reflected in the output.

This is remarkable, given what's happening under the hood: tens of thousands of numbers being optimized by gradient descent until they capture statistical regularities in natural language. You didn't program those regularities in. The math found them.

Why GPT-4 Is Better (And Why That's Okay)

GPT-4 is better at this than your model. A lot better. Here's why:

	YOUR MODEL	GPT-4
Parameters	~50K	~1.8T (est.)
Training data	One text file	Most of the internet
Context window	3–10 tokens	128K tokens
Architecture	2-layer MLP	Deep transformer
Training compute	Your laptop, minutes	Thousands of GPUs, months
Cost	Free	\$100M+

The differences are *quantitative*, not qualitative. Your model uses the same core ideas: - Word embeddings - Linear transformations - Non-linear activations - Cross-entropy loss - Gradient descent

GPT-4 is a transformer, which is a different architecture from our MLP: but transformers also use embeddings, linear layers, and gradient descent. The concepts you've learned transfer directly. If you went on to study transformers, you'd find most of the machinery already familiar.

What You Actually Built

Let's list everything you implemented from scratch:

- ✓ A tokenizer that normalizes and splits text into tokens
- ✓ A vocabulary builder that handles frequency-based selection and unknown tokens
- ✓ A sliding-window example generator
- ✓ A train/validation split with reproducible shuffling
- ✓ Dense vector (Vec) and matrix types with row-major storage
- ✓ Linear algebra: dot product, matrix-vector multiply, outer product, matrix multiply
- ✓ An embedding lookup table
- ✓ A linear layer with bias

- ✓ Tanh and ReLU activation functions
- ✓ Numerically stable softmax
- ✓ Cross-entropy loss
- ✓ Backpropagation through softmax + linear + activation + embeddings
- ✓ SGD with L2 regularization and gradient clipping
- ✓ Mini-batch training with averaged gradients
- ✓ Epoch loop with early stopping and learning rate decay
- ✓ Plain-text checkpoint serialization/deserialization
- ✓ Vocabulary persistence
- ✓ Temperature, top-K, and top-P sampling
- ✓ Autoregressive generation with a sliding context window
- ✓ A full CLI with train, predict, chat, and benchmark commands

That's a complete machine learning system. Built from integer arithmetic and the chain rule.

Where to Go Next

You have a working foundation. Here are natural next steps:

Improve the model: - Train on more data: the TinyStories dataset is a good next step - Increase `contextSize` to capture longer-range dependencies - Try a larger `hiddenDim` (128 or 256) - Experiment with learning rate schedules

Extend the code: - Add a second hidden layer (a deeper MLP) - Implement attention: the mechanism that makes transformers powerful - Try character-level tokenization instead of word-level - Add top-P renormalization and beam search for generation

Understand the theory: - Read the original backpropagation paper (Rumelhart, Hinton, Williams, 1986) - Work through the "Attention Is All You Need" transformer paper - Explore the "Neural Probabilistic Language Model" paper (Bengio et al., 2003): it introduced the architecture you just built

The optional extension chapters: - Extension A: Speed up training with Apple Metal GPU acceleration - Extension B: Train across multiple corpora without forgetting with replay buffers - Extension C: Observe and profile your model's performance

Your Final Milestone

If you haven't already:

```
sbt 'run chat --temperature 0.8 --topK 40 --topP 0.9 --banUnk true --maxTokens 50'
```

Type something. Read the response. It's not perfect, but it's yours.

You built the neural network that generated those words. You wrote the training loop that optimized those weights. You implemented the sampling algorithm that chose those tokens.

The mathematical machinery that, at a much larger scale, powers the most sophisticated language technology ever built: you understand it now. Not as a black box, not as a metaphor, but from the ground up.

That's the payoff.

Epilogue: What Language Really Is

Building a language model forces you to confront a strange fact: **language, at a statistical level, is very predictable.**

Given "the cat sat on the", most humans would guess "mat" (or "floor", or "chair"). The model learns this. Given "once upon a", almost everyone completes with "time". The model learns this too.

Does this mean language is *just* statistics? No. Meaning, intention, creativity, humor: none of these are captured by next-word prediction. The model has no idea what words mean. It only knows which words tend to follow which other words, and it's learned to imitate that pattern.

But it's deeply interesting that so much of what makes language feel coherent can be approximated by a relatively simple statistical model. The model we built is, at its core, a very compact summary of the statistical regularities in its training data. And that summary, when you sample from it, produces text that looks surprisingly like language.

Language models don't understand language. But they've taught us something about what understanding might be built on top of.

That's worth sitting with.

End of the main text. Optional extension chapters follow.

Extension A: Speed Up Training with Apple Metal

This chapter is optional. It requires an Apple Silicon Mac (M1 or later) and the compiled Metal JNI library.

Training a language model is arithmetic (millions of multiplications and additions per second. CPUs are good at sequential arithmetic. GPUs are good at *parallel* arithmetic) they can perform thousands of operations simultaneously.

This extension adds Apple Metal GPU acceleration to the training loop. On M1 hardware, GPU training typically runs 2–4× faster than CPU training for matrix-heavy workloads.

The Backend Abstraction

Every compute-intensive operation in the forward and backward passes goes through a

`ComputeBackend` :

```
trait ComputeBackend:
  def matVecMul(m: Matrix, v: Vec): Vec
  def matMul(a: Matrix, b: Matrix): Matrix
  def linearActivation(W: Matrix, x: Vec, b: Vec, activation: String): (Vec, Vec)
  def softmaxStable(logits: Vec): Vec
  def crossEntropy(probs: Vec, target: Int): Double
  def outer(a: Vec, b: Vec): Matrix
  def hadamard(a: Vec, b: Vec): Vec
  def vecAdd(a: Vec, b: Vec): Vec
  def tanhGrad(v: Vec): Vec
  def reluGrad(v: Vec): Vec
  def addRowBias(m: Matrix, bias: Vec): Matrix
  def reduceSumRows(m: Matrix): Vec
  def softmaxStableBatch(logits: Matrix): Matrix
  def crossEntropyBatch(probs: Matrix, targets: Vector[Int]): Vector[Double]
```

This trait is the contract: any backend (CPU or GPU) must provide these operations. The neural network code calls operations through the backend; it doesn't know or care whether the computation runs on CPU or GPU.

Source: `src/main/scala/compute/ComputeBackend.scala`

The CPU Backend

`CpuBackend` implements `ComputeBackend` using pure Scala:

```
object CpuBackend extends ComputeBackend:
  val Default = new CpuBackend

class CpuBackend extends ComputeBackend:
  def matVecMul(m: Matrix, v: Vec): Vec = LinearAlgebra.matVecMul(m, v)
  def matMul(a: Matrix, b: Matrix): Matrix = LinearAlgebra.matMul(a, b)
  // ...etc
```

Every operation delegates to the `LinearAlgebra` functions we built in Chapters 4–5.

Source: `src/main/scala/compute/CpuBackend.scala`

The Metal Backend

`MetalBackend` sends compute-intensive operations to the Apple Metal GPU:

```
class MetalBackend(bridge: MetalNativeBridge) extends ComputeBackend:
  def matVecMul(m: Matrix, v: Vec): Vec =
    bridge.matVecMul(m.data.toArray, m.rows, m.cols, v.toArray)
      .map(identity)
      .toVector
  // ...etc
```

Instead of running Scala loops, it passes the data to the GPU bridge, waits for the result, and converts back.

Source: `src/main/scala/compute/MetalBackend.scala`

JNI: Calling Native Code from the JVM

The JVM (Java Virtual Machine) can call native code via **JNI**: the Java Native Interface. JNI lets you write performance-critical operations in C, C++, or Objective-C and call them from Scala or Java.

The bridge:

```
Scala (MetalNativeBridge.scala)
  ↓ JNI call
Objective-C (MetalBridge.m)
  ↓ Metal framework
Apple GPU
```

`MetalNativeBridge.scala` declares the JNI methods:


```
class MetalNativeBridge:
  @native def matVecMul(data: Array[Double], rows: Int, cols: Int, vec:
Array[Double]): Array[Double]
  @native def matMul(aData: Array[Double], aRows: Int, aCols: Int, bData:
Array[Double], bRows: Int, bCols: Int): Array[Double]
  @native def softmaxRowwise(data: Array[Double], rows: Int, cols: Int): Array[Double]
  // ...etc
```

The `@native` annotation tells the JVM: this method is implemented in native (non-JVM) code. When called, the JVM looks up the corresponding function in the loaded native library.

Source: `src/main/scala/compute/MetalNativeBridge.scala`

The Objective-C Bridge

`MetalBridge.m` is the native implementation. It: 1. Initializes a Metal device (the GPU) 2. Compiles Metal Shading Language kernels (GPU programs) 3. Allocates GPU memory buffers 4. Dispatches computation to the GPU 5. Reads results back to CPU memory 6. Returns to the JVM

A Metal kernel for matrix-vector multiplication:

```
kernel void matVecMulKernel(
  device const float* mat    [[ buffer(0) ]],
  device const float* vec    [[ buffer(1) ]],
  device float*          result [[ buffer(2) ]],
  constant uint2&        dims  [[ buffer(3) ]],
  uint                   row    [[ thread_position_in_grid ]])
{
  if (row >= dims.x) return;
  float sum = 0.0;
  for (uint c = 0; c < dims.y; c++)
    sum += mat[row * dims.y + c] * vec[c];
  result[row] = sum;
}
```

Each "thread" on the GPU computes one output element. With a GPU that has thousands of cores, all rows are computed simultaneously.

Note: the GPU uses `float` (32-bit) arithmetic. JVM `double` (64-bit) values are cast to `float` before dispatch. This is the "fp32" designation in the benchmark output. Slightly less precision, significantly faster.

Source: `metal-jni/src/main/objectivec/MetalBridge.m`

Backend Selection

`BackendSelector` decides which backend to use based on the configuration and what's available:

```
object BackendSelector:
  def fromConfig(backendName: String): (ComputeBackend, String, String) =
    backendName.toLowerCase match
      case "gpu" | "metal" =>
        try
          val bridge = MetalNativeBridge.load()
          val backend = MetalBackend(bridge)
          (backend, "gpu", "Metal fp32")
        catch
          case _: Exception =>
            (CpuBackend.Default, "cpu", "Metal unavailable, using CPU")
      case _ =>
        (CpuBackend.Default, "cpu", "CPU fp64")
```

If Metal fails to load (no Apple Silicon, library not compiled, etc.), it silently falls back to CPU. You get the best available hardware automatically.

Source: `src/main/scala/compute/BackendSelector.scala`

Building the Metal JNI Library

The native library must be compiled before use. From the project root:

```
bash metal-jni/scripts/build-metal-jni.sh
```

This script runs:

```
clang -fobjc-arc \  
  -framework Foundation \  
  -framework Metal \  
  -framework MetalKit \  
  -shared -fPIC \  
  -I "$JAVA_HOME/include" \  
  -I "$JAVA_HOME/include/darwin" \  
  metal-jni/src/main/objectivec/MetalBridge.m \  
  -o metal-jni/build/libmetal_jni.dylib
```

After successful compilation, `libmetal_jni.dylib` appears in `metal-jni/build/`. The JVM loads it at startup via the system property `-Dmetal.jni.lib=...` in `build.sbt`.

Checking GPU Status

```
sbt "run gpu-info"
```

Output:

```
Metal device: Apple M1  
Memory: 16.00 GB  
Supported: true  
JNI library: metal-jni/build/libmetal_jni.dylib (loaded)
```

Or, if Metal isn't available:

```
Metal device: not found  
JNI library: not found at expected path  
Fallback: using CPU backend
```

Training with GPU

```
sbt "run train --input data/corpus/tinystories_train.txt --preset thorough --yes --  
backend gpu"
```

The `--backend gpu` flag selects the Metal backend. The benchmark output will show `GPU (Metal fp32)` in the summary.

Compare against CPU:

```
sbt "run benchmark --input data/corpus/example-corpus.txt --sample 2000"
```

The benchmark matrix tests both backends and reports examples/second for each. On M1, GPU training is typically 2–4× faster for the matrix-heavy operations (matMul, softmax, cross-entropy).

The Fallback Strategy

Each GPU operation has a fallback. If a particular kernel fails to execute on the GPU (due to memory pressure, unsupported operation, etc.), the backend transparently falls back to the CPU equivalent for that operation. Training continues without interruption.

This means GPU acceleration is entirely opt-in and failure-safe. If anything goes wrong with the Metal setup, you're on CPU.

What This Extension Taught You

- The `ComputeBackend` trait is an interface that abstracts computation from its location
- JNI bridges Scala/Java to native code (C, Objective-C, C++)
- Metal kernels are GPU programs that run massively parallel
- `BackendSelector` chooses the best available backend with graceful fallback

- GPU training is faster (2–4× on M1) for matrix-heavy workloads, at the cost of fp32 precision
 - The JNI library must be compiled separately; `build-metal-jni.sh` handles this
-

Source Reference

- `src/main/scala/compute/ComputeBackend.scala` : the backend trait
 - `src/main/scala/compute/CpuBackend.scala` : CPU implementation
 - `src/main/scala/compute/MetalBackend.scala` : Metal GPU backend
 - `src/main/scala/compute/MetalNativeBridge.scala` : JNI declarations
 - `src/main/scala/compute/BackendSelector.scala` : backend selection and fallback
 - `metal-jni/src/main/objectivec/MetalBridge.m` : native Objective-C/Metal implementation
 - `metal-jni/scripts/build-metal-jni.sh` : compilation script
 - `metal-jni/README.md` : setup instructions
-

Extension B: Teaching Across Multiple Corpora: Replay Buffers and Continual Learning

This chapter is optional. It covers advanced training scenarios for models trained on multiple datasets.

The Catastrophic Forgetting Problem

Suppose you train your model on a corpus of news articles. It learns journalistic language patterns well: vocabulary, sentence structure, common topics.

Now you want to improve it by training on children's stories. You run another training session on the new data.

After training on children's stories, you test the model on news-style text. It performs terribly: as if it never saw the news corpus.

This is **catastrophic forgetting**: when a neural network learns a new task, it overwrites the weights that encoded the old task. The new data's gradients push the weights in a new direction, erasing what was learned before.

For a model trained on a single corpus, this isn't an issue. But if you want to train incrementally (adding new data over time without losing old knowledge) catastrophic forgetting is a fundamental challenge.

Solution 1: The Replay Buffer

The simplest solution: when training on new data, occasionally also train on examples from the old data.

A **replay buffer** is a reservoir of examples from previous training phases. During new-data training, a fraction of each mini-batch comes from the replay buffer instead of the new corpus.

```
batch = [  
    example from new corpus,    ← 80% of batch (replayRatio=0.2)  
    example from new corpus,  
    example from new corpus,  
    example from new corpus,  
    example from replay buffer, ← 20% of batch  
    example from new corpus,  
    ...  
]
```

By mixing old examples into training, the model maintains its performance on old data while learning new patterns.

The trade-off: you need to store old examples. And the replay buffer can't hold everything: it has a fixed capacity. Once full, new examples replace old ones (FIFO).

The `ReplayBuffer` Class

```
final case class ReplayBuffer(  
  capacity: Int,  
  entries: Vector[(Example, String)], // (example, domain_label)  
  pointer: Int  
)
```

- `capacity`: maximum number of examples to store
- `entries`: the stored examples with their domain labels ("news", "stories", etc.)
- `pointer`: the write position for FIFO eviction

Key operations:

```
object ReplayBuffer:  
  def empty(capacity: Int): ReplayBuffer = ReplayBuffer(capacity, Vector.empty, 0)  
  
  extension (buf: ReplayBuffer)  
    def add(examples: Vector[Example], domain: String): ReplayBuffer =  
      // Add examples to buffer, evicting oldest if at capacity  
      ...  
  
    def sample(n: Int, rnd: Random): Vector[(Example, String)] =  
      // Sample n random examples from the buffer  
      ...
```

Source: `src/main/scala/train/ReplayBuffer.scala`

Phased Training

Training across multiple corpora uses `trainPhased` :

```
case class TrainingPhase(
  label: String,
  trainSet: Vector[Example],
  valSet: Vector[Example],
  weight: Double
)

def trainPhased(
  phases: Vector[TrainingPhase],
  config: ModelConfig,
  trainConfig: TrainConfig,
  replay: ReplayBuffer
): TrainResult
```

Each phase trains on one corpus. Between phases, the replay buffer accumulates examples from the completed phase. During subsequent phases, `replayRatio` of each batch comes from the replay buffer.

Retention Metrics

How do you know if catastrophic forgetting is happening?

After each training phase, `trainPhased` evaluates the model on the *original* validation sets from all previous phases. If the model's performance on an old domain degrades, forgetting is occurring.

```
RetentionMetrics:
  domain:          "news"
  baselineValLoss: 3.2141 ← loss on news corpus after phase 1
  currentValLoss:  3.5812 ← loss on news corpus after phase 2
  retentionPct:    90.1%  ← (1 - degradation) × 100
  status:         "OK"   ← "OK" if retentionPct > threshold
```

If `retentionPct` drops below your tolerance (say, 85%), you're experiencing meaningful forgetting. Increase `replayRatio` or `replayBufferSize` to retain more old knowledge.

Solution 2: Elastic Weight Consolidation (EWC)

Replay buffers work by retraining on old data. EWC works differently: it penalizes the model for changing weights that were important for old tasks.

The idea: after training on corpus A, compute which weights were most important (using the Fisher information matrix: a measure of how much the loss changes when each weight changes). When training on corpus B, add a regularization term that penalizes large changes to those important weights.

$$\text{total_loss} = \text{new_corpus_loss} + \lambda \times \sum F_i \times (w_i - w_i^*)^2$$

- λ (ewcLambda): the strength of the penalty
- F_i : the Fisher information for weight i : how important it was for the old task
- w_i : the current weight
- w_i^* : the optimal weight from the old task

Weights that were important (F_i large) are penalized heavily for changing. Weights that were unimportant can change freely.

To enable EWC:

```
sbt "run train --input new_corpus.txt --ewcLambda 1000 --ewcSamples 500"
```

EWC is more computationally expensive than replay (computing Fisher information requires extra forward passes) but can preserve old knowledge more precisely for small models.

Mixed Validation Loss

When training across multiple corpora, the validation loss is a weighted combination of each corpus's validation loss:

$$\text{mixed_val_loss} = \sum \text{weight}_i \times \text{val_loss}_i$$

The weights reflect how much you care about each corpus. If news articles are your primary use case and children's stories are supplemental, you might weight news at 0.7 and stories at 0.3.

The `mixedValWeights` parameter in `TrainConfig` controls this.

Using the Replay Buffer: End-to-End

Here's how to train on two corpora with replay:

```
# Phase 1: Train on news corpus
sbt "run train --input data/corpus/news.txt --preset balanced --yes"
# Model and replay buffer saved to data/models/latest.*

# Phase 2: Train on stories corpus, replaying news examples
sbt "run train --input data/corpus/stories.txt --preset balanced --yes \
    --replayRatio 0.2 --replayBufferSize 5000"
```

The second command loads the replay buffer from `data/models/latest.replay`, adds news examples from the buffer to each batch (20% of each batch), and saves an updated buffer.

After phase 2, check retention:

```
sbt "run predict --context 'the president announced'"
# Should still produce news-style predictions
sbt "run predict --context 'once upon a time'"
# Should produce story-style predictions
```

If both work, catastrophic forgetting was avoided.

What This Extension Taught You

- **Catastrophic forgetting:** training on new data overwrites learned weights for old data
- **Replay buffer:** stores old examples and mixes them into new training batches

- `ReplayBuffer` is capacity-bounded FIFO with domain labels for retention tracking
 - **Phased training:** `trainPhased` coordinates multi-corpus training with retention evaluation
 - **EWC** penalizes weight changes based on their importance to previous tasks (Fisher information)
 - **Mixed validation loss** combines per-domain losses with configurable weights
 - Retention metrics measure how much the model has forgotten after each phase
-

Source Reference

- `src/main/scala/train/ReplayBuffer.scala` : buffer implementation
 - `src/main/scala/train/Trainer.scala` : `trainPhased` , `RetentionMetrics` , EWC logic
 - `src/main/scala/train/CheckpointIO.scala` : replay buffer serialization (saved alongside model)
-

Extension C: Observability: Measuring What Your Model Really Does

This chapter is optional. It covers the metrics and profiling infrastructure built into the project.

What Is Observability?

In software engineering, **observability** is the ability to understand the internal state of a system from its external outputs: without adding extra instrumentation every time you have a new question.

For a language model training system, observability means: when something goes wrong (training is slow, loss isn't converging, memory is growing), you have enough data to understand why, without having to modify the code and retrain.

The project collects extensive metrics on every training and benchmark run, writing them to `data/metrics/runs.jsonl`.

RunObservability : The Metrics Collector

`RunObservability` is the central metrics collection object. It captures:

Platform metadata:

- OS name, version, architecture
- Java version and vendor
- Device name ("Apple M1")

Backend info:

- Requested backend (cpu/gpu)
- Effective backend (after fallback)
- Enabled GPU operations
- Backend diagnostic messages

Training metrics:

- Run ID (unique timestamp + random suffix)
- Final train loss, val loss, perplexity
- Generalization gap
- Examples per second
- Total training time

Memory snapshots:

- JVM heap used (start, end, peak)
- JVM non-heap (start, end, peak)
- RSS (Resident Set Size: actual RAM used by the process)

GC metrics:

- Garbage collection count delta (start to end)
- GC time delta (milliseconds)

Operation profiling:

- Time spent in each operation type (matMul, softmaxBatch, ceBatch, etc.)
- As percentage of total profiled time

Source: `src/main/scala/observability/RunObservability.scala`

The `runs.jsonl` Metrics Log

Each training or benchmark run appends one JSON line to `data/metrics/runs.jsonl` :

```
{
  "runId": "train-2026-05-09T17-35-55.591214Z-980a189f",
  "type": "train",
  "timestamp": "2026-05-09T17:35:55.591Z",
  "platform": { "os": "Mac OS X", "arch": "aarch64", "device": "Apple M1" },
  "backend": { "requested": "gpu", "effective": "gpu", "precision": "fp32" },
  "model": { "vocabSize": 18384, "contextSize": 5, "embedDim": 48, "hiddenDim": 128 },
  "training": {
    "epochs": 13, "finalValLoss": 4.3017, "finalValPerplexity": 73.82,
    "throughputExPerSec": 1529.24, "totalSeconds": 3171.92
  },
  "memory": {
    "heapStartBytes": 524288, "heapEndBytes": 52428800, "heapPeakBytes": 104857600,
    "rssStartBytes": 1073741824, "rssEndBytes": 4831838208
  },
  "gcDelta": { "count": 12, "timeMs": 847 },
  "profile": {
    "matMul": 0.545, "softmaxBatch": 0.382, "ceBatch": 0.058, "other": 0.015
  }
}
```

This file accumulates across runs. You can analyze it to track how changes affect performance over time.

Memory Probes

`MemoryProbe.snapshot()` captures the current JVM memory state:

```
object MemoryProbe:
  case class Snapshot(
    heapUsedBytes: Long,
    heapMaxBytes: Long,
    nonHeapUsedBytes: Long,
    rssBytes: Long
  )

  def snapshot(): Snapshot =
    val bean = ManagementFactory.getMemoryMXBean
    val heap = bean.getHeapMemoryUsage
    val nonHeap = bean.getNonHeapMemoryUsage
    val rss = readRssFromProc() // /proc/self/status on Linux, ps on Mac
    Snapshot(heap.getUsed, heap.getMax, nonHeap.getUsed, rss)
```

RSS (Resident Set Size) is the actual amount of RAM the process is using, including native memory. The JVM heap is what Java code allocates; RSS includes everything: heap, non-heap, native libraries.

Why does this matter? If RSS grows faster than the heap, native memory might be leaking: a potential issue with JNI code.

GC Probes

`GcProbe.snapshot()` records garbage collection statistics:

```
object GcProbe:
  case class Snapshot(totalCount: Long, totalTimeMs: Long)

  def snapshot(): Snapshot =
    ManagementFactory.getGarbageCollectorMXBeans.asScala.foldLeft(Snapshot(0, 0))
    { (acc, gc) =>
      Snapshot(acc.totalCount + gc.getCollectionCount, acc.totalTimeMs +
        gc.getCollectionTime)
    }
```

By taking snapshots before and after training and computing the delta, we know exactly how much GC pressure the training loop produces. High GC counts suggest excessive allocation: a candidate for optimization.

Operation Profiling

The backend can be instrumented to record how much time each operation type consumes:

```
class ProfiledMetalBackend(base: MetalBackend) extends ComputeBackend:
  private val timings = mutable.Map[String, Long]().withDefaultValue(0L)

  def matMul(a: Matrix, b: Matrix): Matrix =
    val start = System.nanoTime()
    val result = base.matMul(a, b)
    timings("matMul") += System.nanoTime() - start
    result

  def softmaxStableBatch(logits: Matrix): Matrix =
    val start = System.nanoTime()
    val result = base.softmaxStableBatch(logits)
    timings("softmaxBatch") += System.nanoTime() - start
    result
```

After training, the accumulated timings are normalized to percentages:

```
matMul:      54.5%  ← most time in matrix multiplication
softmaxBatch: 38.2%  ← softmax is expensive
ceBatch:     5.8%   ← cross-entropy
other:       1.5%
```

This tells you where to focus optimization effort. If `matMul` dominates, a faster matrix multiply (GPU, BLAS library) would have the most impact.

The Benchmark Command

The benchmark measures pure throughput across different configurations:

```
sbt "run benchmark --input data/corpus/example-corpus.txt --sample 2000"
```

It runs a training loop on 2000 randomly sampled examples with: - Multiple batch sizes (1, 8, 32, 128) - CPU and GPU backends (if available)

And reports a matrix:

Backend	Batch	Examples/sec	Avg ms/batch	P95 ms/batch
-----	-----	-----	-----	-----
CPU fp64	1	312.4	3.20	4.1
CPU fp64	8	891.2	8.98	11.2
CPU fp64	32	1243.7	25.7	29.4
GPU fp32	1	418.9	2.39	3.1
GPU fp32	8	1187.4	6.74	8.2
GPU fp32	32	1529.2	20.9	22.4

The optimal batch size for your hardware can be read directly from this table.

Regression Detection

The metrics system compares each new run against a baseline. If performance degrades by more than a threshold (5% by default), it's flagged.

```
Run ID:      train-2026-05-09T17-35-55-980a189f
Baseline:    train-2026-04-27T09-12-34-abc12345
Throughput:  1529.24 vs 1505.11 (baseline): +1.6% ← OK
Val perp:    73.82 vs 74.51 (baseline): -0.9%      ← OK
Memory RSS:  4.52 GB vs 4.48 GB: +0.9%             ← OK
```

This is useful when you make code changes: you can check whether your change accidentally slowed training down or increased memory usage.

The baseline is the run marked as "current" in `data/metrics/runs-index.tsv`. To update the baseline:

```
sbt "run train ... --saveBaseline"
```


Reading the Latest Summary

After any run, `data/metrics/latest-summary.txt` contains a human-readable summary:

```
=== Training Run Summary ===
Run ID:      train-2026-05-09T17-35-55.591214Z-980a189f
Platform:    Mac OS X 25.5.0 (aarch64): Apple M1
Java:        OpenJDK 25.0.2

Model:       vocabSize=18384, contextSize=5, embedDim=48, hiddenDim=128
Backend:     GPU (Metal fp32)
Epochs:     13 / 50 (early stopped)

Performance:
  Throughput:      1529.24 ex/s
  Total time:      3171.92s

Losses:
  Train loss:      3.8412 (perplexity: 46.48)
  Val loss:        4.3017 (perplexity: 73.82)
  Gen gap:         10.7%

Memory:
  RSS start:       1.00 GB
  RSS end:         4.52 GB
  JVM heap peak:   1.37 GB
  GC collections:  12 (847ms)

Profile (GPU):
  matMul:          54.5%
  softmaxBatch:    38.2%
  ceBatch:         5.8%
  other:           1.5%
```

This is the first thing to check after a training run. It tells you: did training converge? Was memory reasonable? Where did time go?

What This Extension Taught You

- **Observability** means having enough data to diagnose problems without modifying code

- `RunObservability` collects platform info, training metrics, memory snapshots, GC stats, and operation timings
 - `runs.jsonl` accumulates all runs for historical comparison
 - `MemoryProbe` captures JVM heap and OS-level RSS
 - `GcProbe` measures GC pressure: high GC suggests excessive allocation
 - Operation profiling identifies where time is spent: `matMul` and `softmaxBatch` typically dominate
 - The benchmark command tests throughput across batch sizes and backends
 - Regression detection compares each run to a baseline, flagging unexpected performance changes
-

Source Reference

- `src/main/scala/observability/RunObservability.scala` : the core metrics collection
- `src/main/scala/app/Main.scala` : `renderBenchmarkMetricsReport` , benchmark matrix rendering
- `src/main/scala/train/TrainingDisplay.scala` : live progress bar with P95 timing
- `data/metrics/runs.jsonl` : accumulated run history
- `data/metrics/latest-summary.txt` : human-readable latest run summary